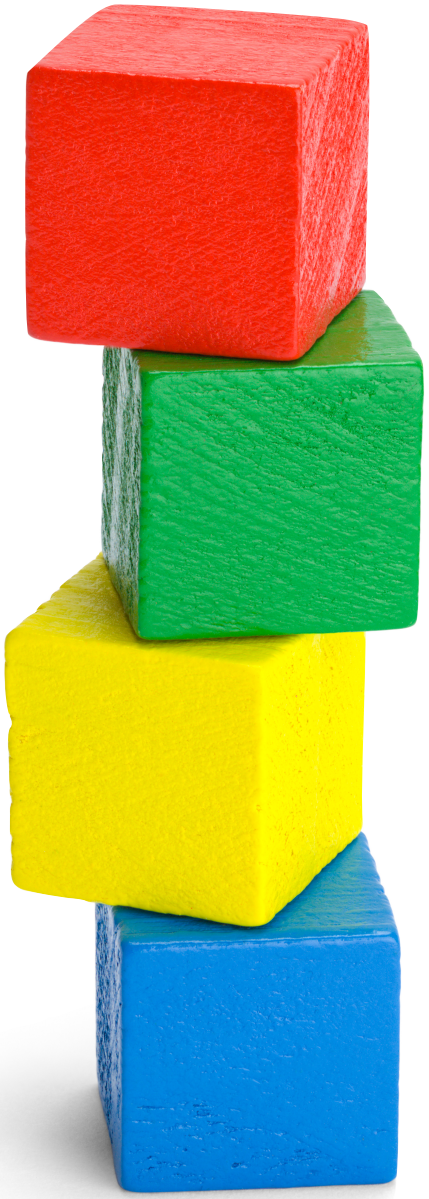
ODSL Block Course

Lecture 8: Convolutional Neural Networks

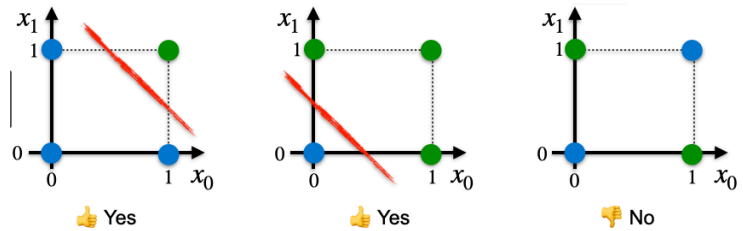
Nicole Hartman
nicole.hartman@tum.de

ODSL Block Course
24 Mar 2026



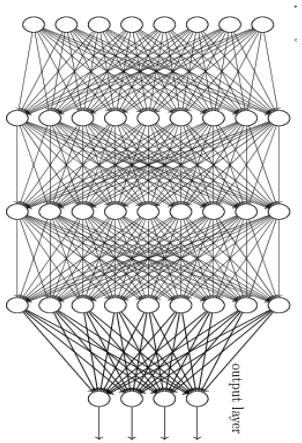
Recap from last time

1. **Perecpton: linear decision boundaries**



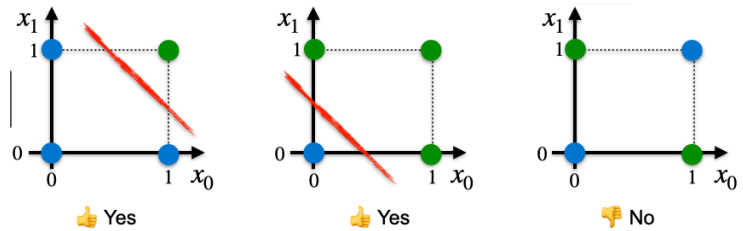
2. **Multi-layer perceptron: aggregate these linear decisions**
   ↪ Universal Approx

3. **But deeper offers a more compact way to build up complexity!**



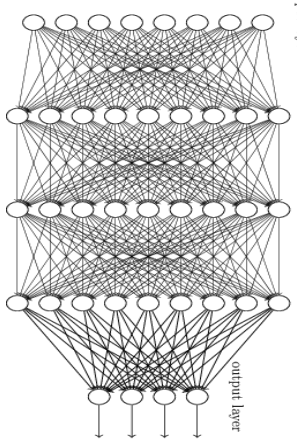
Recap from last time

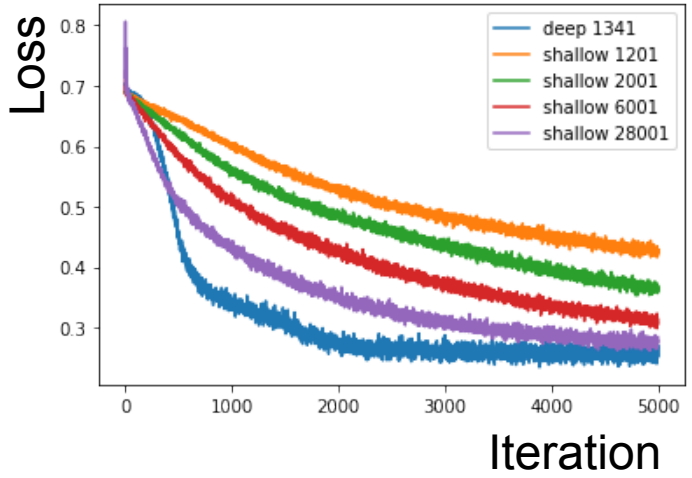
1. **Perecpton: linear decision boundaries**



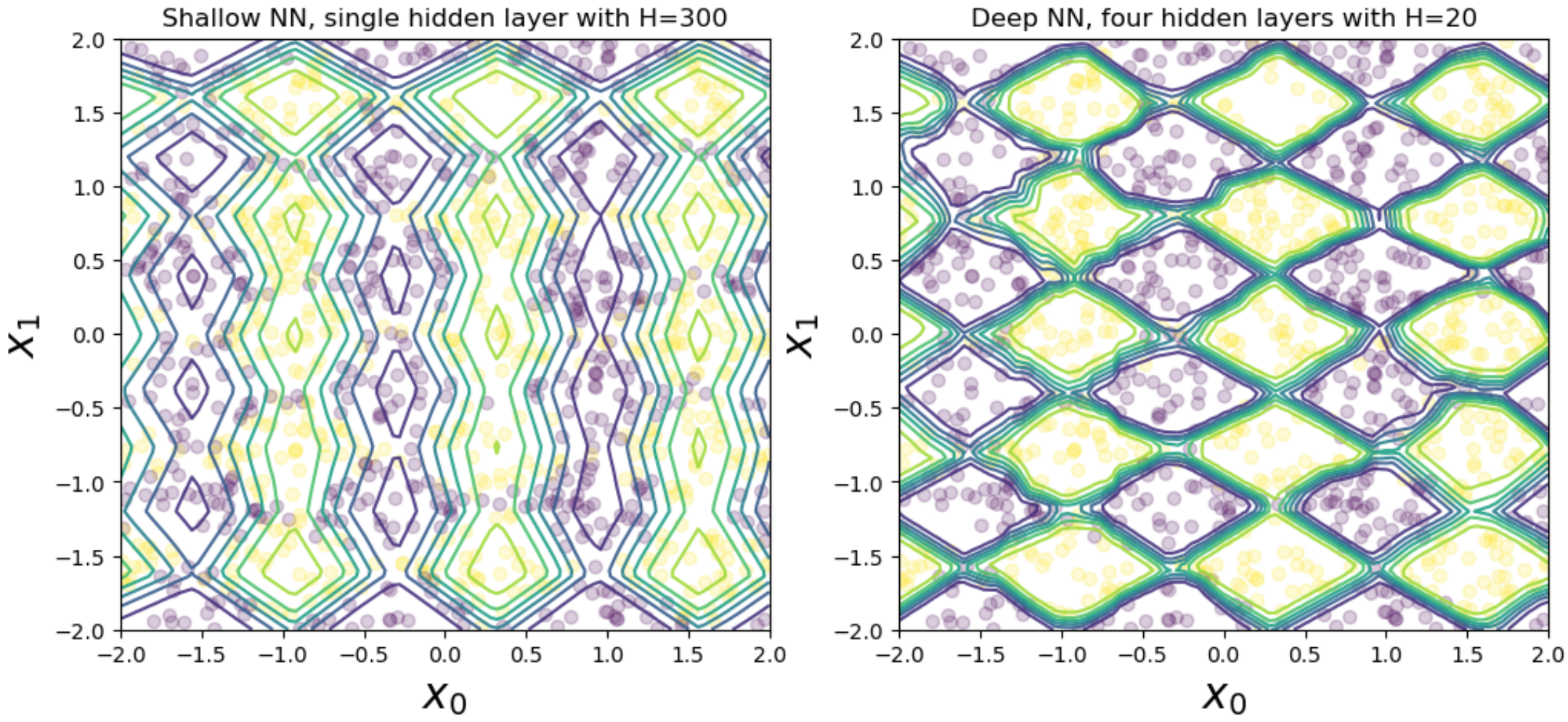
2. **Multi-layer perceptron: aggregate these linear decisions**
   ↪ Universal Approx

3. **But deeper offers a more compact way to build up complexity!**





Tutorial


Shallow NN, single hidden layer with H=300
Deep NN, four hidden layers with H=20
x_0
x_1
2.0
1.5
1.0
0.5
0.0
−0.5
−1.0
−1.5
−2.0

Our dreams for AI

Or my memory of them… circa 2017

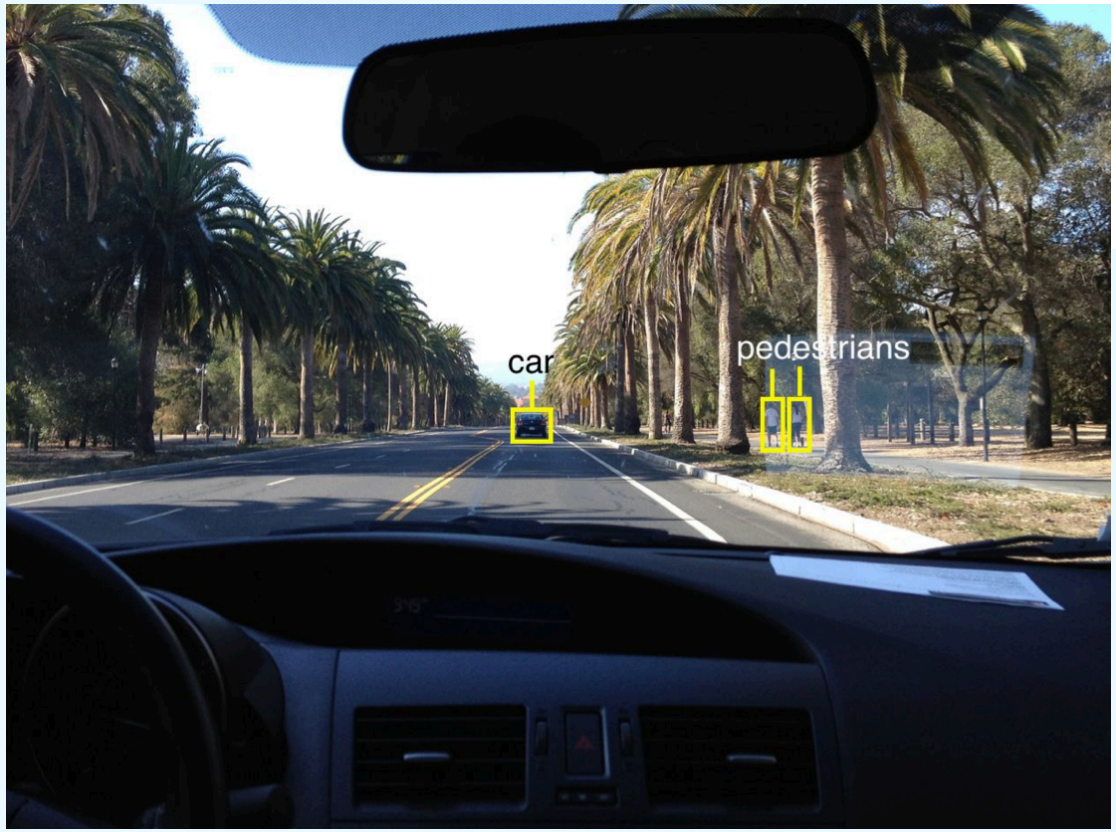


Photo by Lane McIntosh. Copyright CS231n 2017.

A. Karpathy & J. Johnson, ref

Today… a dream come true?

Video credit: Aishik Ghosh

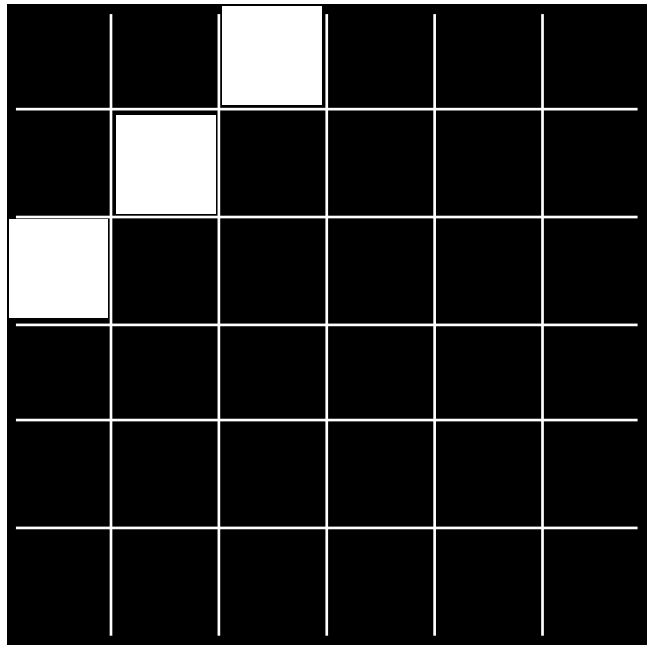
Today… a dream come true?

Video credit: Aishik Ghosh

Plan for today

1. Symmetries and inductive biases
2. Convolutions
3. Convolutional Neural Networks
4. The revolution of depth

Warm-up: edge detector



$x \in \mathbb{R}^{5\times5}$

Q: What weight vector, w, will optimally detect this pattern??

PollEv.com /nicolehartman968

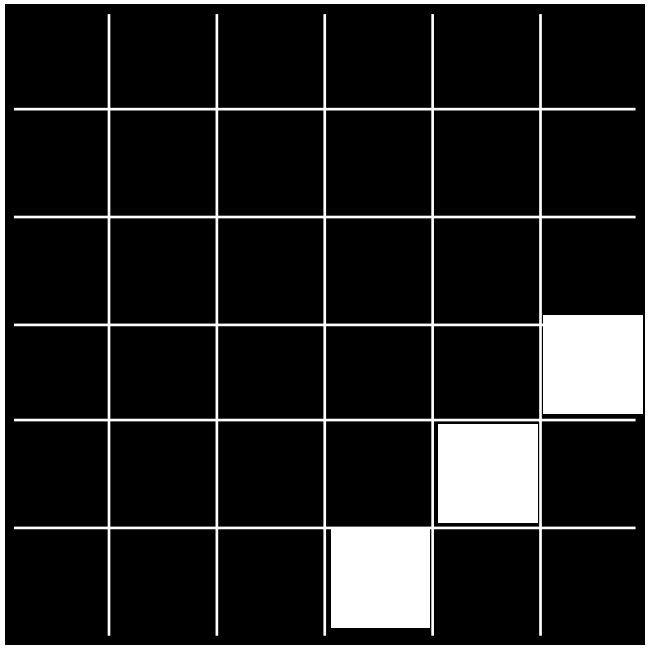
Soln: $w = x^*$

Then you can classify a perceptron

$$f_w(x) = \sigma(w^T x) = \begin{cases} 0, & w^T x < 0 \\ 1, & w^T x > 0 \end{cases}$$

** Or any answer scaled by a positive constant is equally correct.*

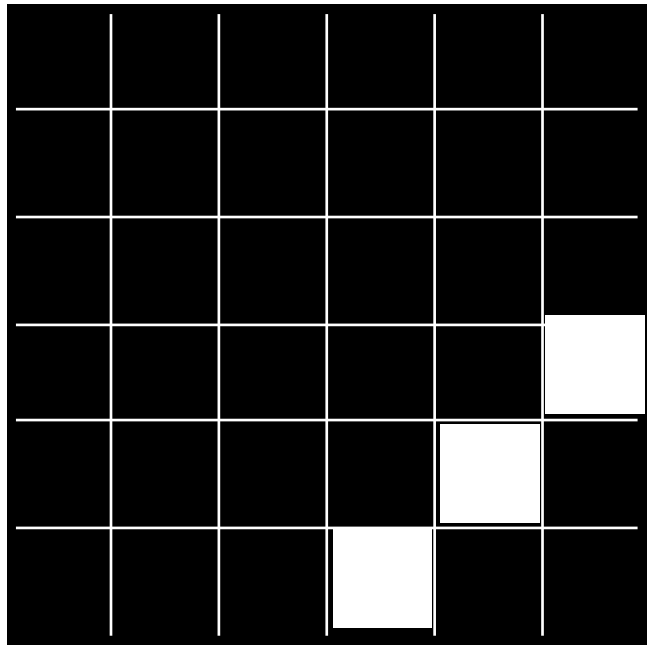
Warm-up: edge detector



$x' \in \mathbb{R}^{5\times5}$

Q: What weight vector, w', will optimally detect this pattern??

Warm-up: edge detector



$x' \in \mathbb{R}^{5\times5}$

Just tell me

Q: What weight vector, w', will optimally detect this pattern??

Soln: $w' = x$

If I want to have my perceptron able to classify *both* of these edges… need to apply both of the ***filters***: $w, w' \in \mathbb{R}^{5\times5}$

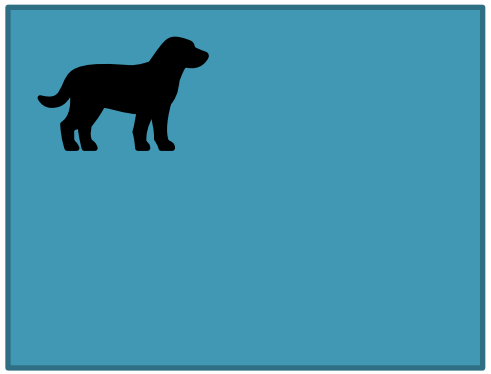
$$W = [w, w'] \in \mathbb{R}^{2\times5\times5}$$

Then sum over the responses of all the filters:

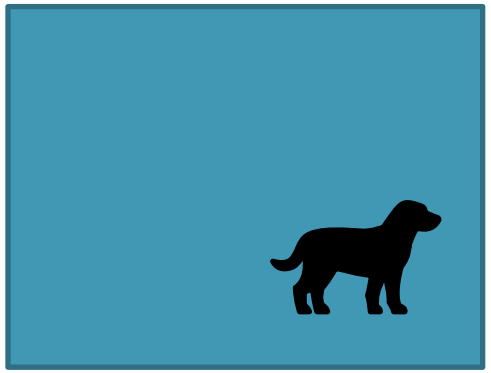
$$z = \sum_{ijk} W_{ijk} x_{jk} \xrightarrow{\text{perceptron}} f_W(z) = \sigma(z)$$

Very memory inefficient!! Can we do better?

Translation invariance



$$\longrightarrow \quad f_\theta(x) = \text{dog}$$



$$\longrightarrow \quad f_\theta(x) = \text{dog}$$

Label invariance: true class doesn't depend on the position in the image.

Plan for today

1 Symmetries and inductive biases

2 **Convolutions**

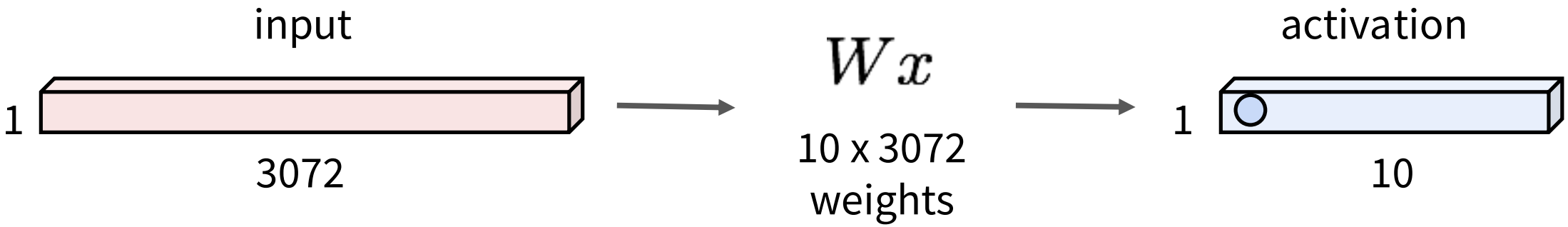
↪ Gratefully drawing from J. Johnson & Fei Fei Li's CS231n course, Lecture 5

3 Convolutional Neural Networks

4 The revolution of depth

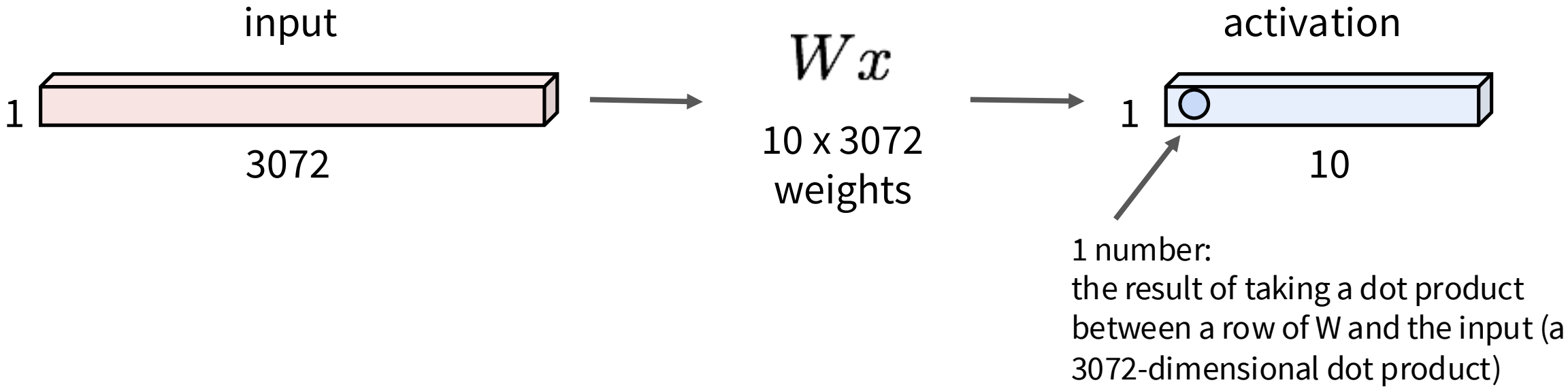
Recap: Fully Connected Layer

32x32x3 image -> stretch to 3072 x 1



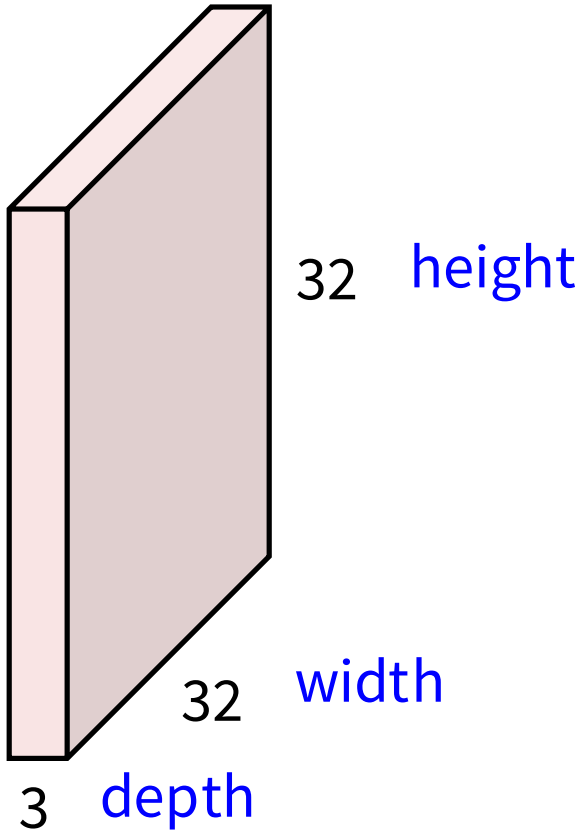
Fully Connected Layer

32x32x3 image -> stretch to 3072 x 1



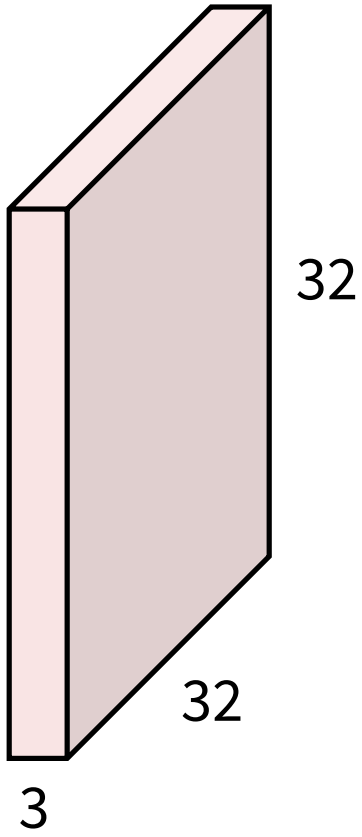
Convolution Layer

32x32x3 image -> preserve spatial structure

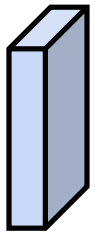


Convolution Layer

32x32x3 image



5x5x3 filter

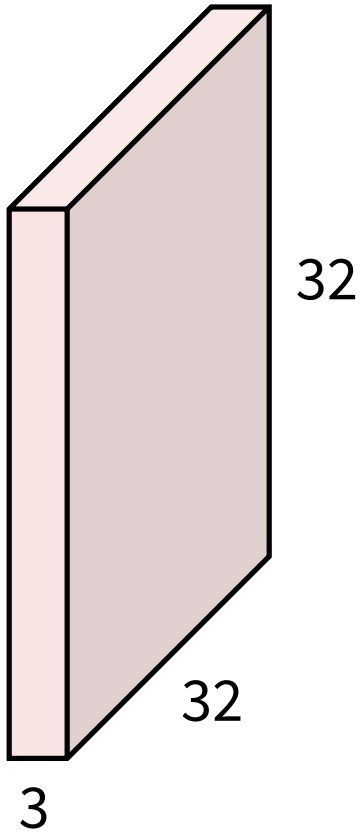


Convolve the filter with the image
i.e. “slide over the image spatially,
computing dot products”

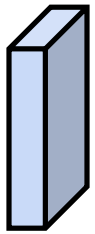
Convolution Layer

Filters always extend the full depth of the input volume

32x32x3 image

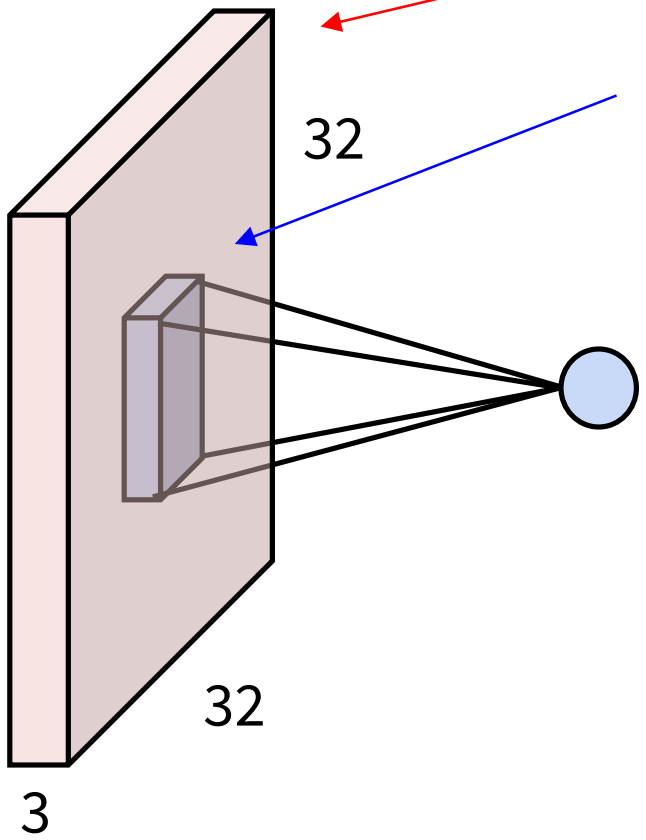


5x5x3 filter



Convolve the filter with the image
i.e. “slide over the image spatially,
computing dot products”

Convolution Layer



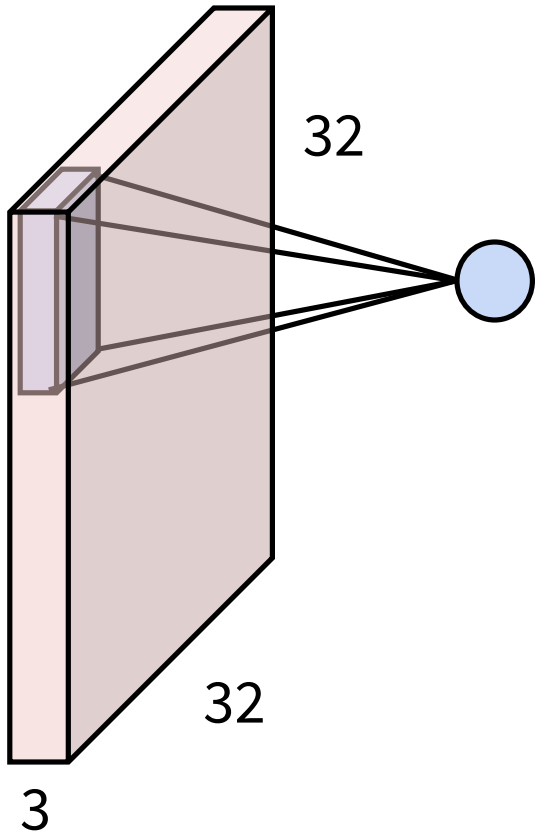
32x32x3 image

5x5x3 filter w

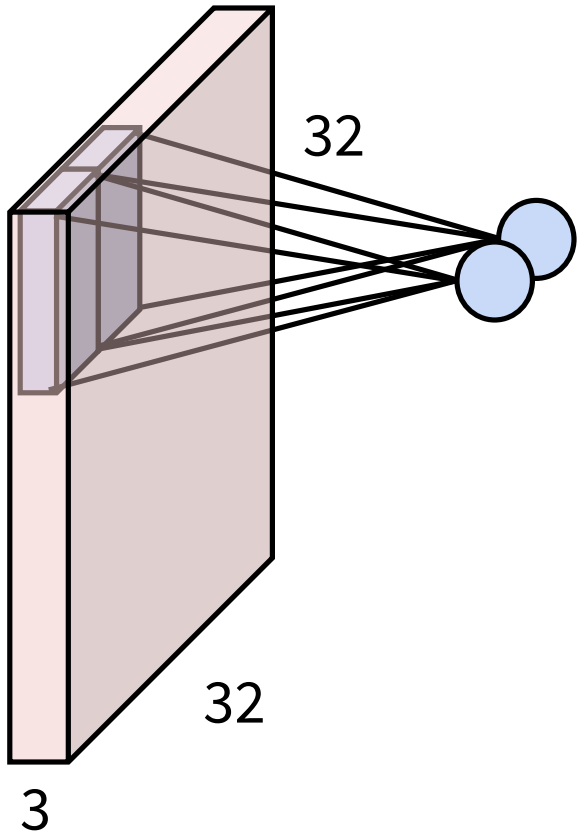
1 number:
the result of taking a dot product between the filter and a small 5x5x3 chunk of the image
(i.e. 5*5*3 = 75-dimensional dot product + bias)

$$w^T x + b$$

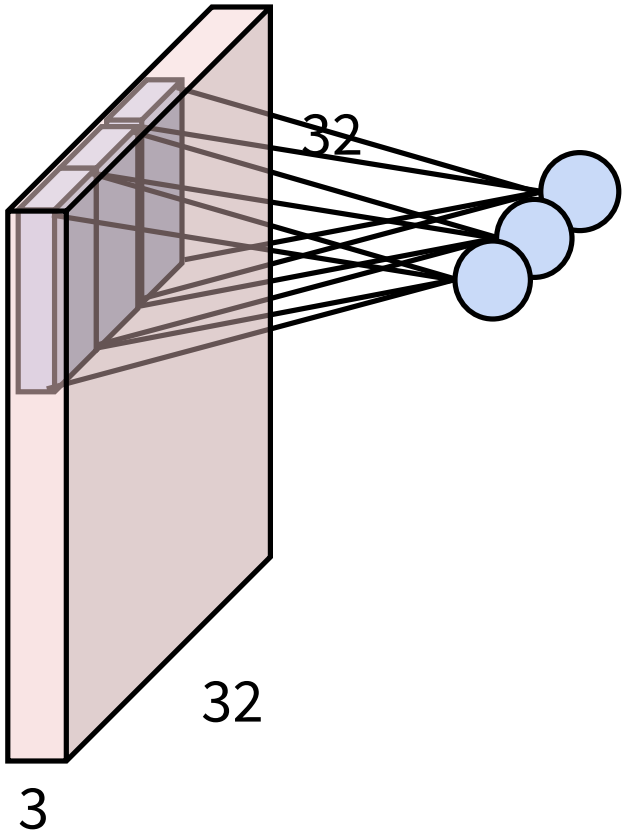
Convolution Layer



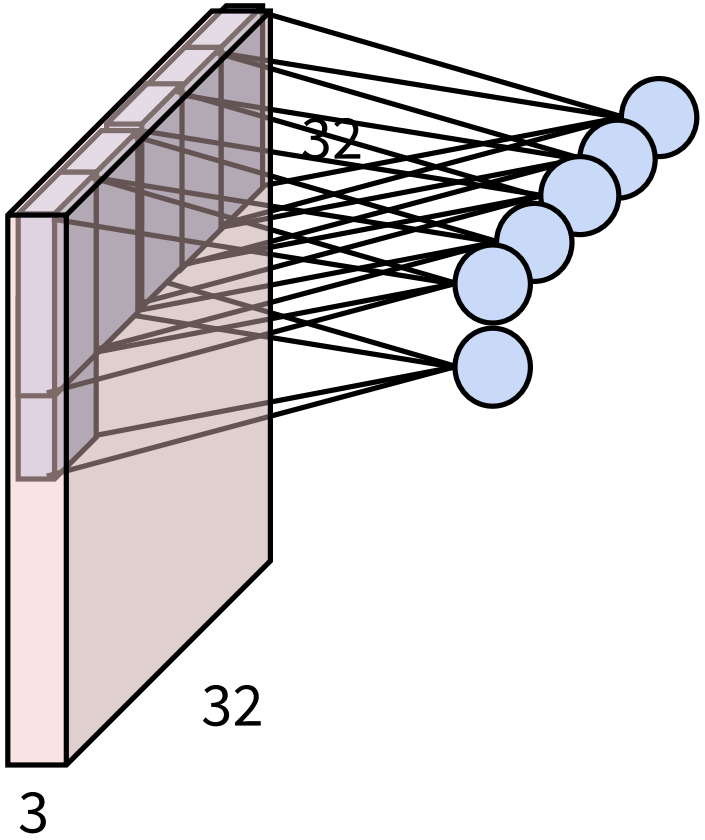
Convolution Layer



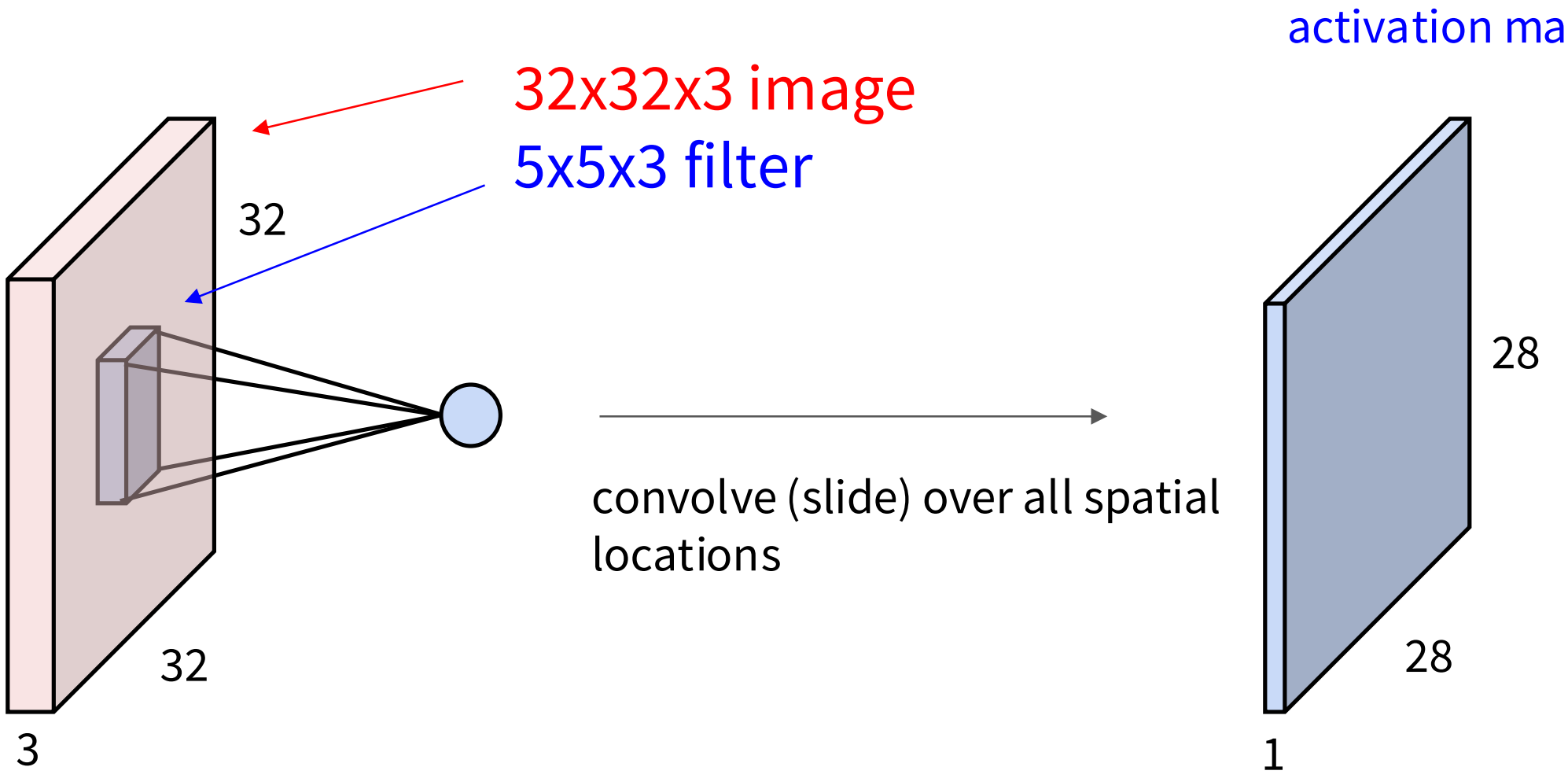
Convolution Layer



Convolution Layer

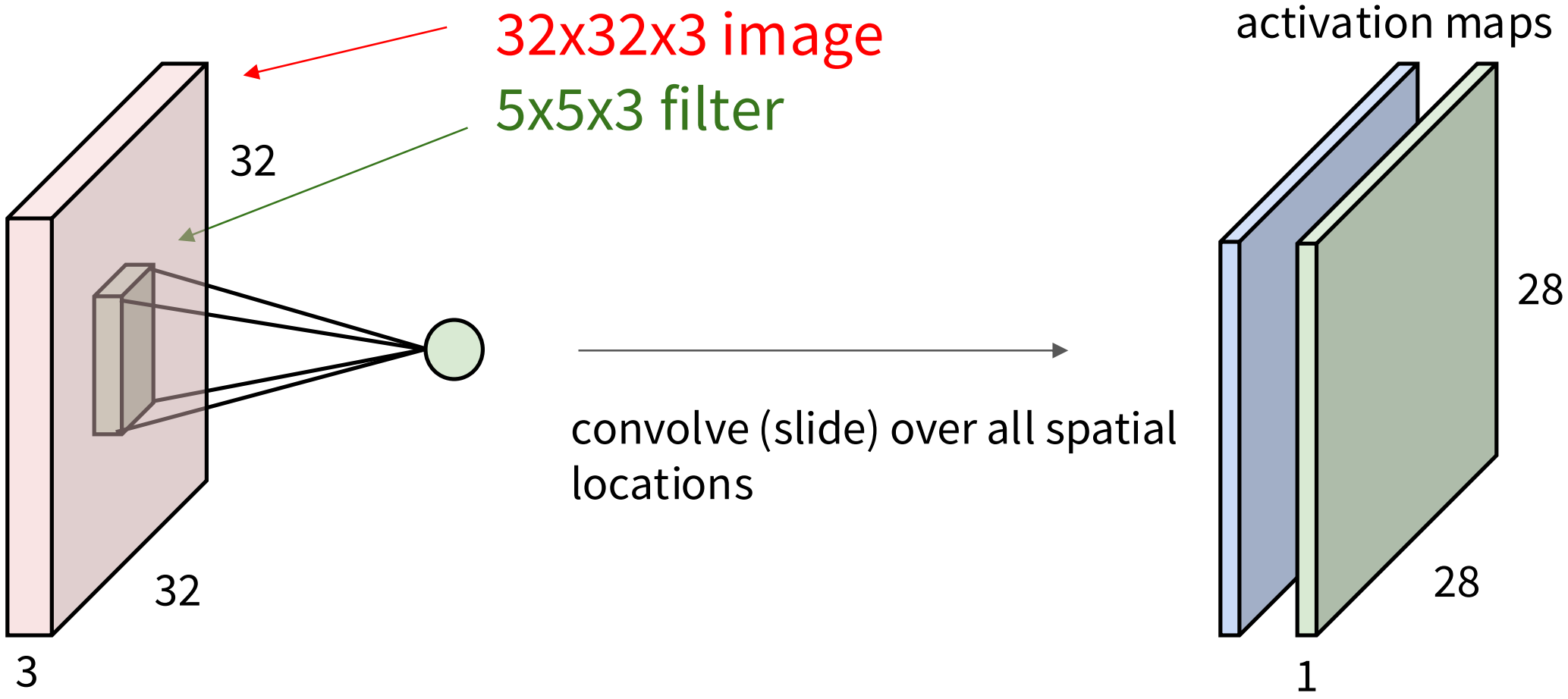


Convolution Layer

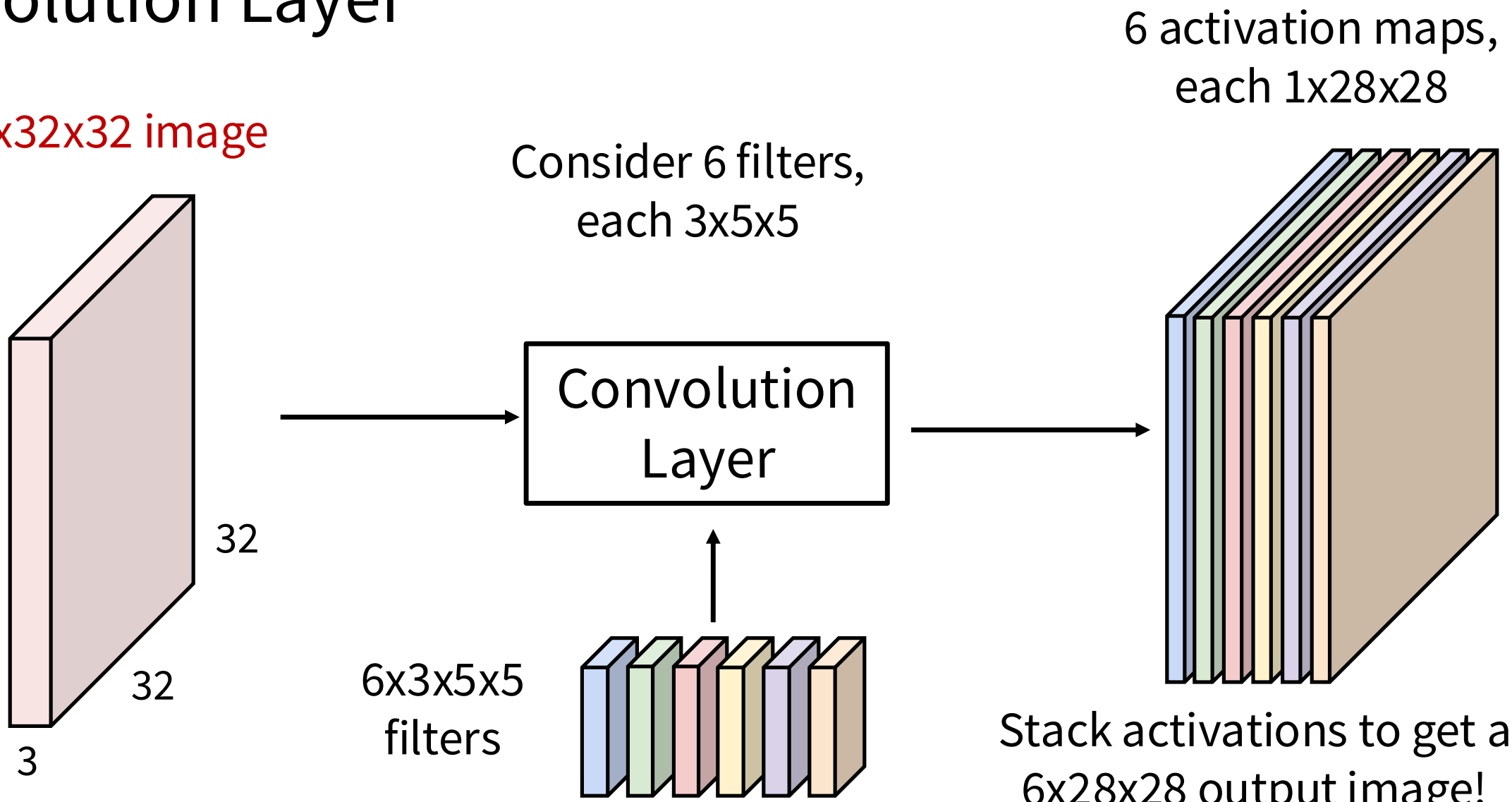


Convolution Layer

consider a second, green filter

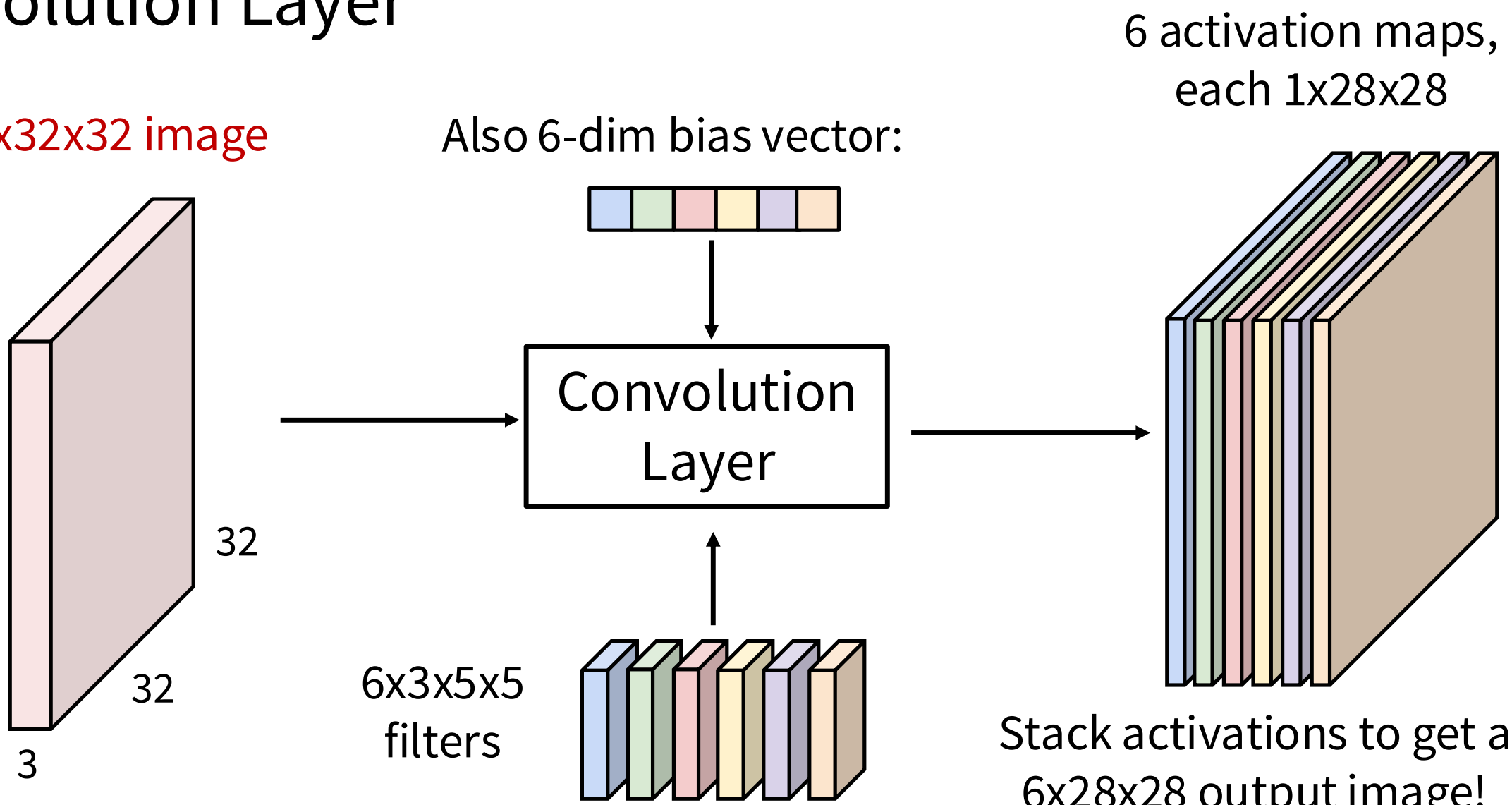


Convolution Layer



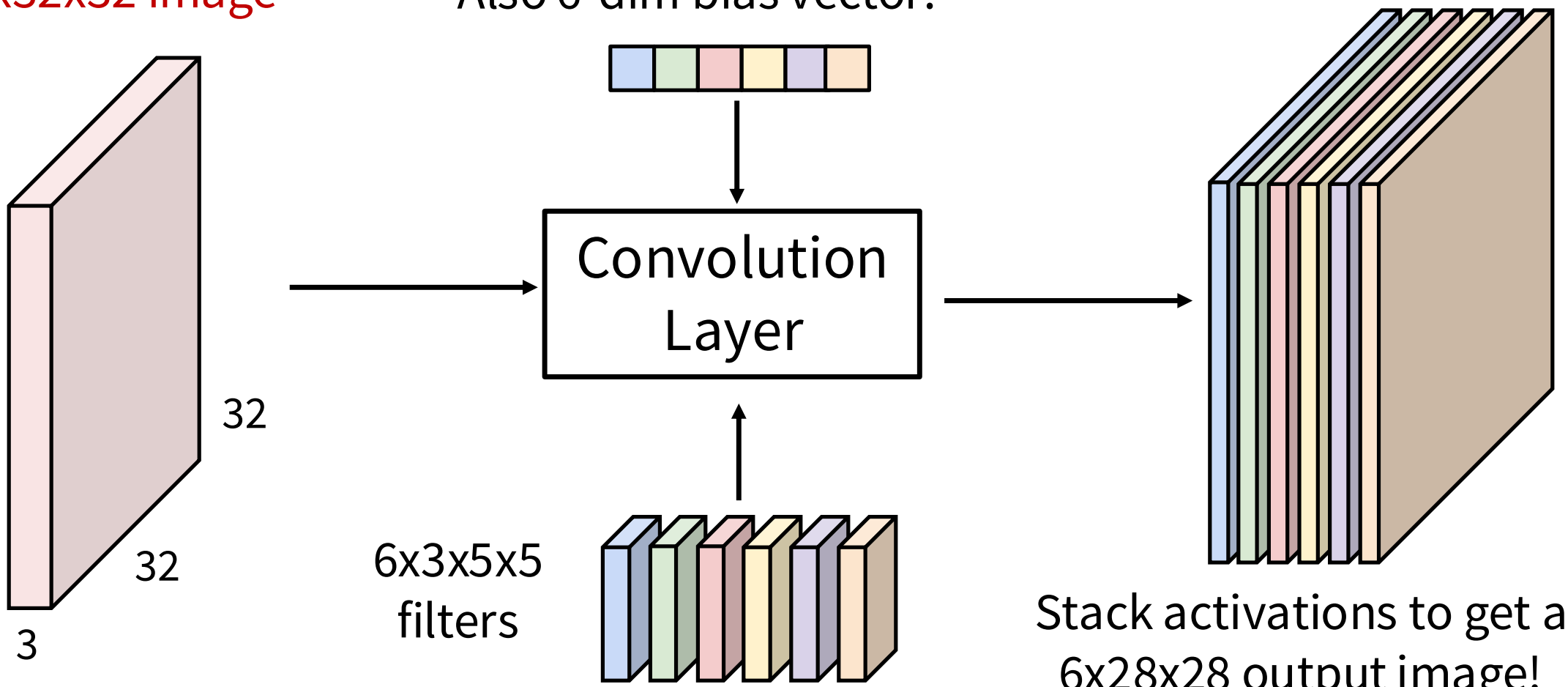
Slide inspiration: Justin Johnson

Convolution Layer



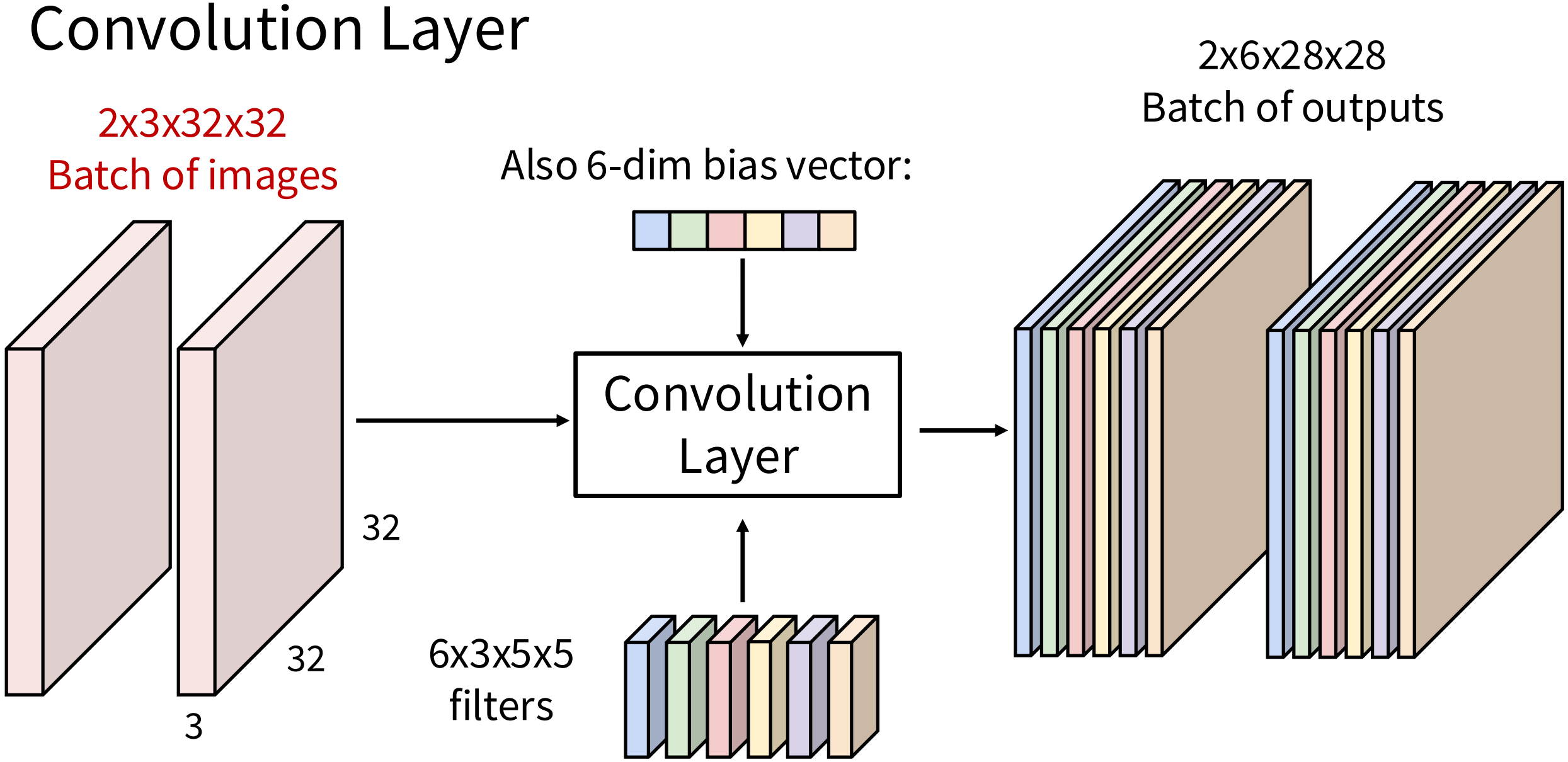
Slide inspiration: Justin Johnson

Convolution Layer



Slide inspiration: Justin Johnson

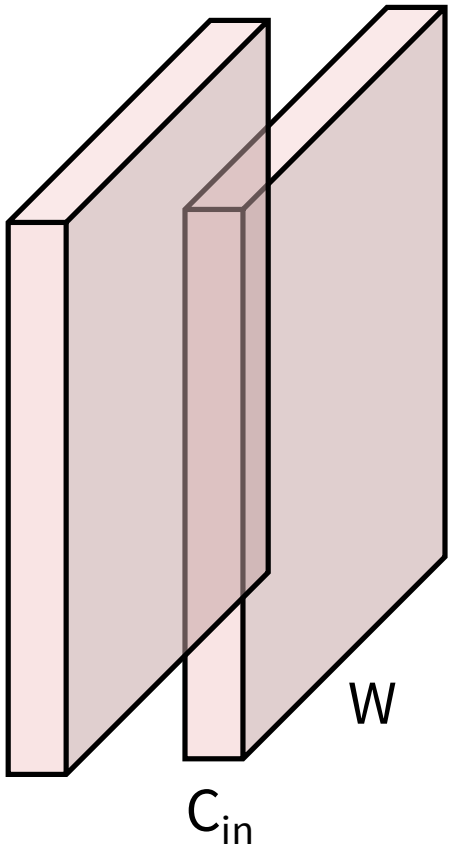
Convolution Layer



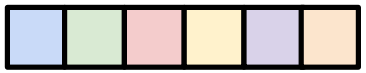
Slide inspiration: Justin Johnson

Convolution Layer

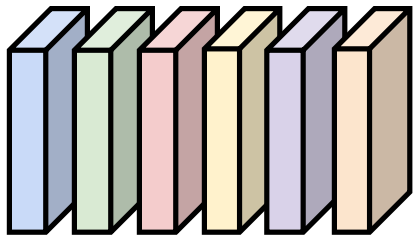
$N \times C_{in} \times H \times W$
Batch of images



Also C_{out}-dim bias vector:

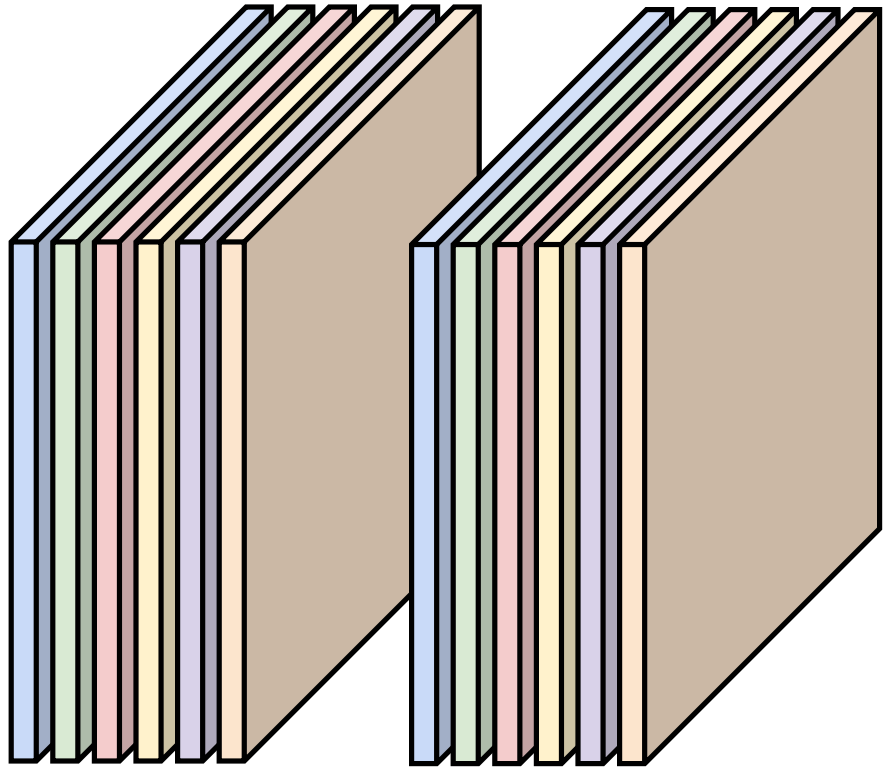


Convolution Layer



$C_{out} \times C_{in} \times K_w \times K_h$
filters

$N \times C_{out} \times H' \times W'$
Batch of outputs



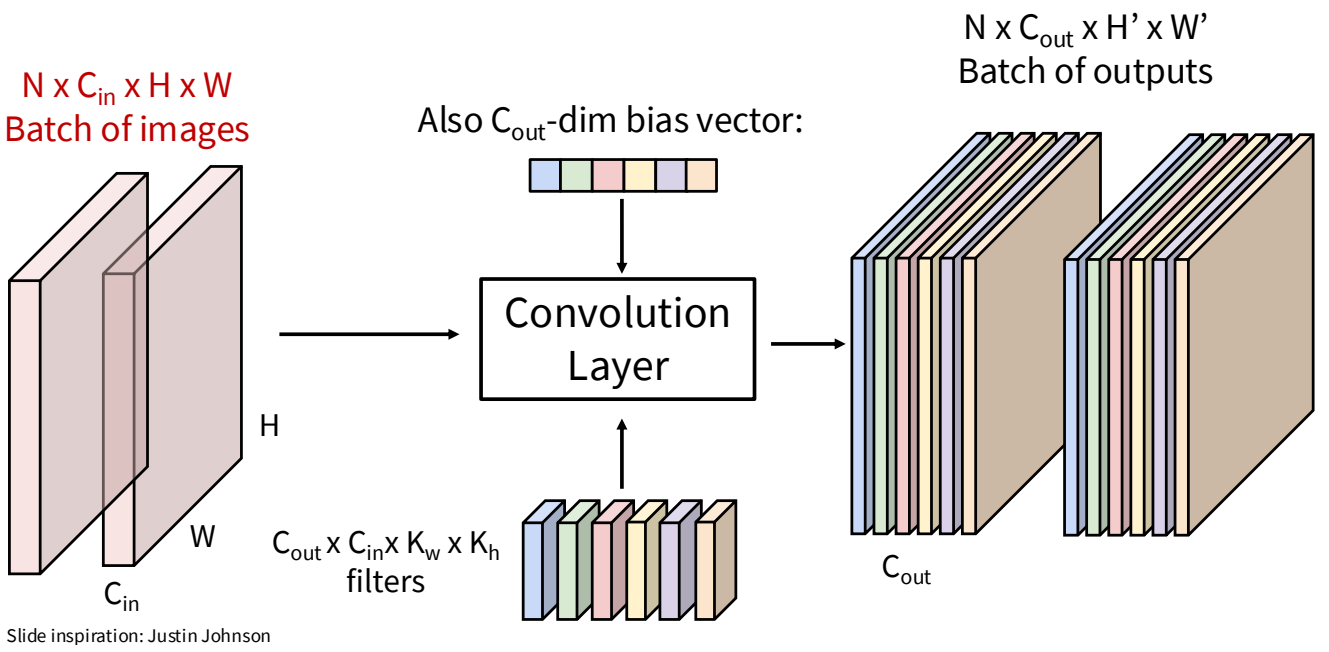
C_{out}

Slide inspiration: Justin Johnson

Understanding checkpoint

Q: How many trainable parameters?

For a batch size of 10 images, image of size $x = \mathbb{R}^{3\times28\times28}$, and conv layers with $C_{in} = 3$, filter size 5 and $C_{out} = 32$.



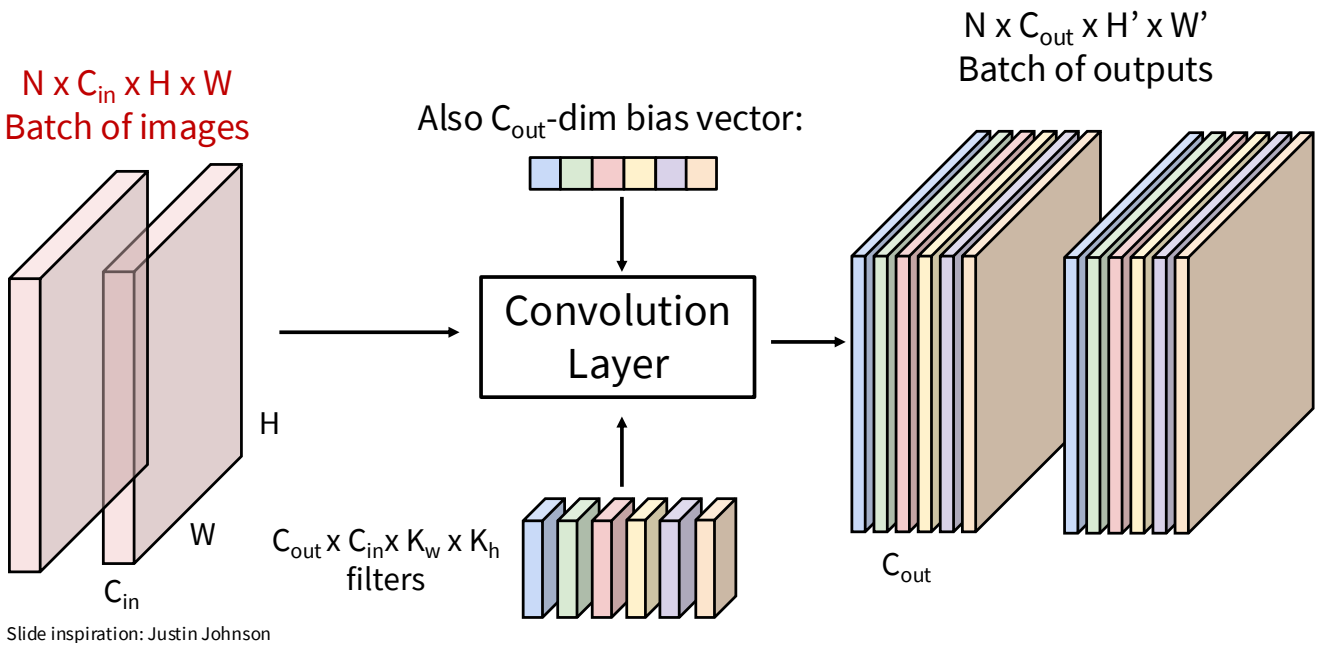
Slide inspiration: Justin Johnson

PollEv.com /nicolehartman968

Understanding checkpoint

Q: How many trainable parameters?

For a batch size of 10 images, image of size $x = \mathbb{R}^{3\times28\times28}$, and conv layers with $C_{in} = 3$, filter size 5 and $C_{out} = 32$.



A: $C_{in} \times K_H \times K_W \times C_{out} + C_{out} = 3 \times 5^2 \times 32 + 32$

$= 2400 + 32$

$= 2432$

Nb: input dimensions don't matter!

Plan for today

1 Symmetries and inductive biases

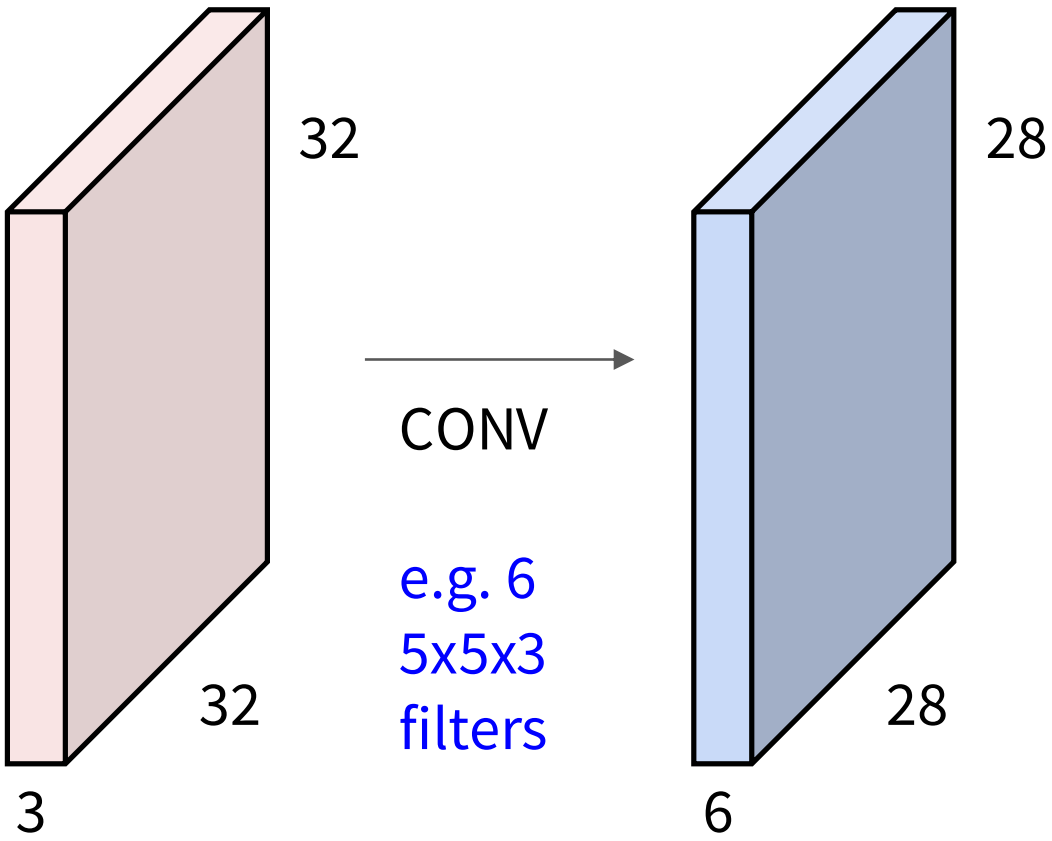
2 Convolutions

3 **Convolutional Neural Networks**

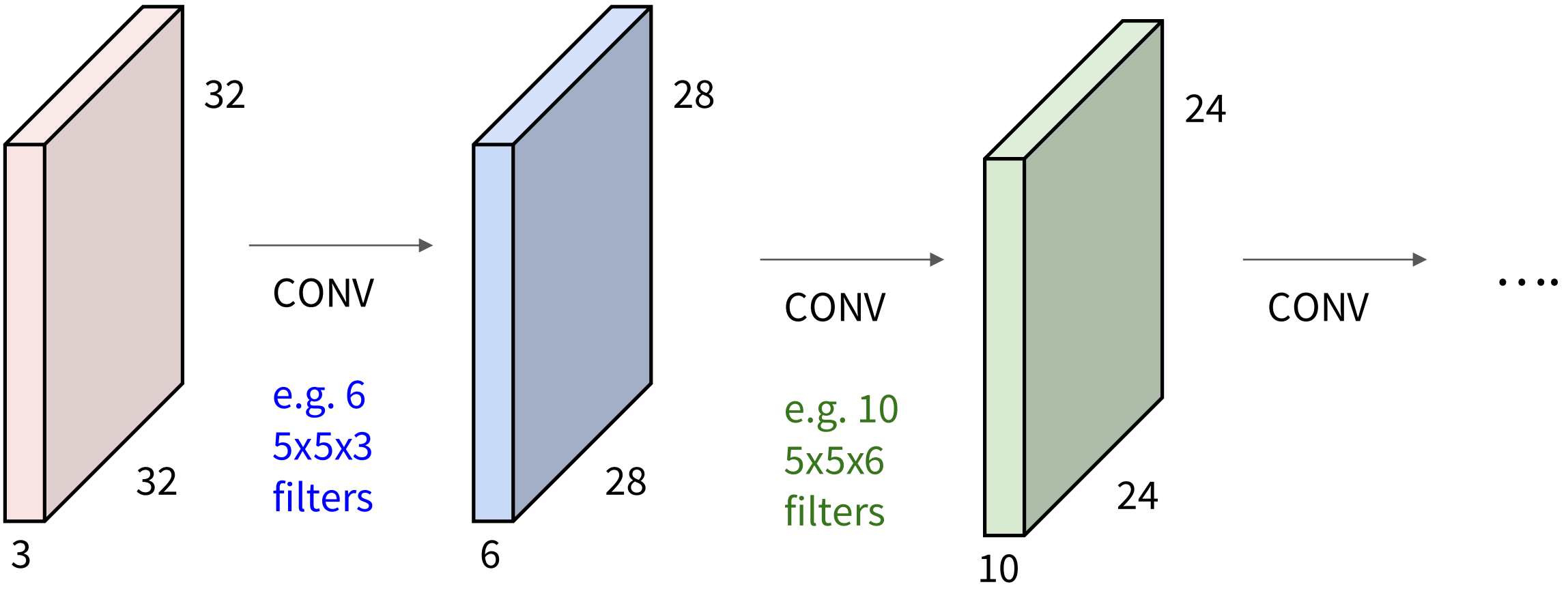
↪ Gratefully drawing from J. Johnson & Fei Fei Li's CS231n course, Lecture 5

4 The revolution of depth

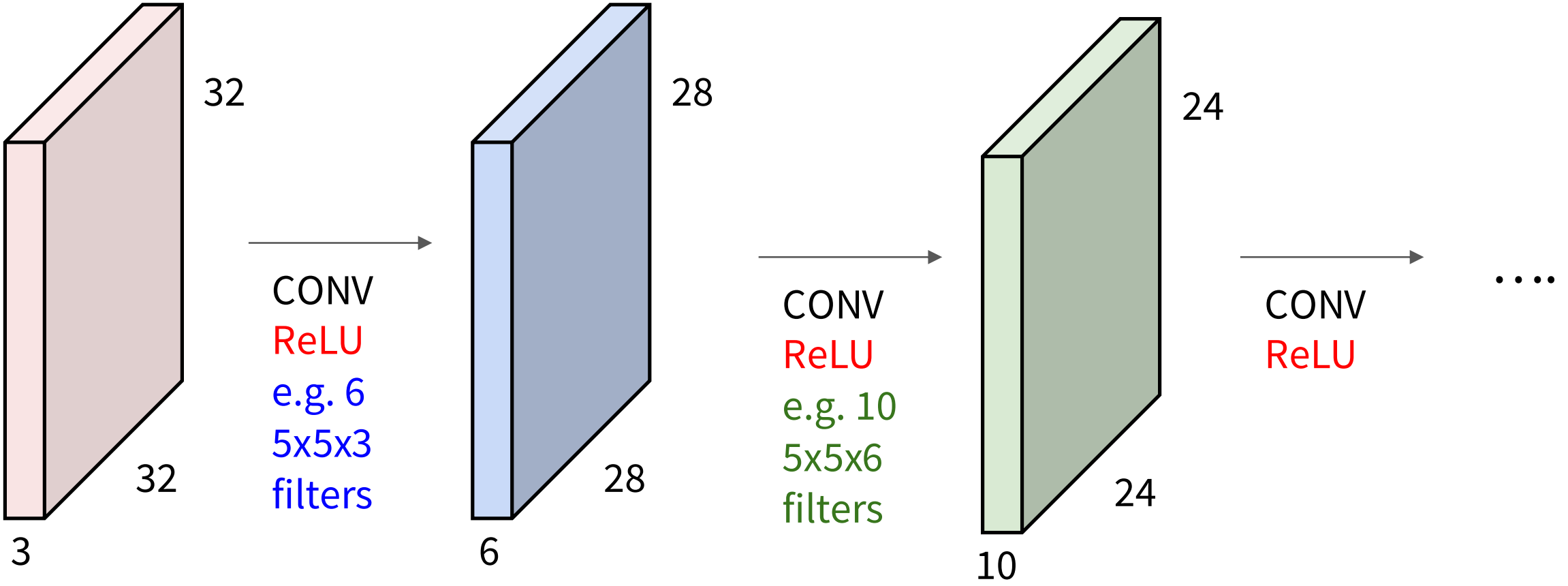
A ConvNet is a neural network with Conv layers



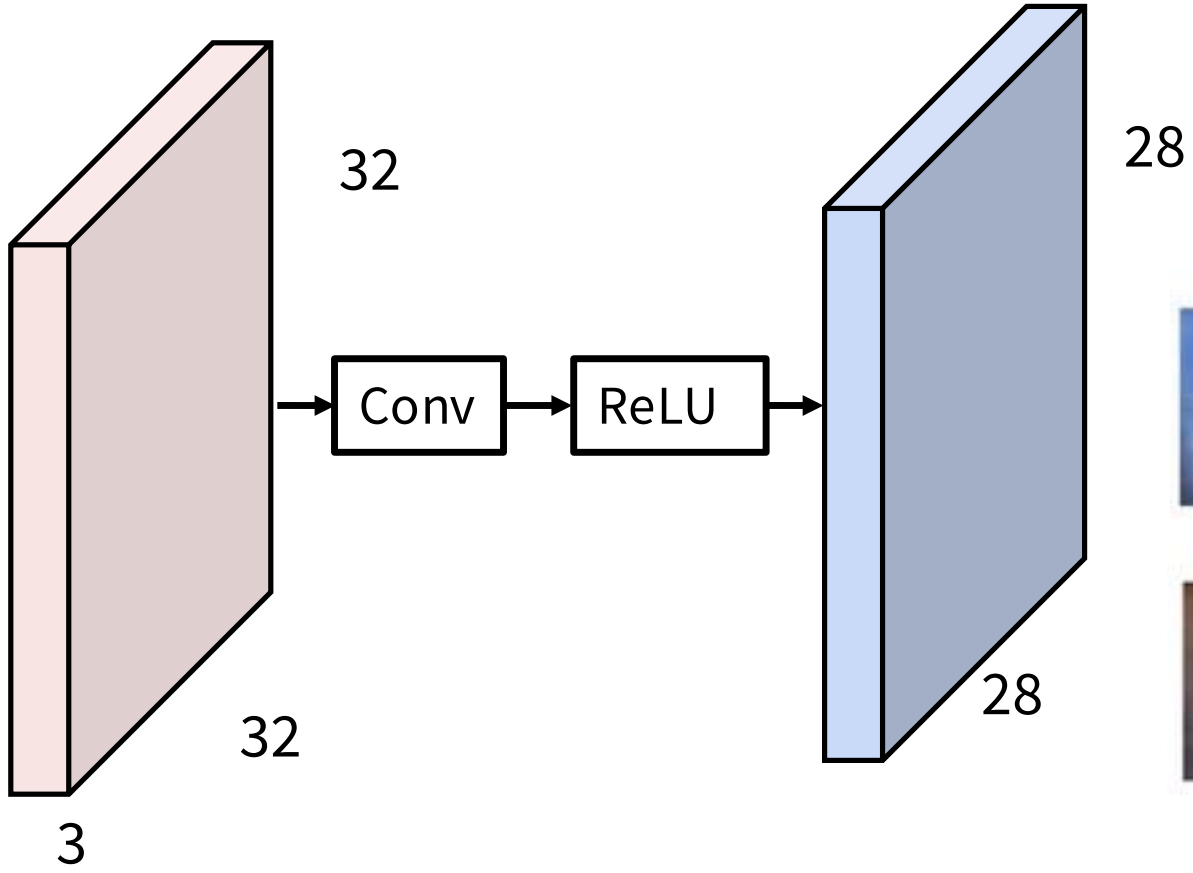
A ConvNet is a neural network with Conv layers



A ConvNet is a neural network with Conv layers with activation functions!

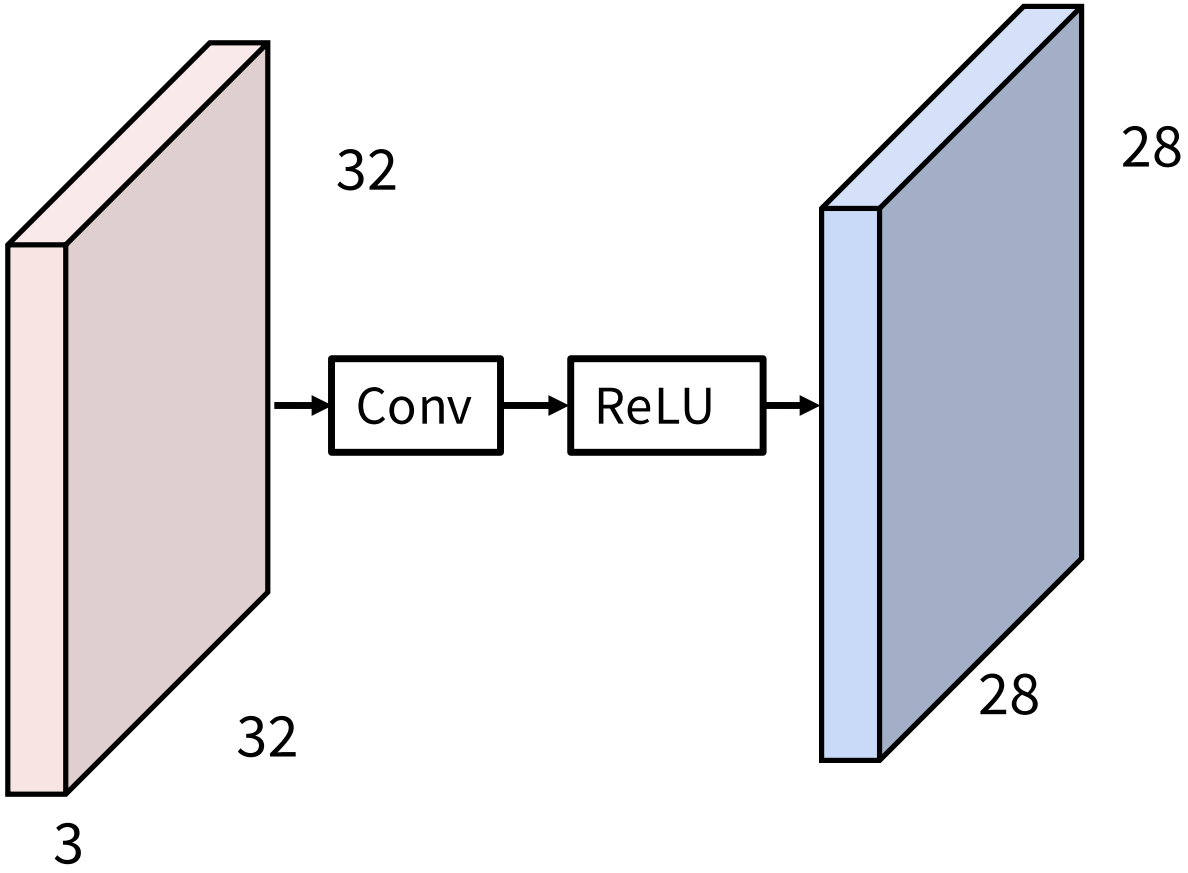


What do Conv filters learn?



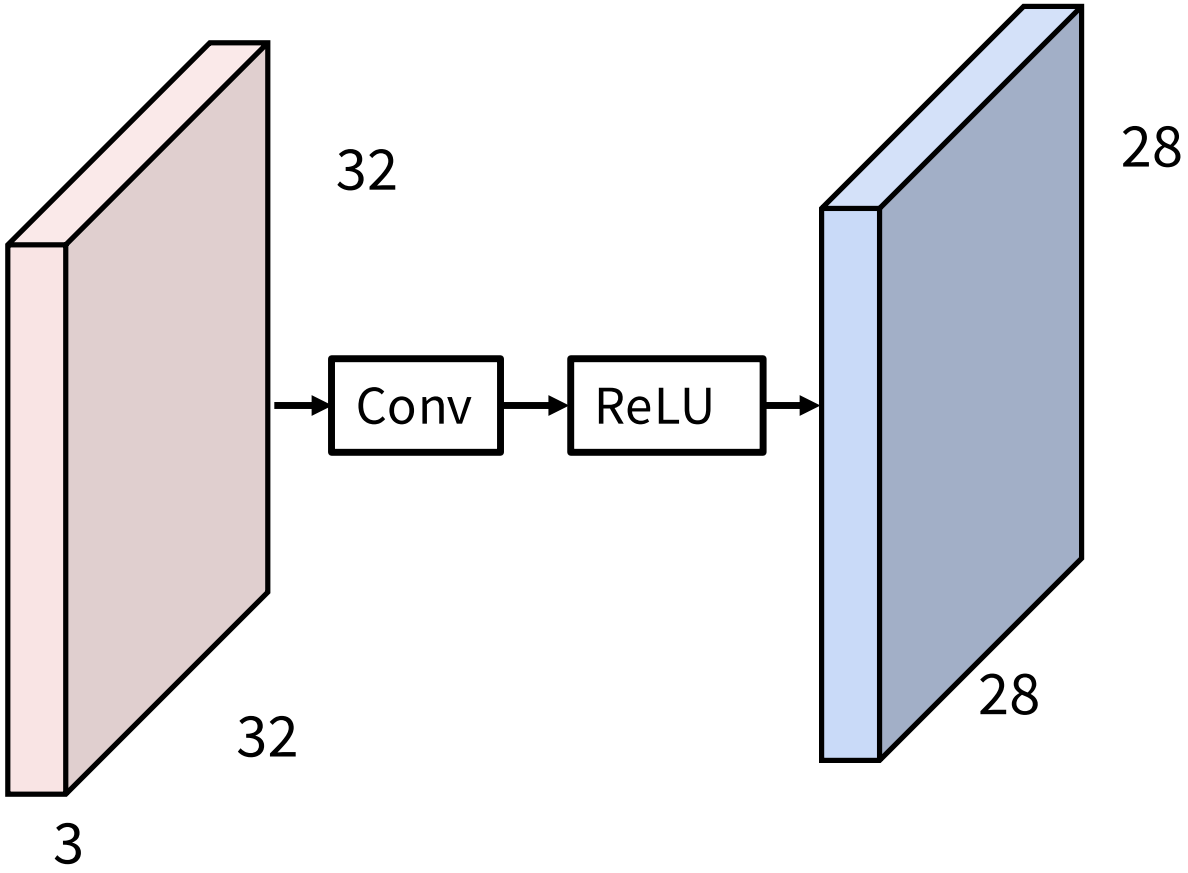
Linear classifier: One template per class

What do Conv filters learn?

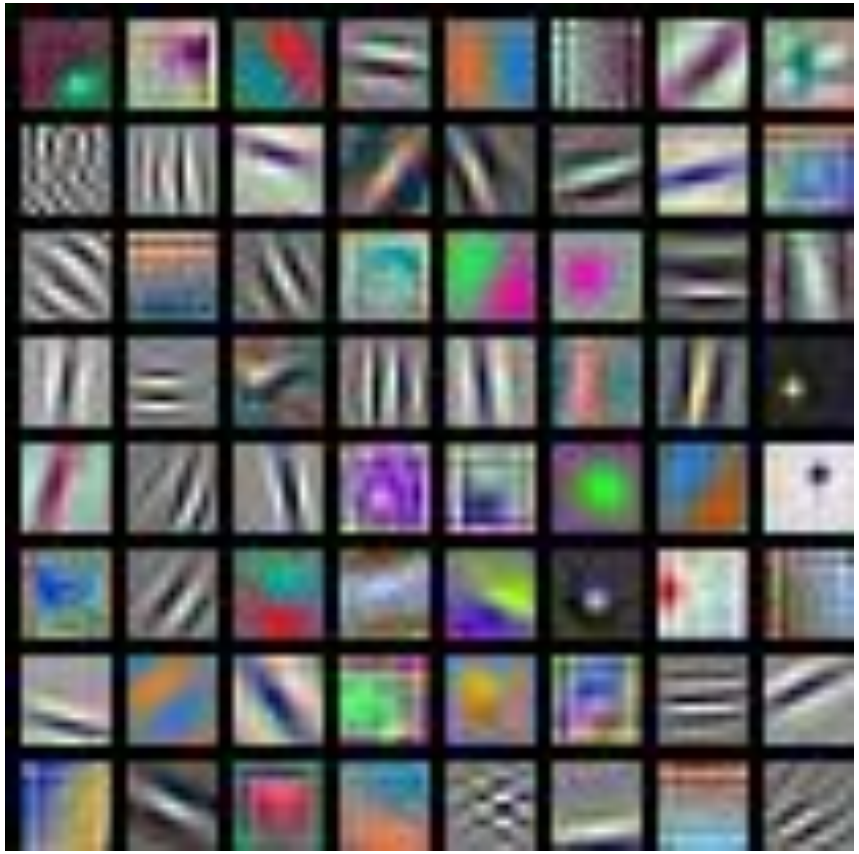


MLP: Bank of whole-image templates

What do Conv filters learn?

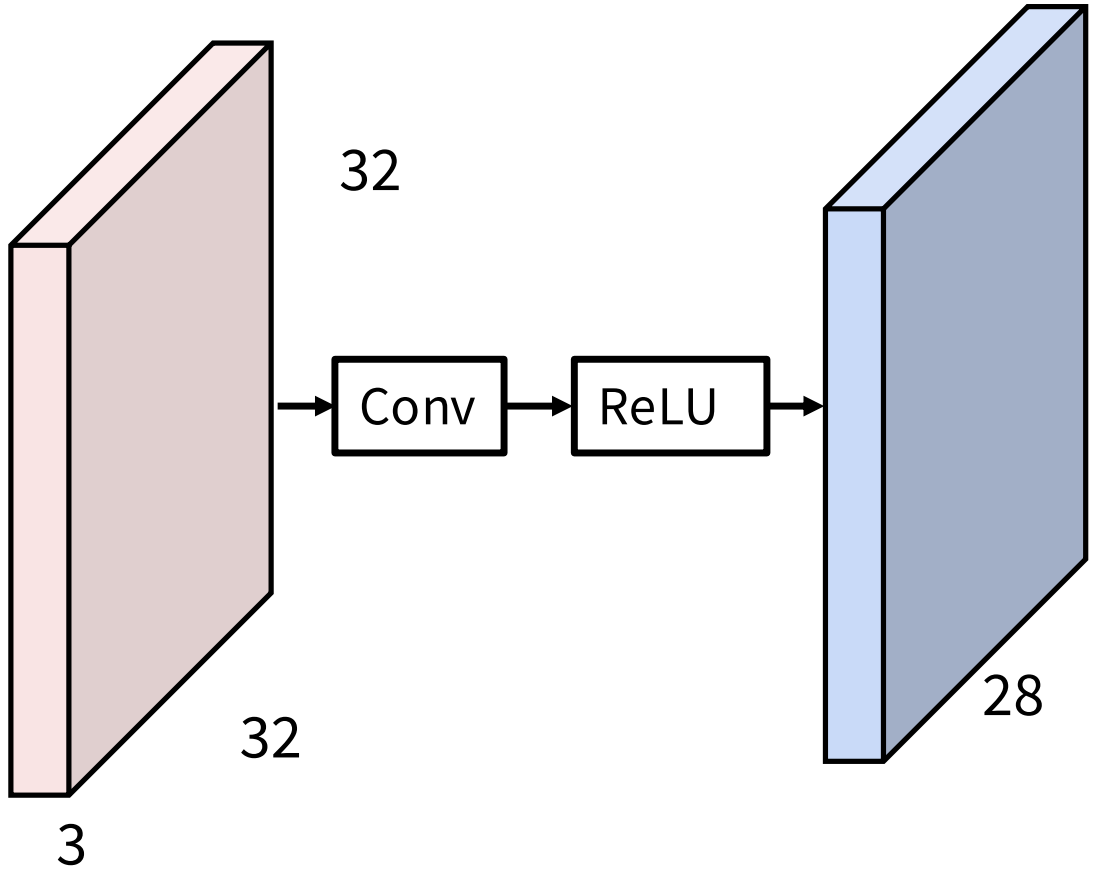


First-layer conv filters: local image templates
(Often learns oriented edges, opposing colors)

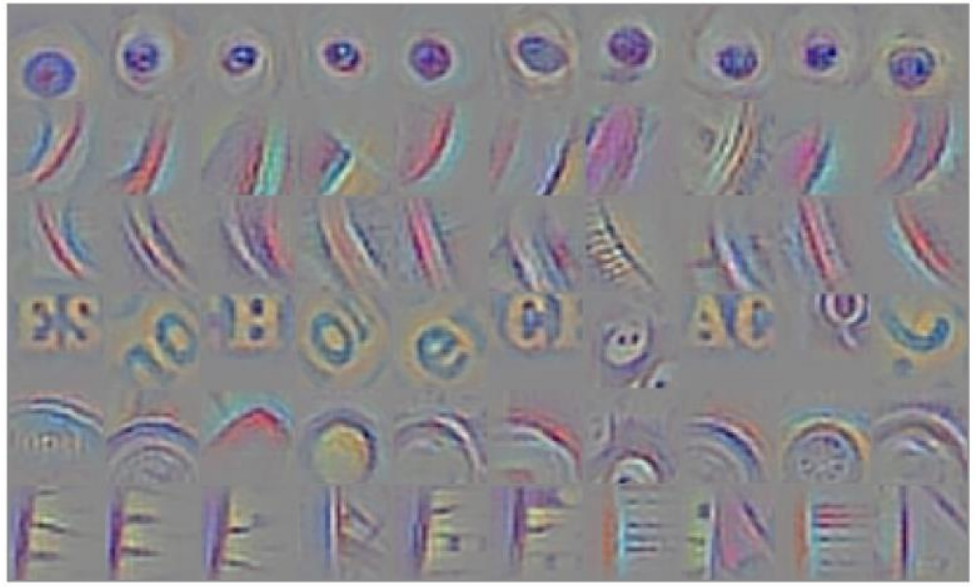


AlexNet: 64 filters, each 3x11x11

What do Conv filters learn?



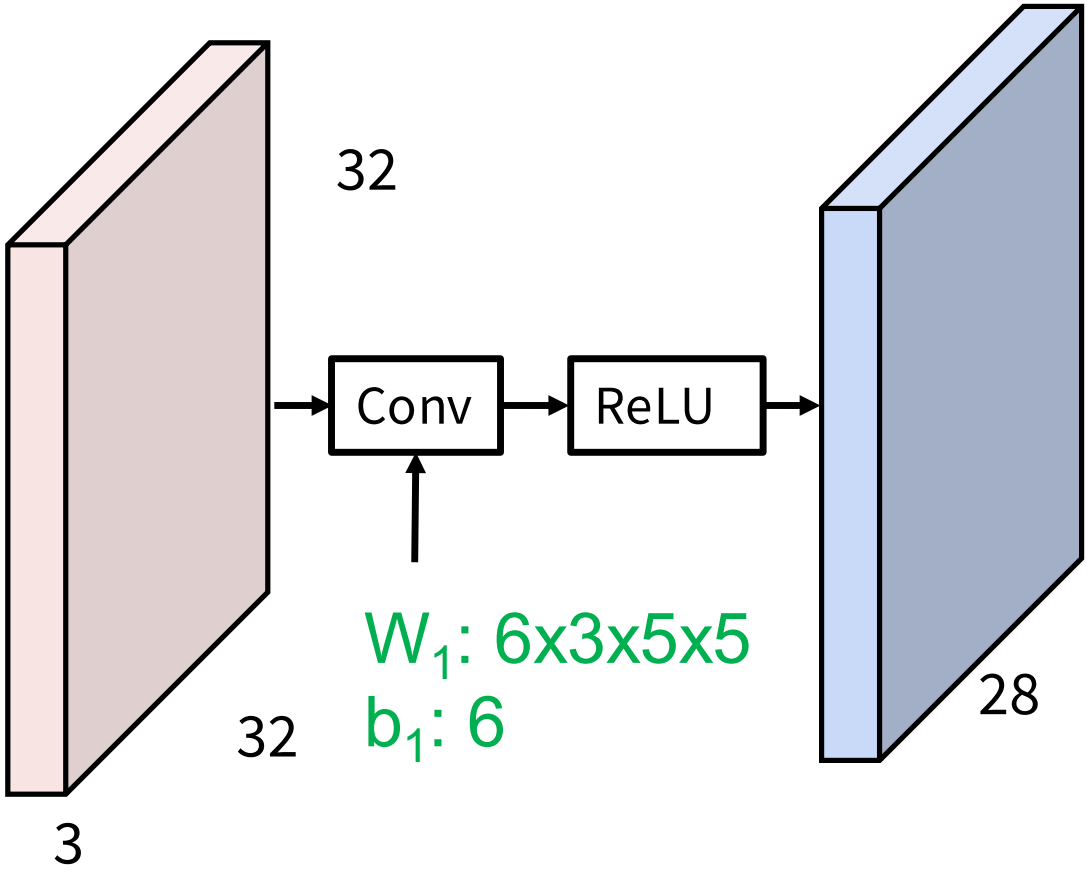
Deeper conv layers: Harder to visualize
Tend to learn larger structures e.g. eyes, letters



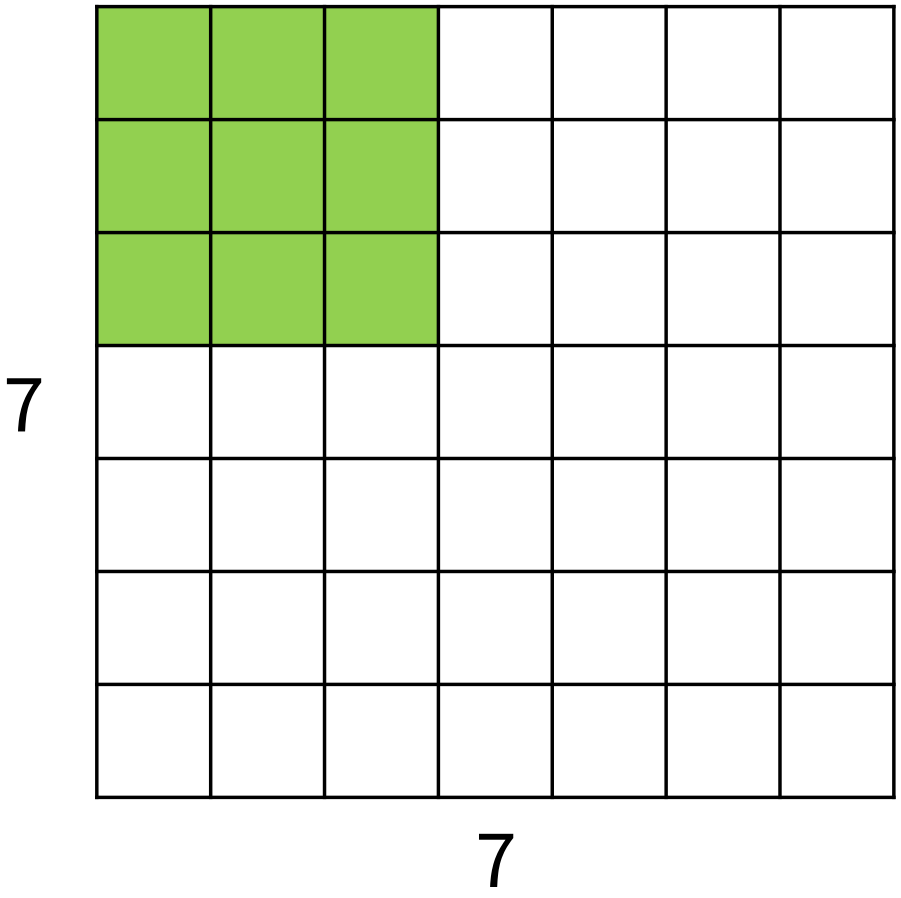
6th layer conv layer from an ImageNet model

Visualization from [Springenberg et al, ICLR 2015]

Convolution: Spatial Dimensions

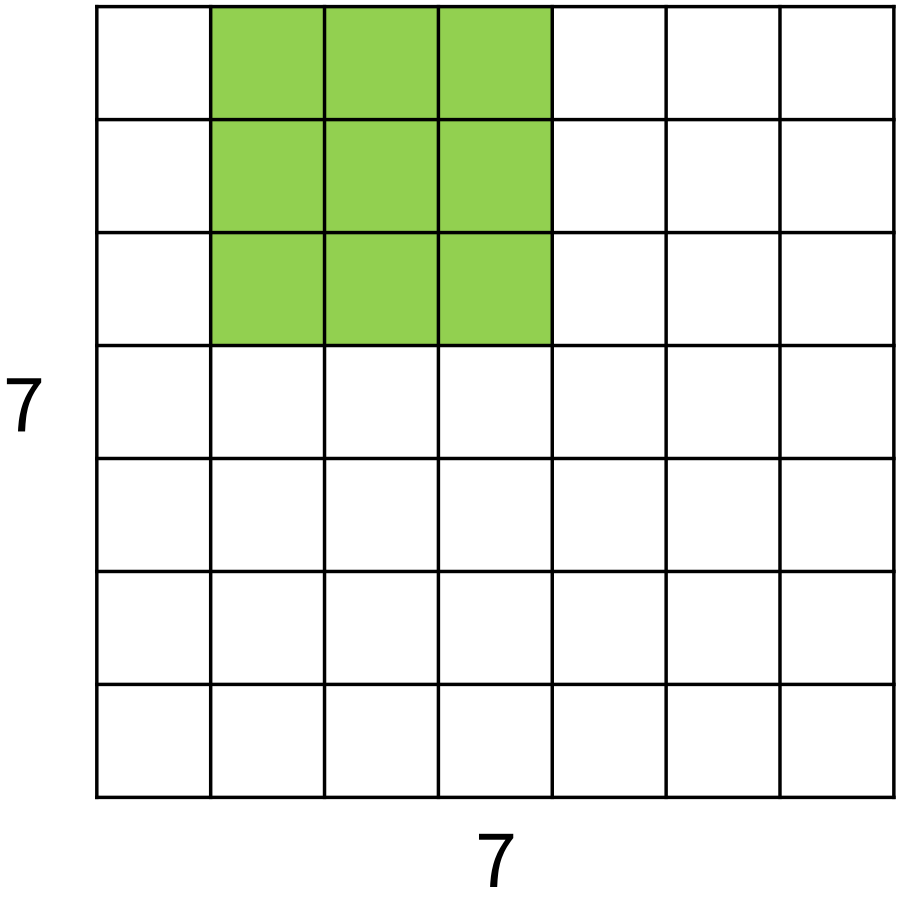


Convolution: Spatial Dimensions



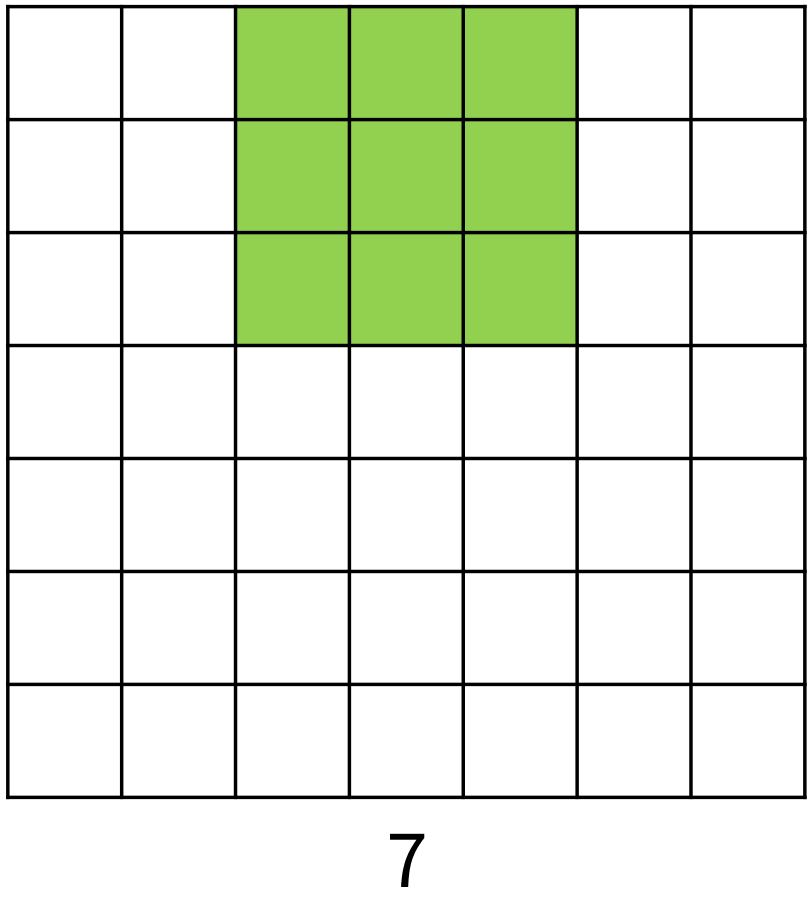
Input: 7x7
Filter: 3x3

Convolution: Spatial Dimensions



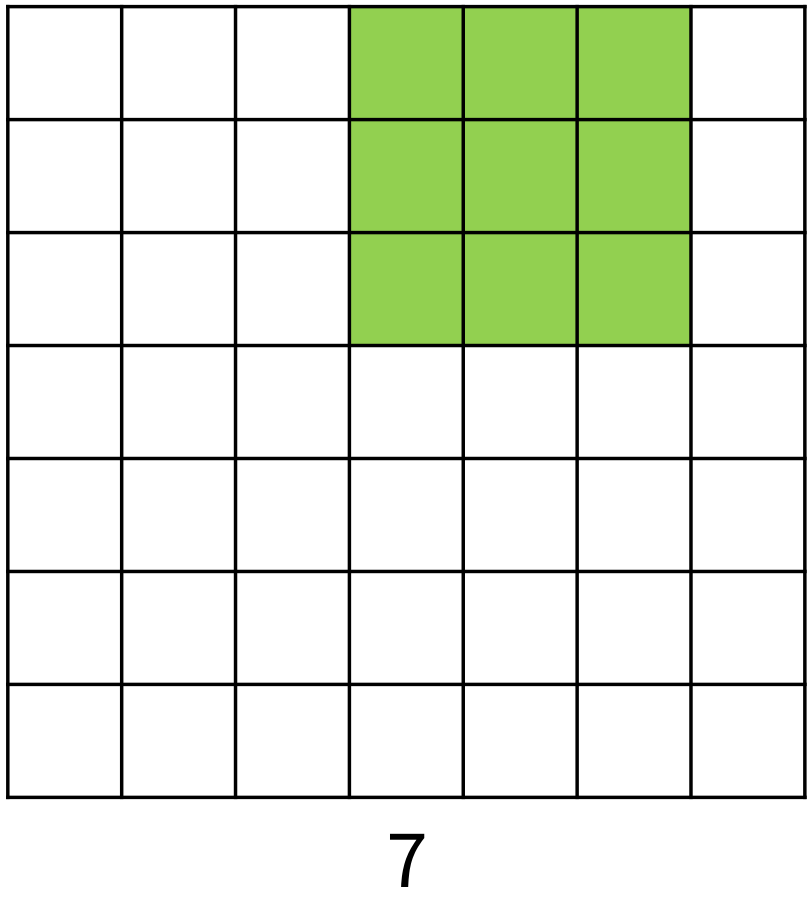
Input: 7x7
Filter: 3x3

Convolution: Spatial Dimensions



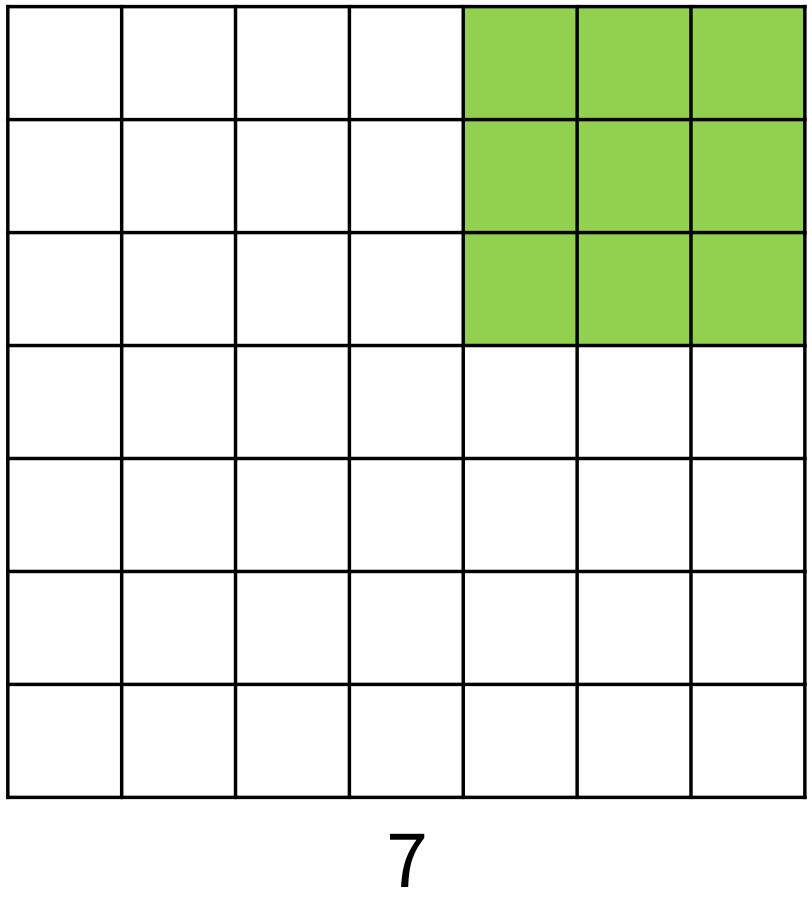
Input: 7x7
Filter: 3x3

Convolution: Spatial Dimensions



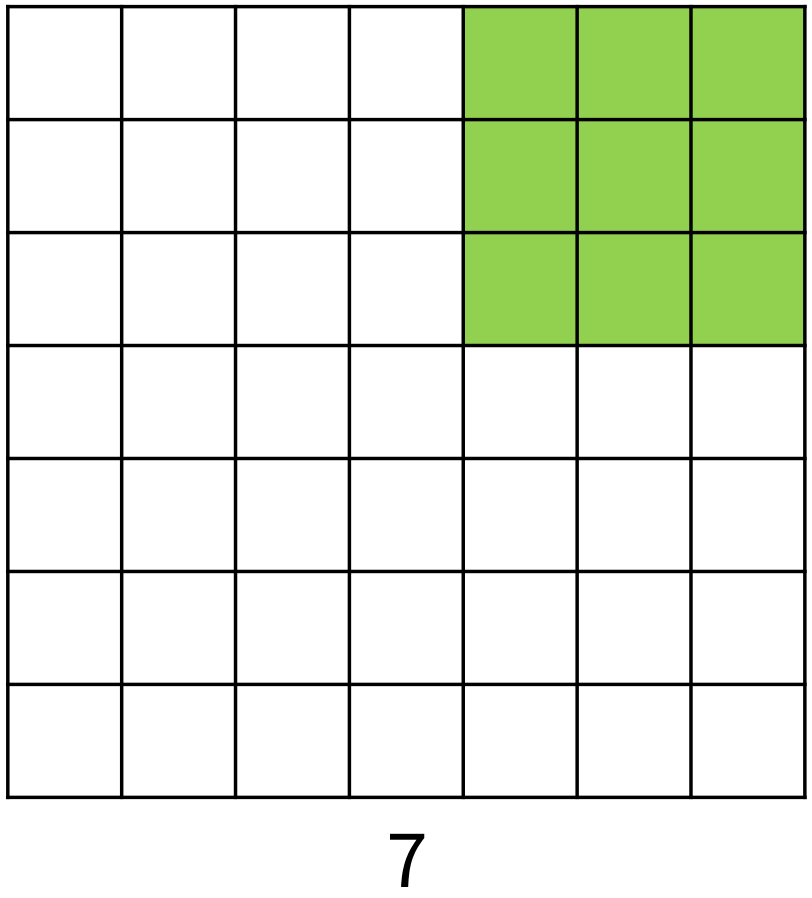
Input: 7x7
Filter: 3x3

Convolution: Spatial Dimensions



Input: 7x7
Filter: 3x3
Output: 5x5

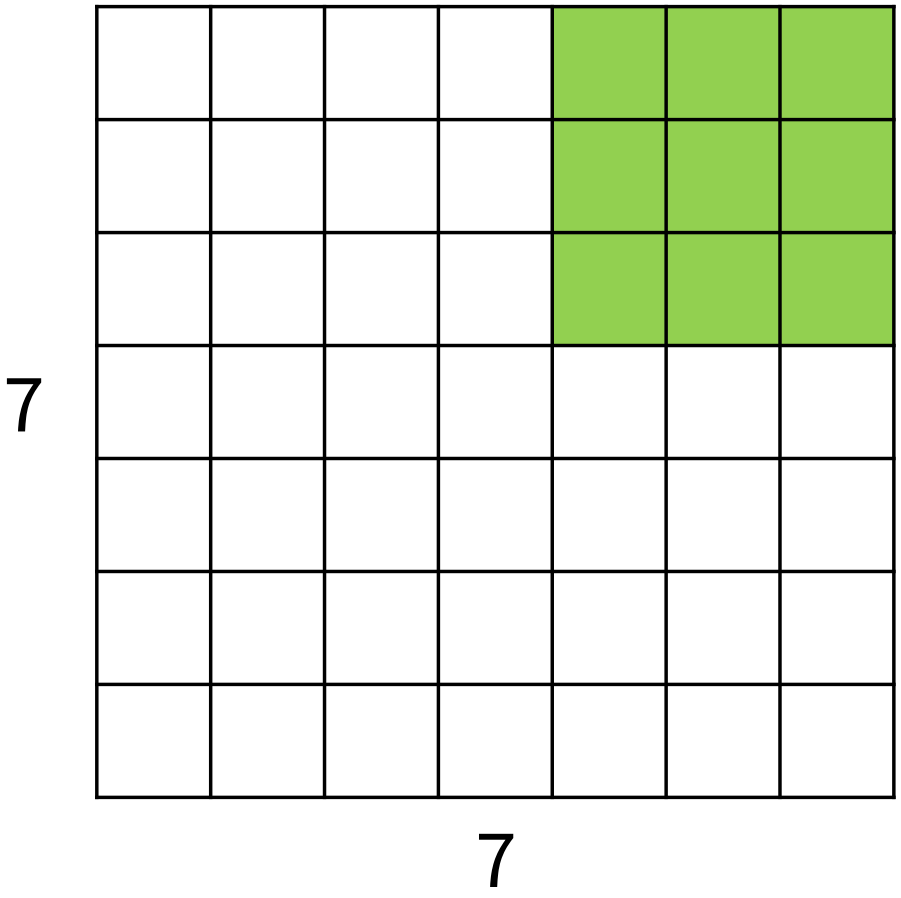
Convolution: Spatial Dimensions



Input: 7x7
Filter: 3x3
Output: 5x5

In general
Input: W
Filter: K
Output: W – K + 1

Convolution: Spatial Dimensions

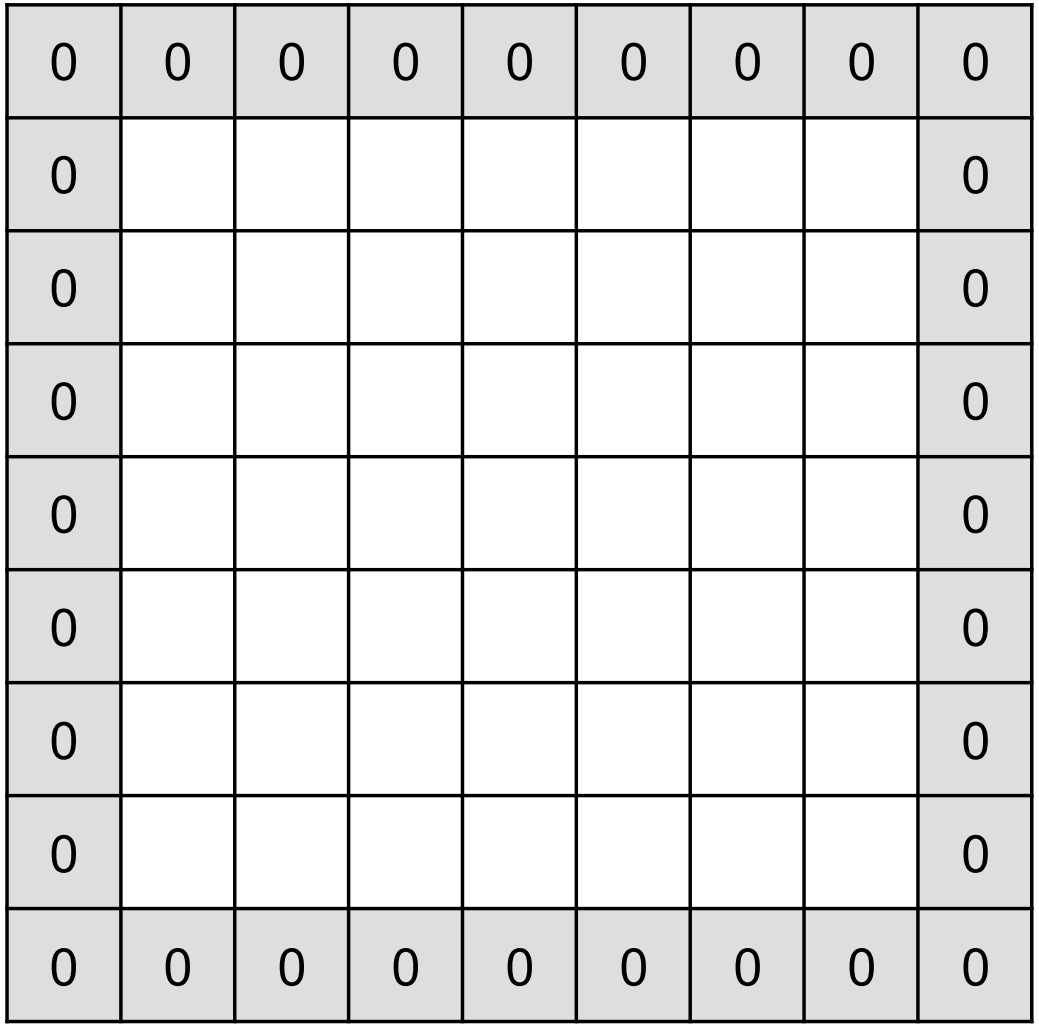


Input: 7x7
Filter: 3x3
Output: 5x5

Problem: Feature maps shrink with each layer!

In general
Input: W
Filter: K
Output: W – K + 1

Convolution: Spatial Dimensions



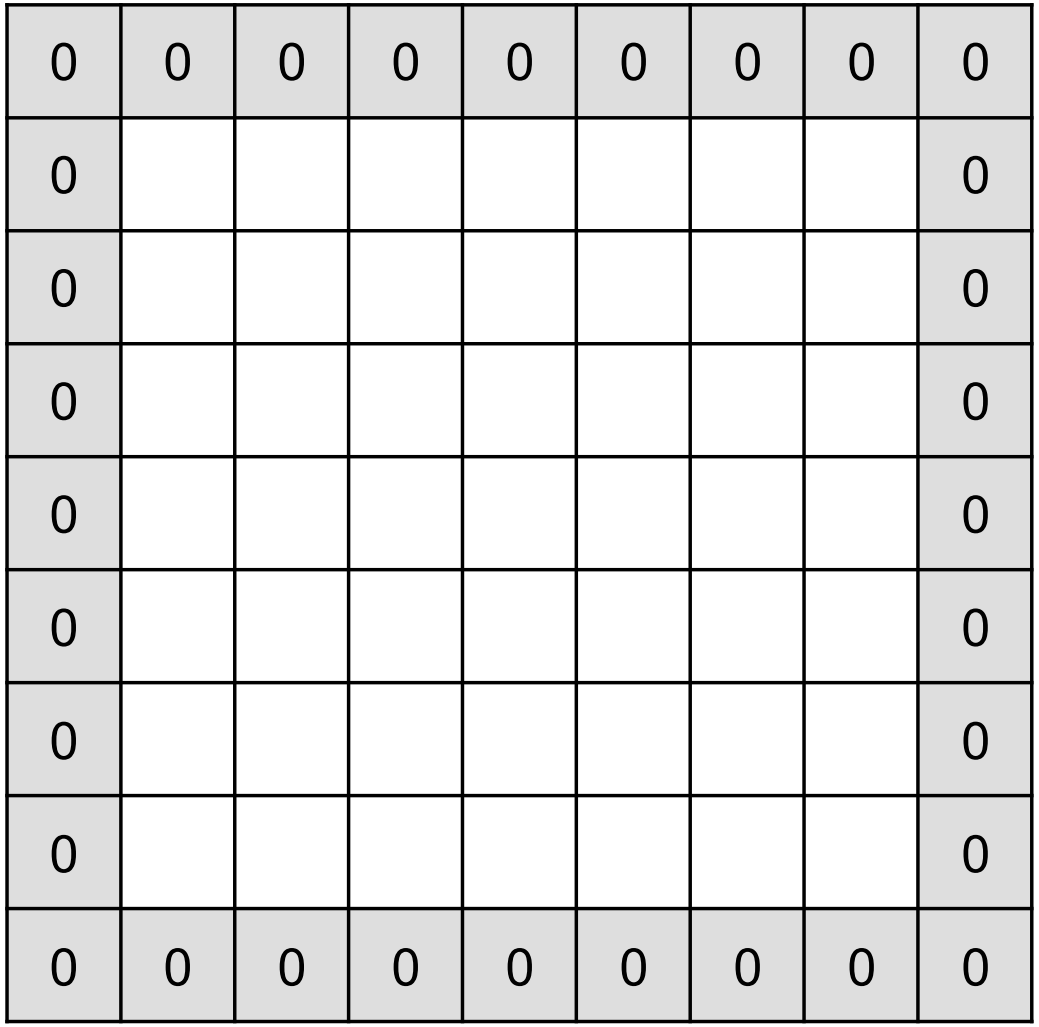
Input: 7x7
Filter: 3x3
Output: 5x5

Problem: Feature maps shrink with each layer!

In general
Input: W
Filter: K
Padding: P
Output: W – K + 1 + 2P

Solution: Add **padding** around the input before sliding the filter

Convolution: Spatial Dimensions

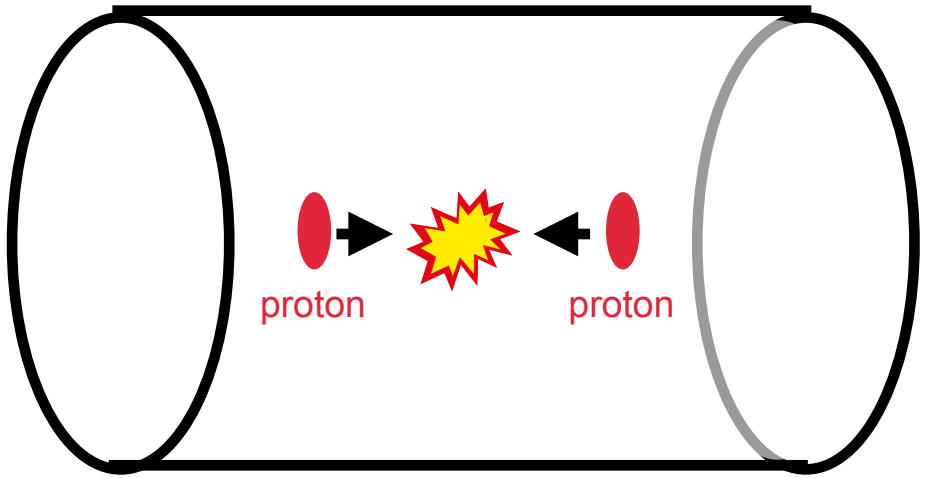


Input: 7x7
Filter: 3x3
Output: 5x5

In general
Input: W
Filter: K
Padding: P
Output: W – K + 1 + 2P

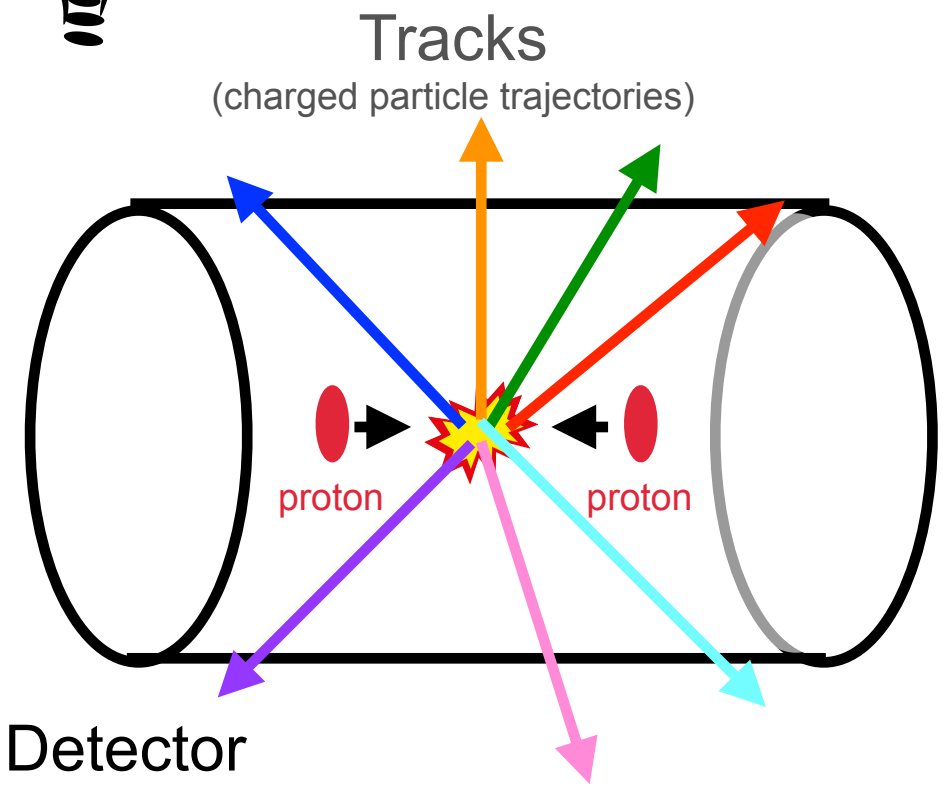
Common setting:
P = (K – 1) / 2
Means output has same size as input

Padding… another application

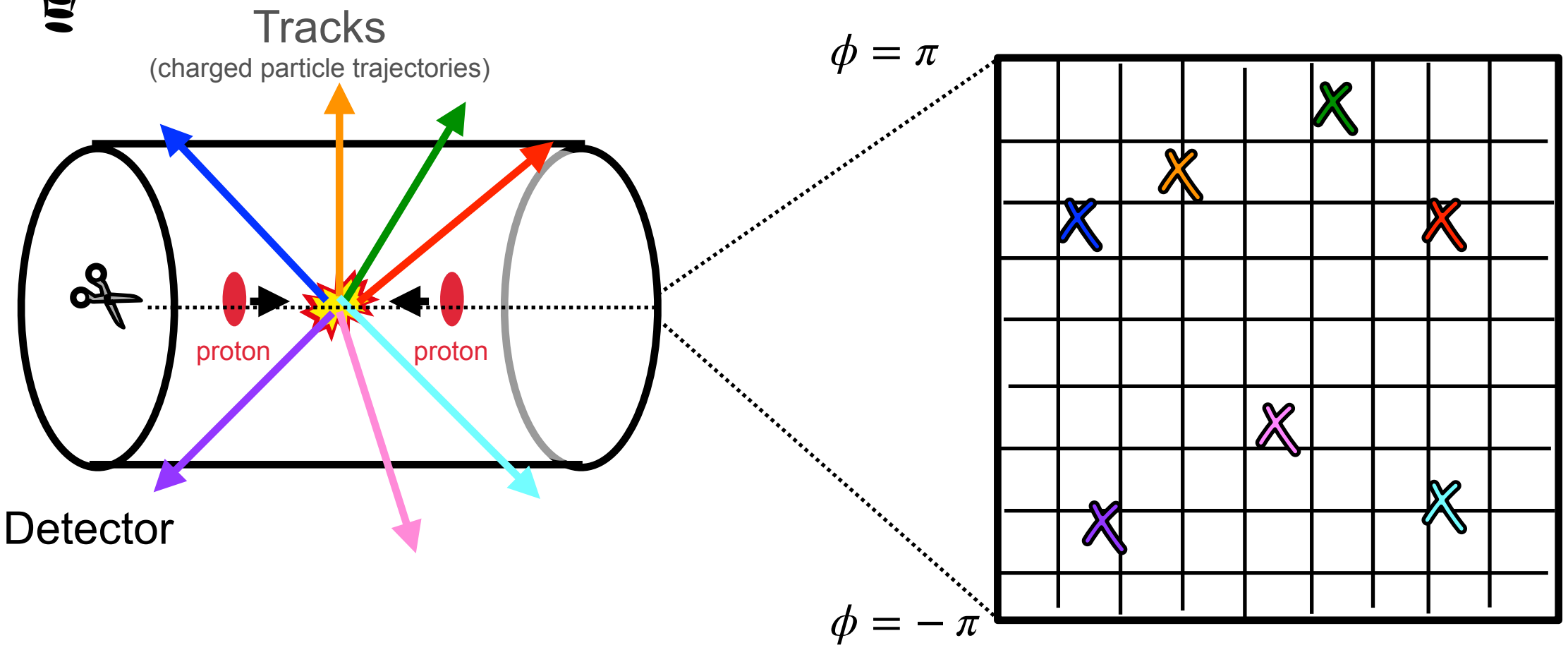


Detector

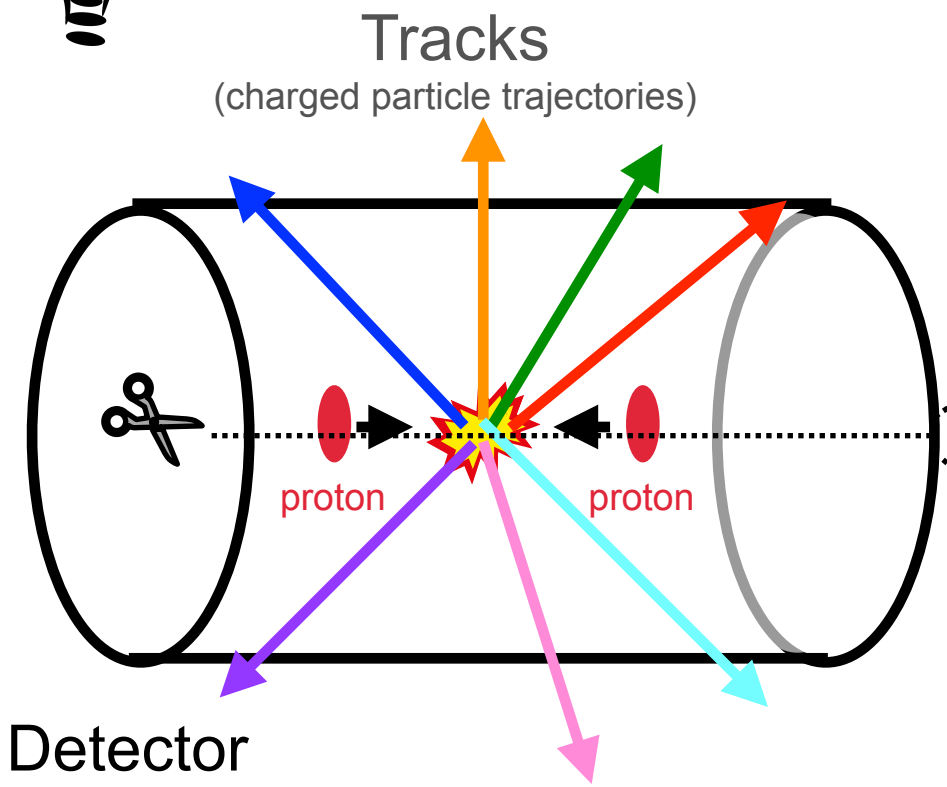
Padding… another application

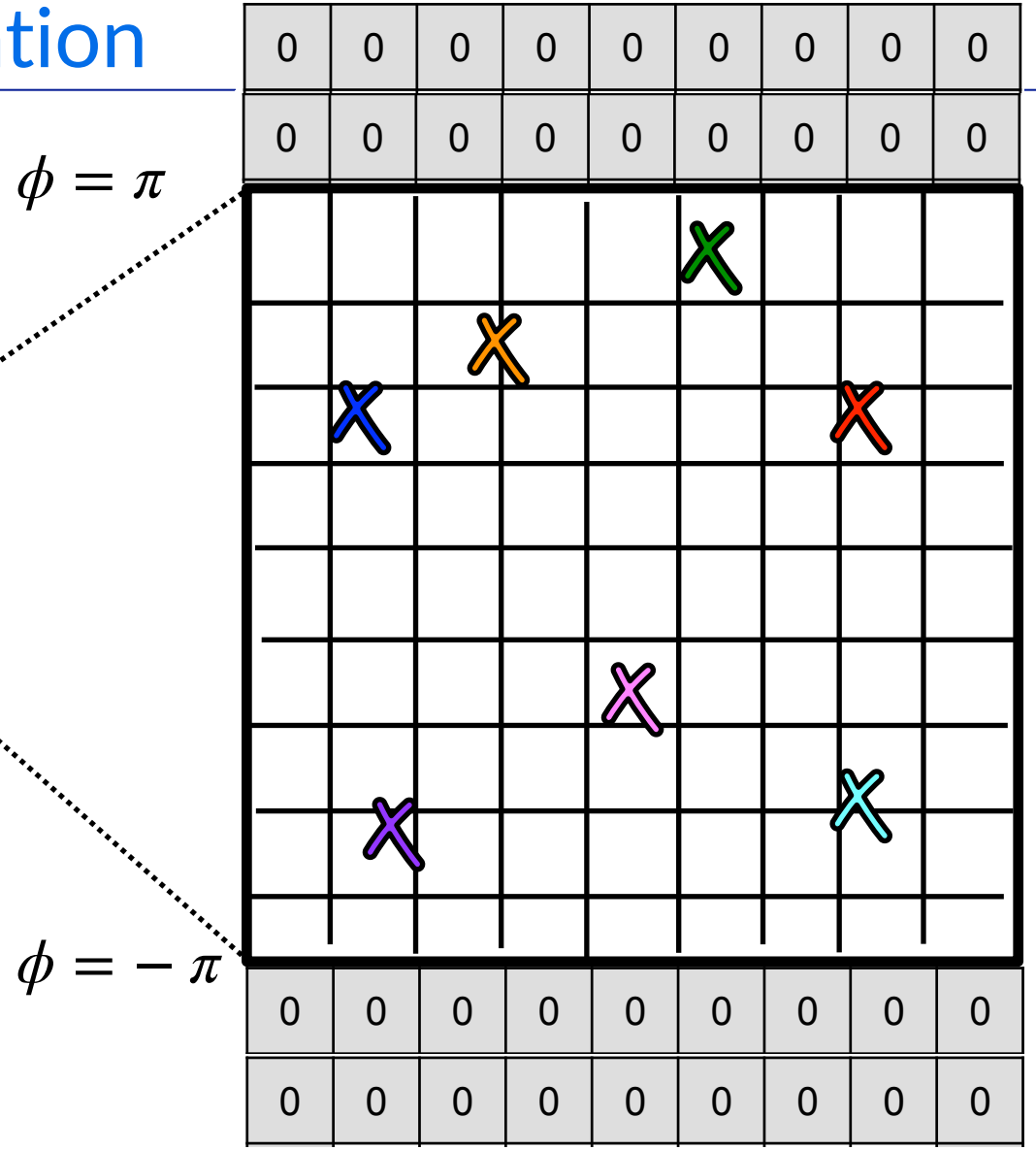


Padding... another application

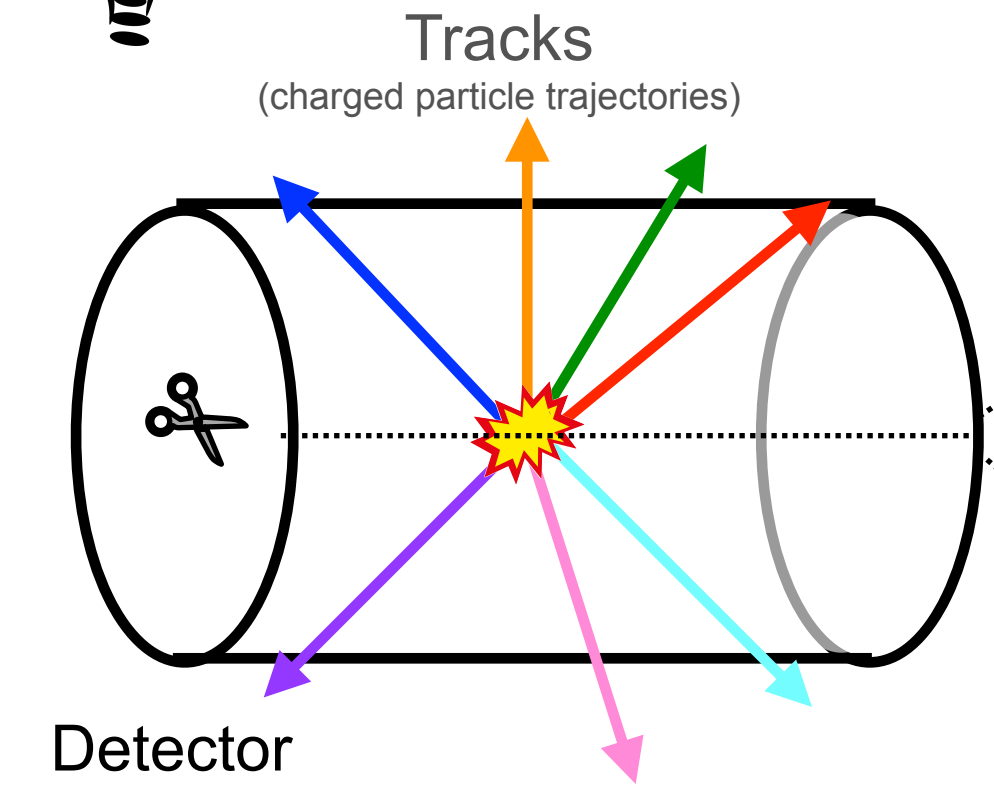


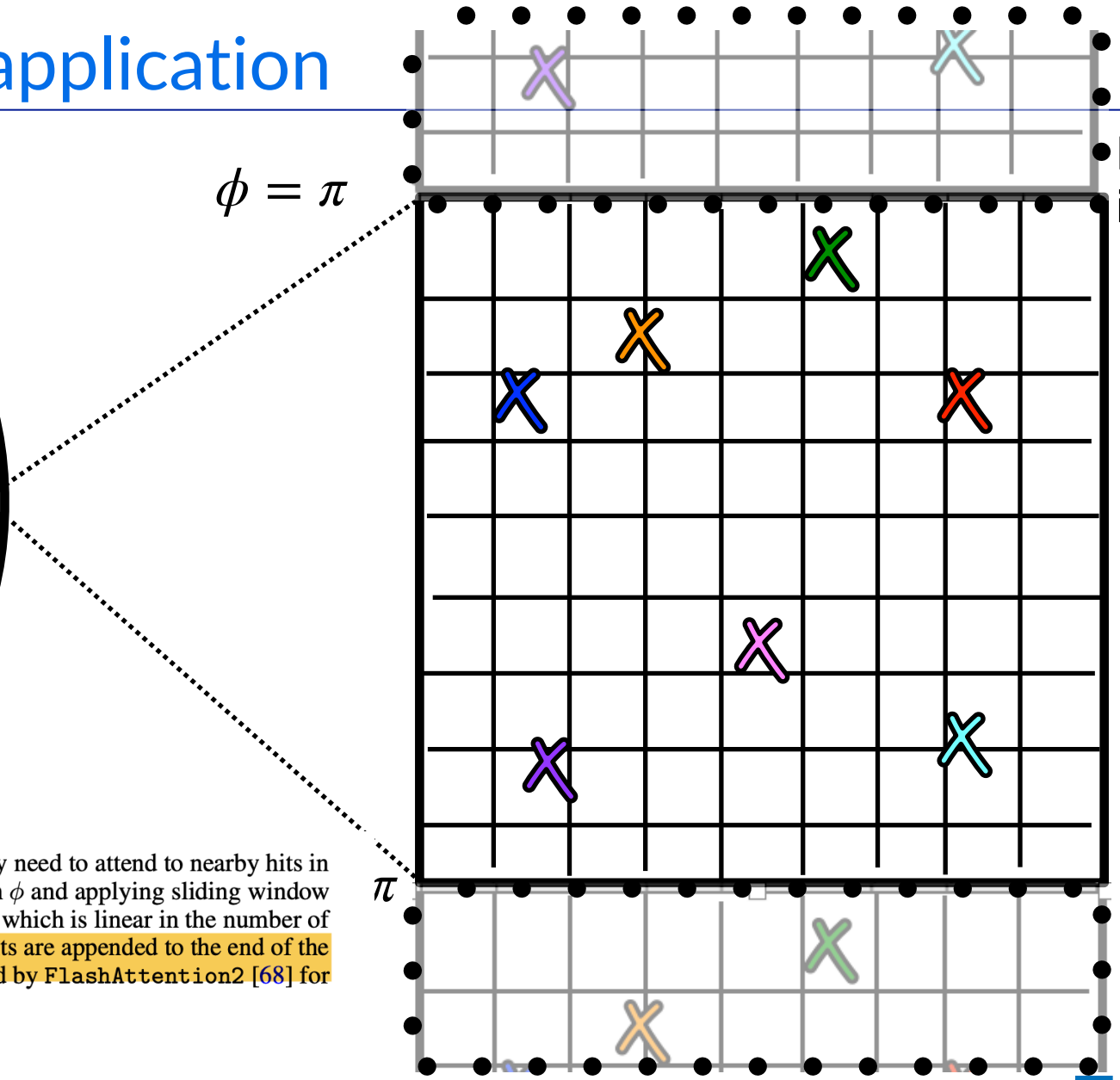
Padding... another application





Padding... another application

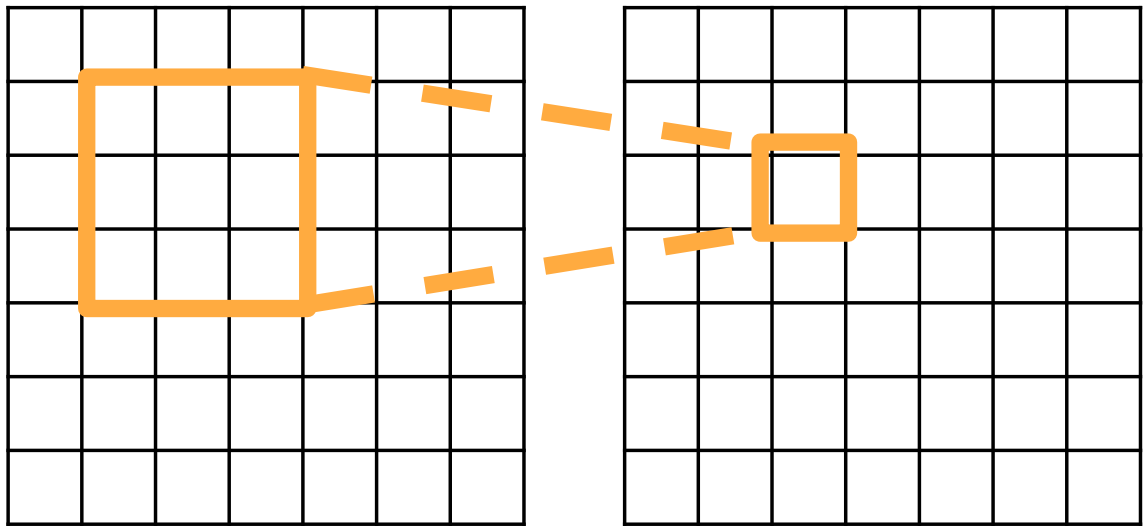




To neutralise the quadratic complexity of Transformers, we hypothesise that hits only need to attend to nearby hits in the azimuthal angle ϕ, and apply this powerful prior of ϕ-locality by ordering hits in ϕ and applying sliding window attention [67] with a window size w. This results in $\mathcal{O}(M \times w)$ complexity scaling, which is linear in the number of input hits M. To allow hits to communicate around the $\pm\pi$ boundary, the first $w/2$ hits are appended to the end of the sequence and vice versa. We further benefit from the fused attention kernels provided by `FlashAttention2` [68] for efficient computation, and `SwiGLU` activation [69] for improved performance.

Receptive Fields

For convolution with **kernel size K**, each element in the output depends on a K x K receptive field in the input



Input Output

Slide inspiration: Justin Johnson

Concept integration

For applying two 3x3 filters in succession, what's the **receptive field** from applying *two* sequential conv layers?

A) 3x3

B) 4x4

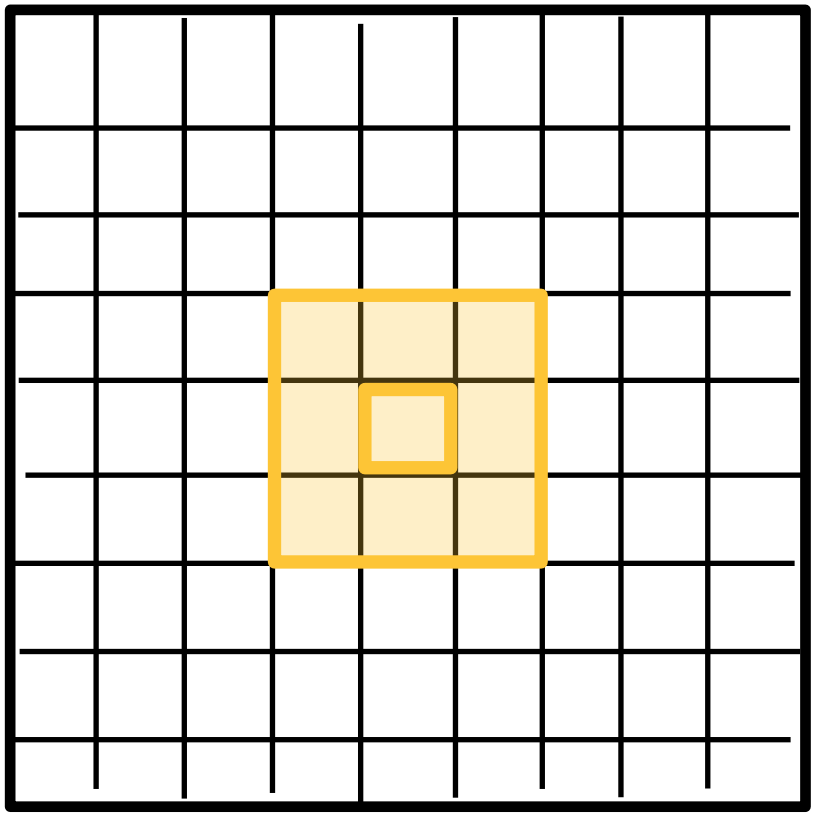
C) 5x5

D) 6x6

E) 7x7

F) The entire image

PollEv.com /nicolehartman968



Concept integration

For applying two 3x3 filters in succession, what's the **receptive field** from applying *two* sequential conv layers?

A) 3x3

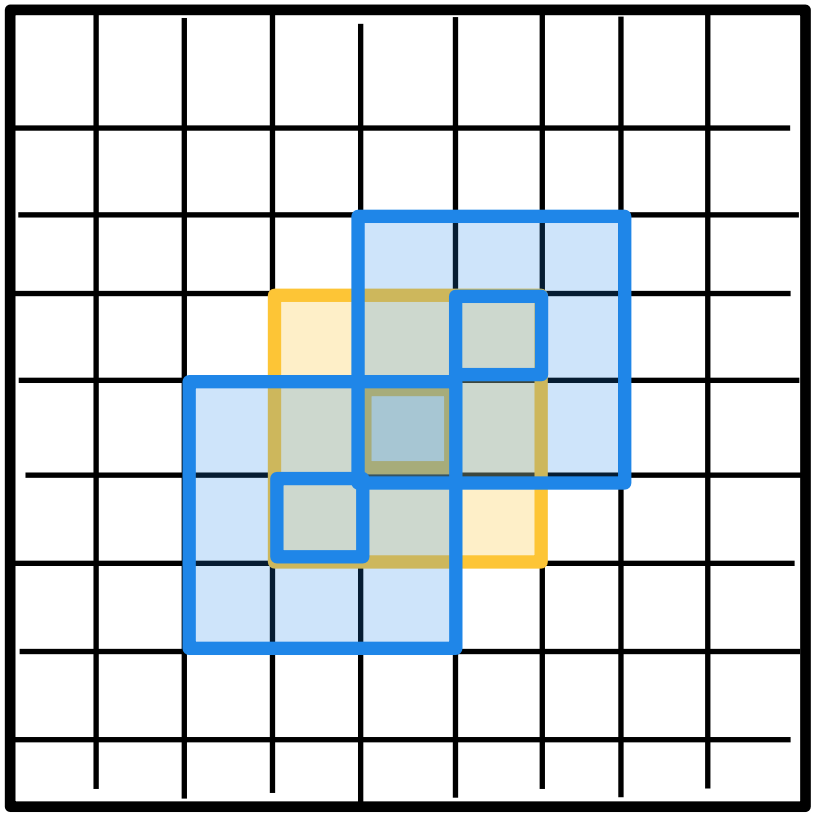
B) 4x4

C) 5x5

D) 6x6

E) 7x7

F) The entire image



Concept integration

For applying two 3x3 filters in succession, what's the **receptive field** from applying *two* sequential conv layers?

A) 3x3

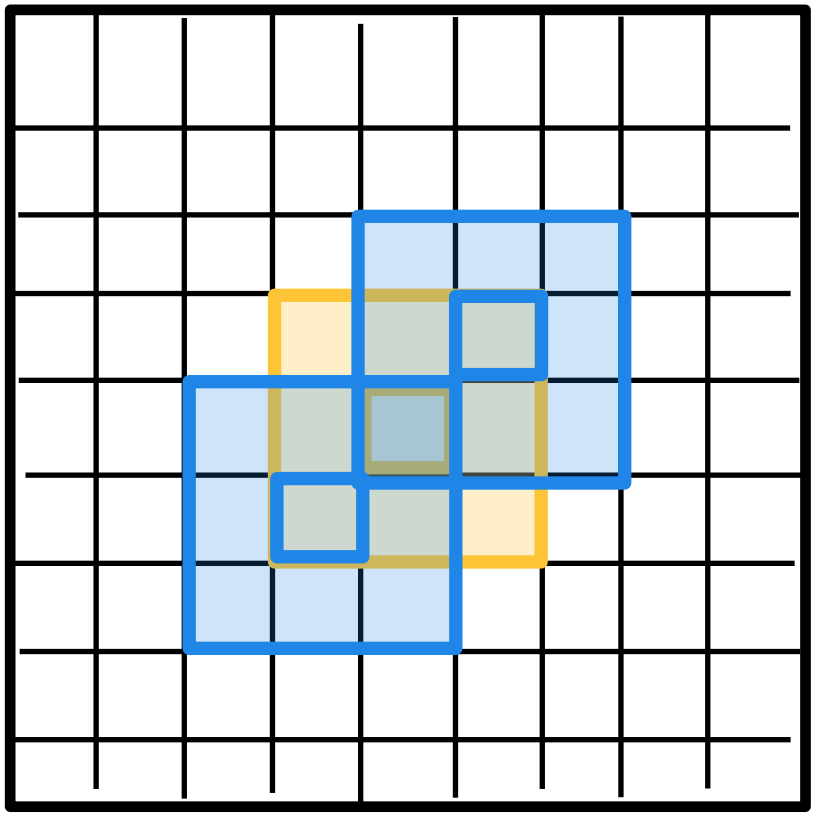
B) 4x4

C) 5x5

D) 6x6

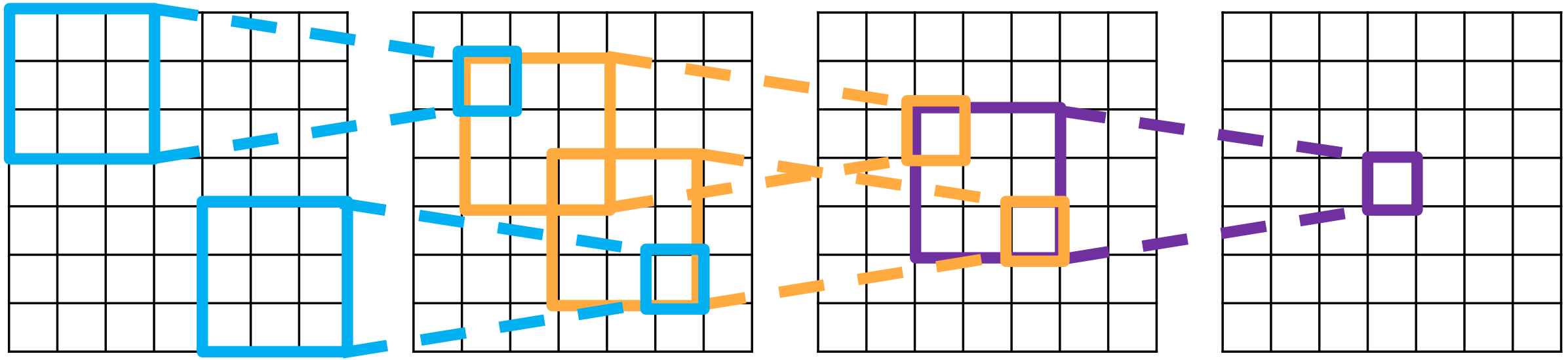
E) 7x7

F) The entire image



Receptive Fields

Each successive convolution adds K – 1 to the receptive field size
With L layers the receptive field size is 1 + L * (K – 1)



Input

Output

Be careful – “receptive field in the input” vs. “receptive field in the previous layer”

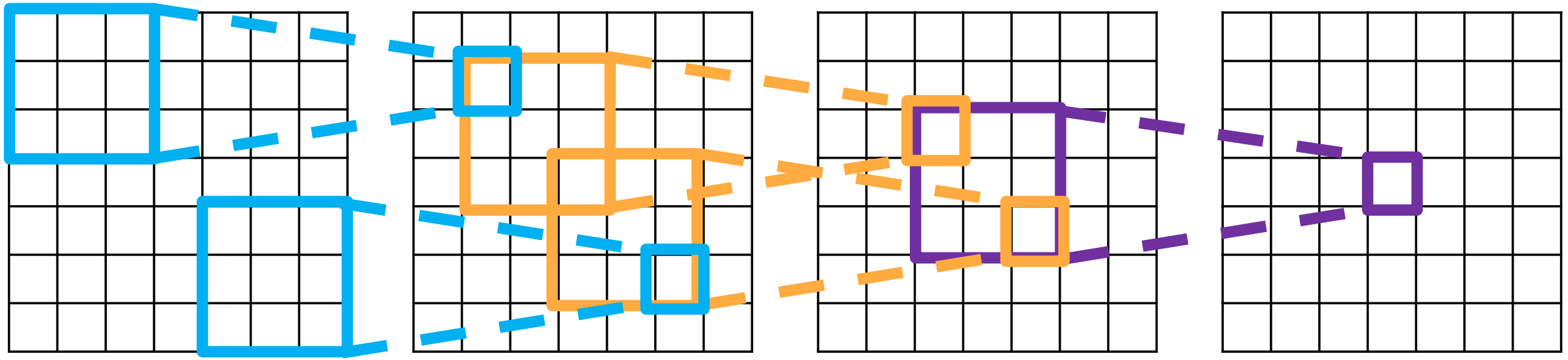
Slide inspiration: Justin Johnson

Receptive Fields

Sanity check:

L = 1: receptive field
1 + 1 * (K-1) = **3**
(or 3x3) ✅

Each successive convolution adds K – 1 to the receptive field size
With L layers the receptive field size is 1 + L * (K – 1)



Input

Output

Be careful – “receptive field in the input” vs. “receptive field in the previous layer”

Slide inspiration: Justin Johnson

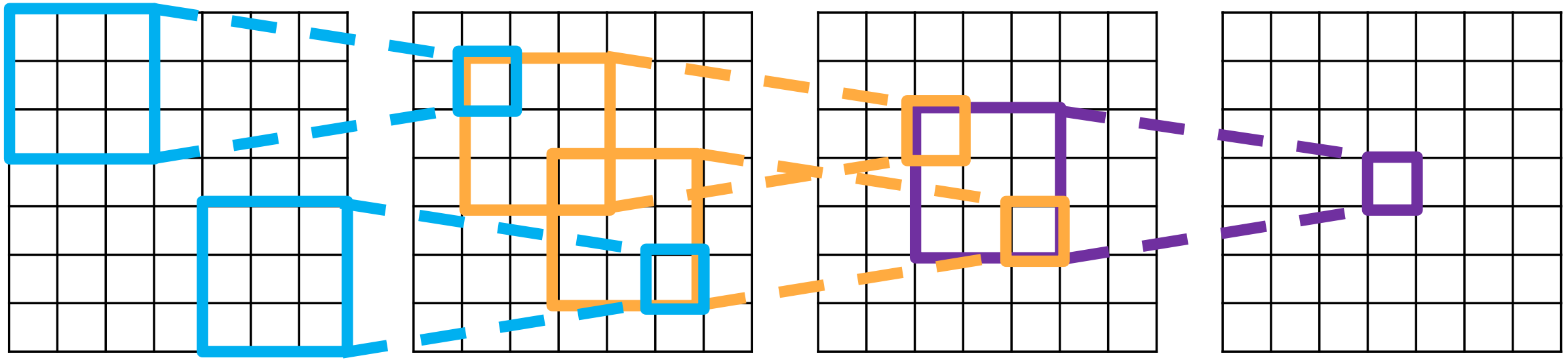
Receptive Fields

Sanity check:

L = 1: receptive field
1 + 1 * (K-1) = **3**
(or 3x3) ✅

L = 2:
1 + 2 * (K-1) = **5**
(or 5x5) ✅

Each successive convolution adds K – 1 to the receptive field size
With L layers the receptive field size is 1 + L * (K – 1)



Input

Output

Be careful – “receptive field in the input” vs. “receptive field in the previous layer”

Slide inspiration: Justin Johnson

Receptive Fields

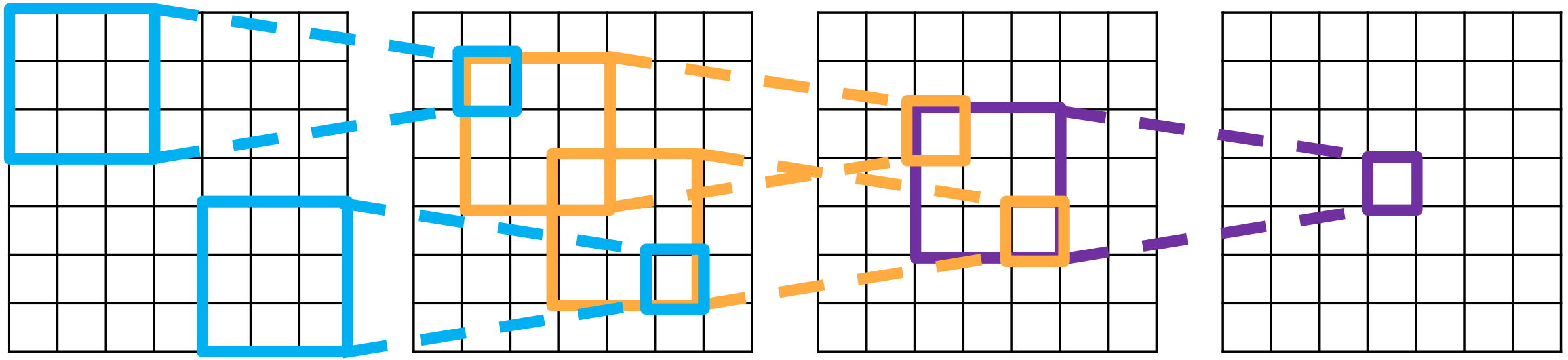
Sanity check:

L = 1: receptive field
1 + 1 * (K-1) = **3**
(or 3x3) ✅

L = 2:
1 + 2 * (K-1) = **5**
(or 5x5) ✅

L = 3: (this img)
1 + 3 * (K-1) = **7**
(or 7x7) (full img)

Each successive convolution adds K – 1 to the receptive field size
With L layers the receptive field size is 1 + L * (K – 1)



Input

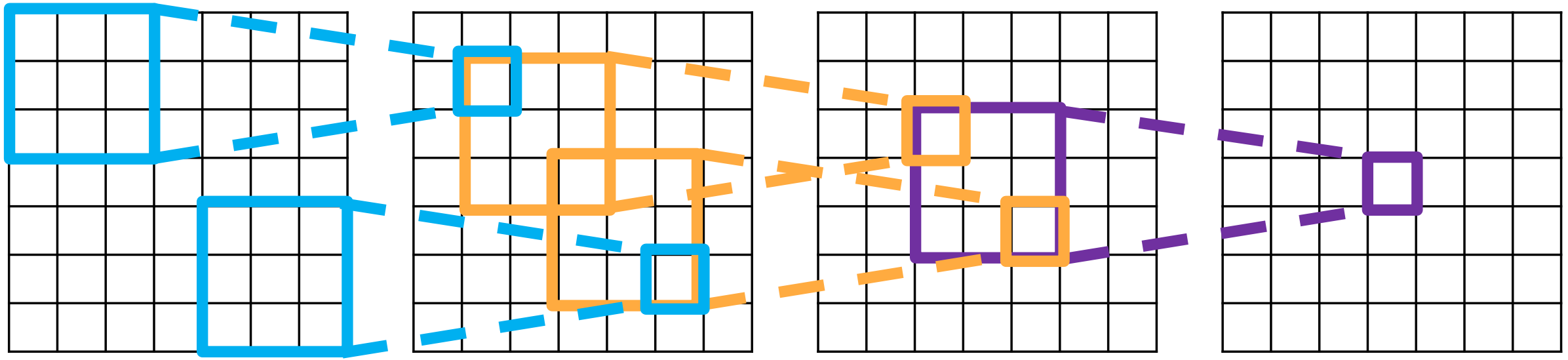
Output

Be careful – “receptive field in the input” vs. “receptive field in the previous layer”

Slide inspiration: Justin Johnson

Receptive Fields

Each successive convolution adds K – 1 to the receptive field size
With L layers the receptive field size is 1 + L * (K – 1)



Input

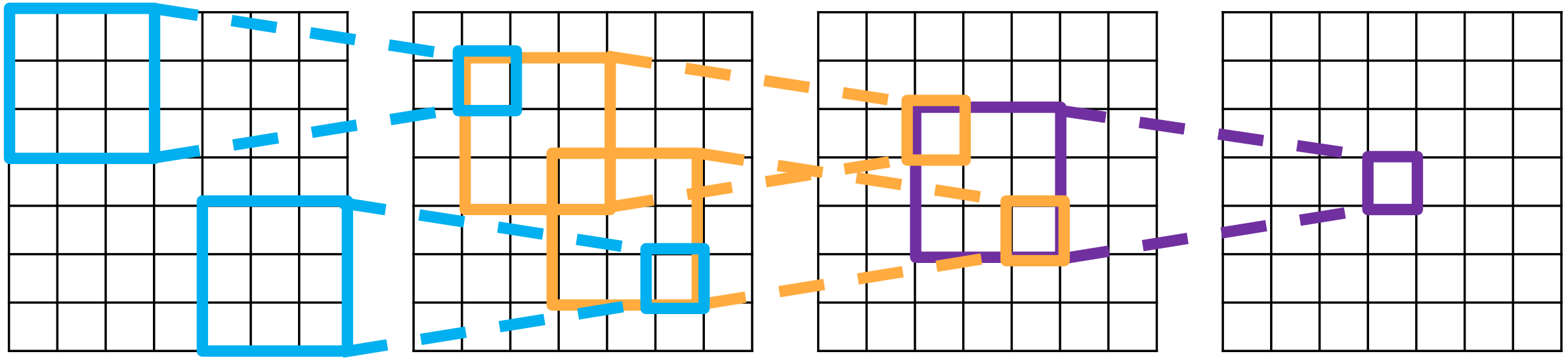
Problem: For large images we need many layers for each output to “see” the whole image image

Output

Slide inspiration: Justin Johnson

Receptive Fields

Each successive convolution adds K – 1 to the receptive field size
With L layers the receptive field size is 1 + L * (K – 1)



Input

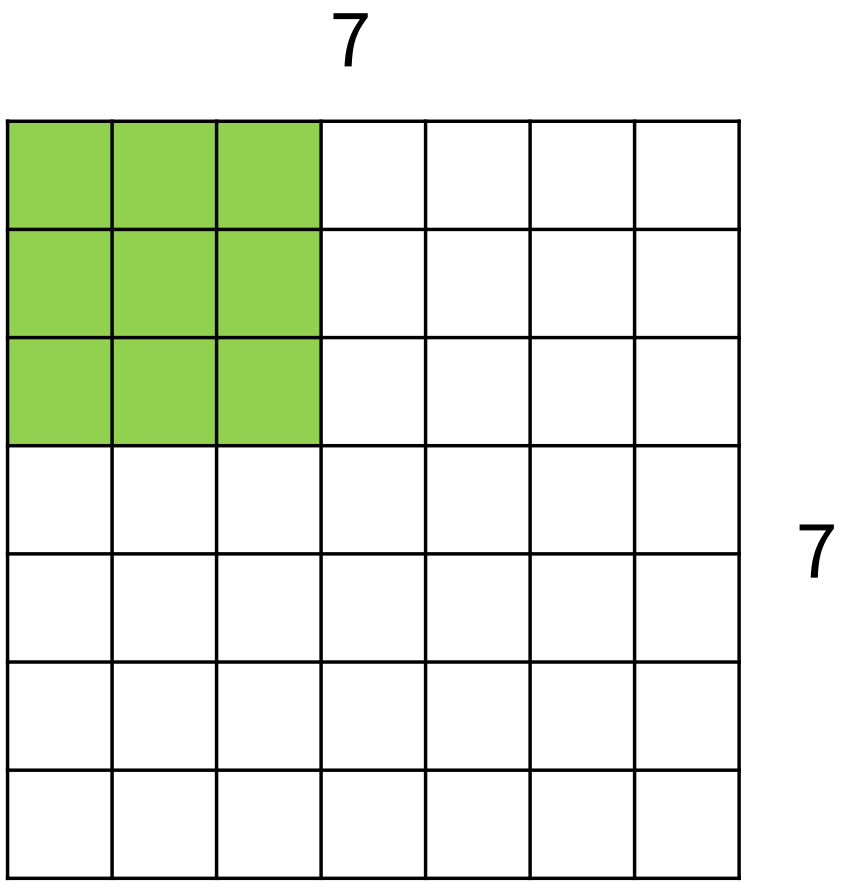
Output

Problem: For large images we need many layers for each output to “see” the whole image image

Solution: Downsample inside the network

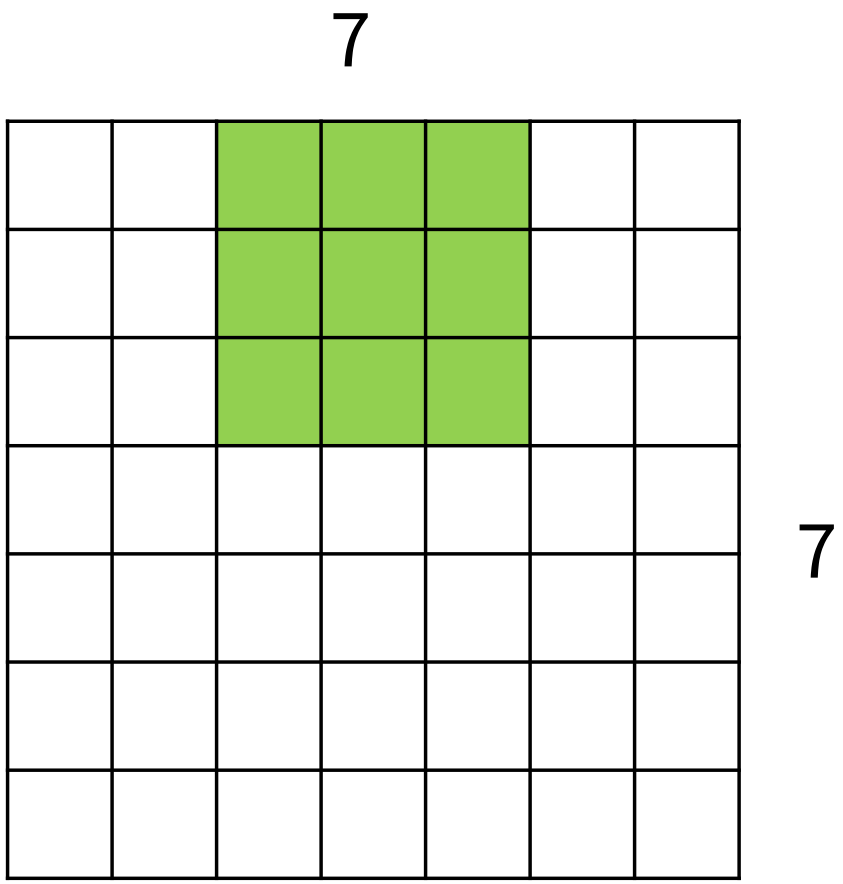
Slide inspiration: Justin Johnson

Strided Convolution



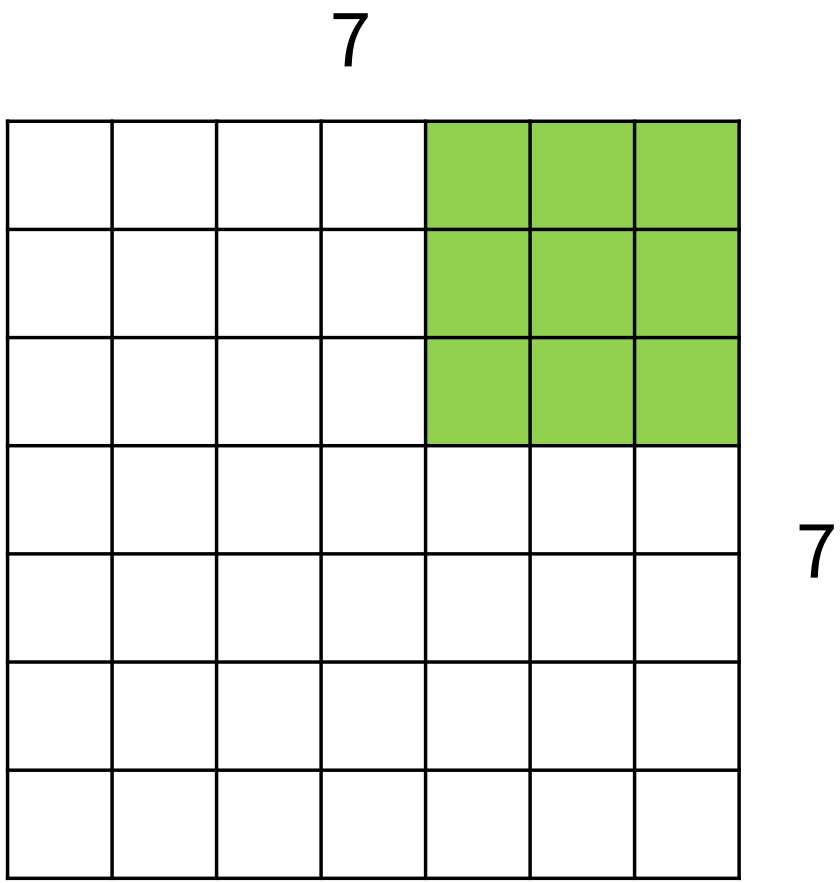
Input: 7x7
Filter: 3x3
Stride: 2

Strided Convolution



Input: 7x7
Filter: 3x3
Stride: 2

Strided Convolution



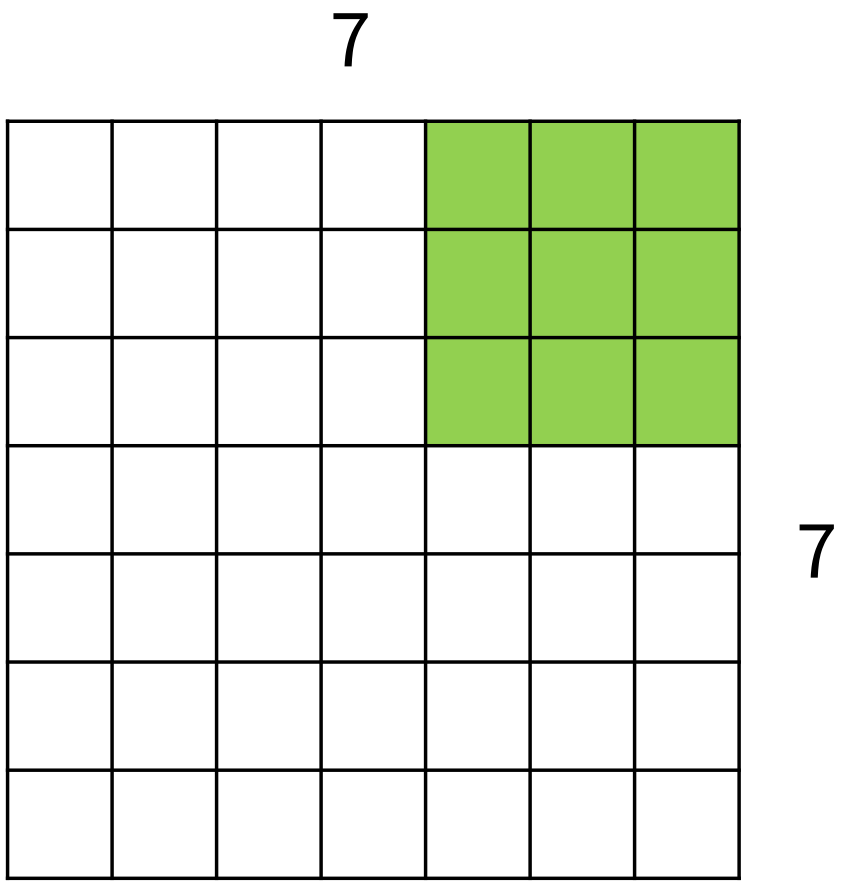
Input: 7x7

Filter: 3x3

Stride: 2

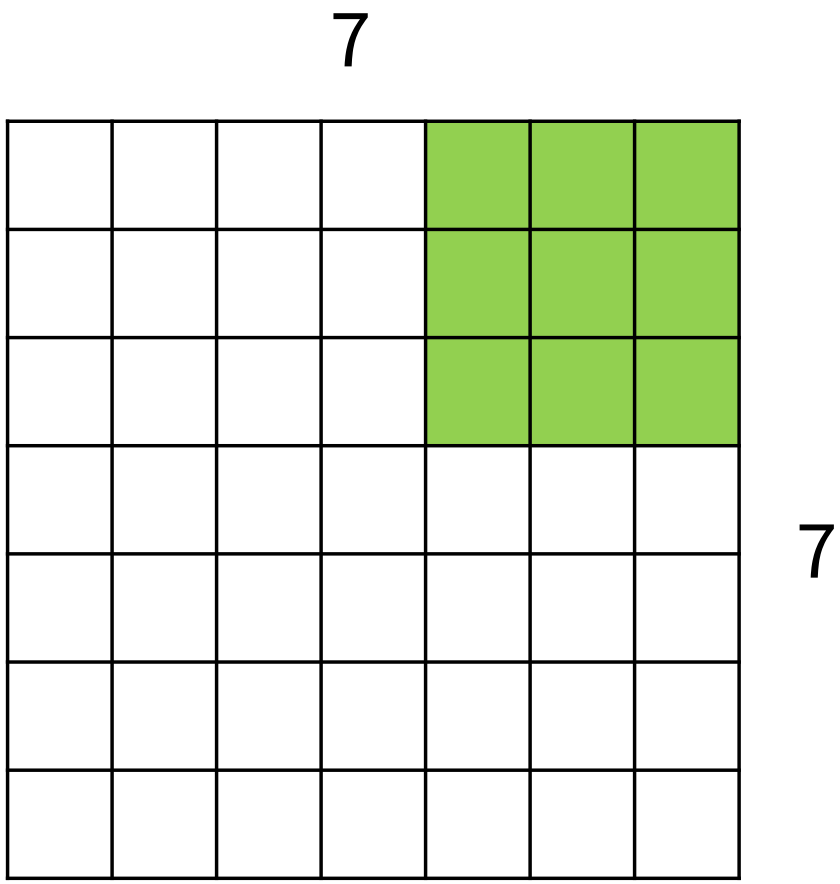
Output: ??

Strided Convolution



Input: 7x7
Filter: 3x3
Stride: 2
Output: 3x3

Strided Convolution



Input: 7x7
Filter: 3x3
Stride: 2
Output: 3x3

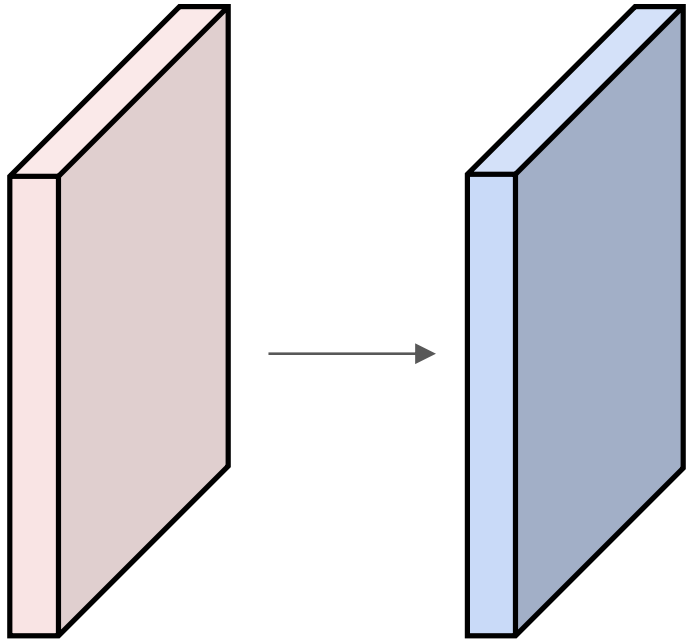
In general:
Input: W
Filter: K
Padding: P
Stride: S

→ Output:
(W – K + 2P) / S + 1

Convolution Example

Input volume: 3 x 32 x 32
10 5x5 filters with stride 1, pad 2

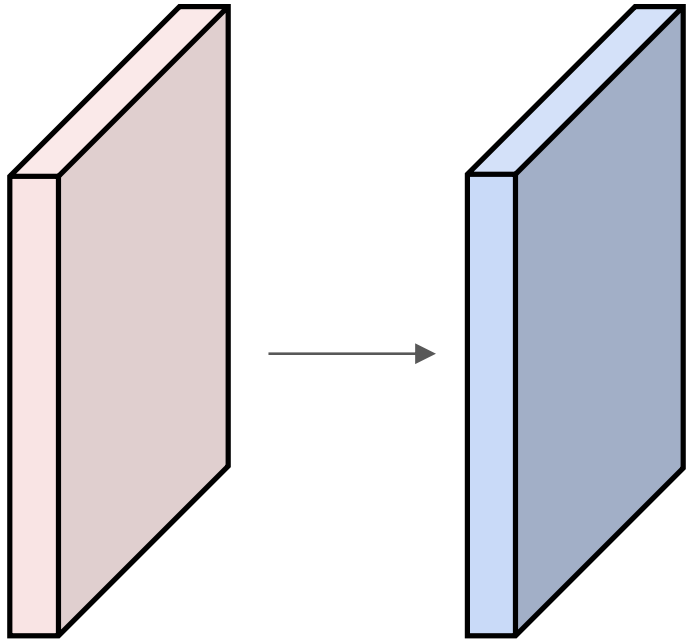
Output volume size: ?



Convolution Example

Input volume: 3 x **32** x **32**
10 **5x5** filters with stride **1**, pad **2**

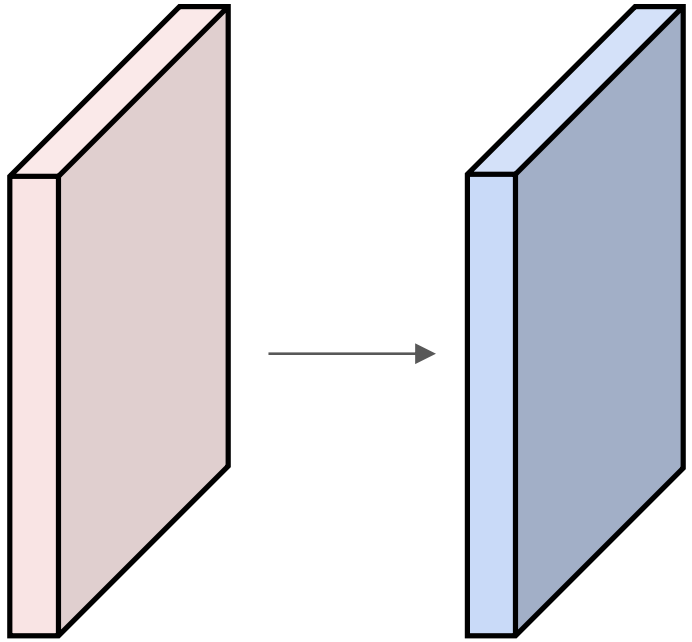
Output volume size: **10** x **32** x **32**
32 = (**32**+2*__2__-**5**)/**1**+1



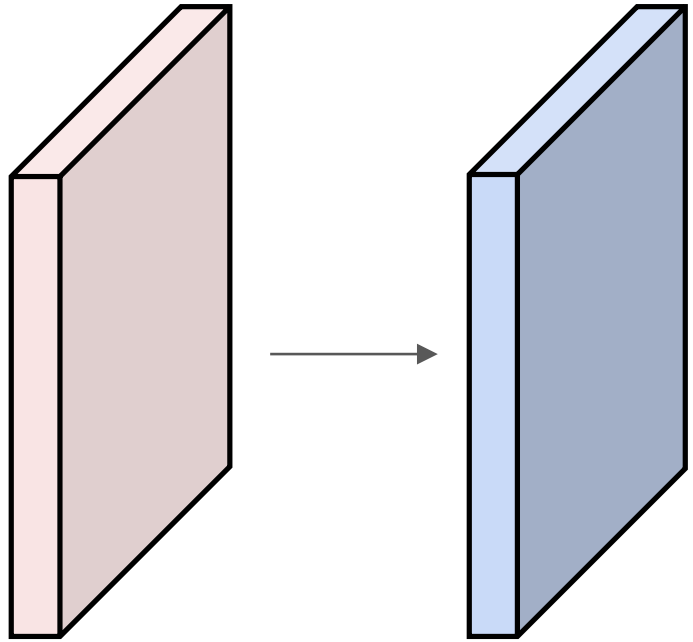
Convolution Example

Input volume: 3 x 32 x 32
10 5x5 filters with stride 1, pad 2

Output volume size: 10 x 32 x 32
Number of learnable parameters: ?



Convolution Example



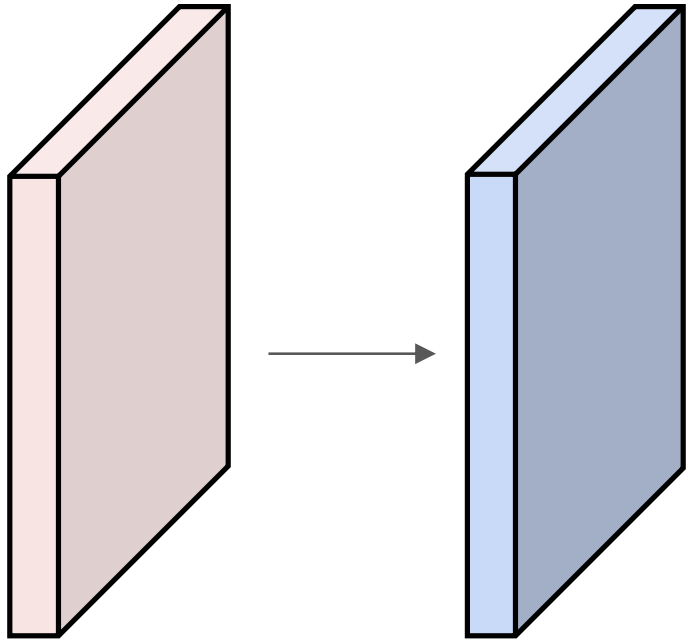
Input volume: **3** x 32 x 32
10 **5**x**5** filters with stride 1, pad 2

Output volume size: 10 x 32 x 32
Number of learnable parameters: 760
Parameters per filter: **3*****5*****5** + 1 (for bias) = **76**
10 filters, so total is **10** * **76** = **760**

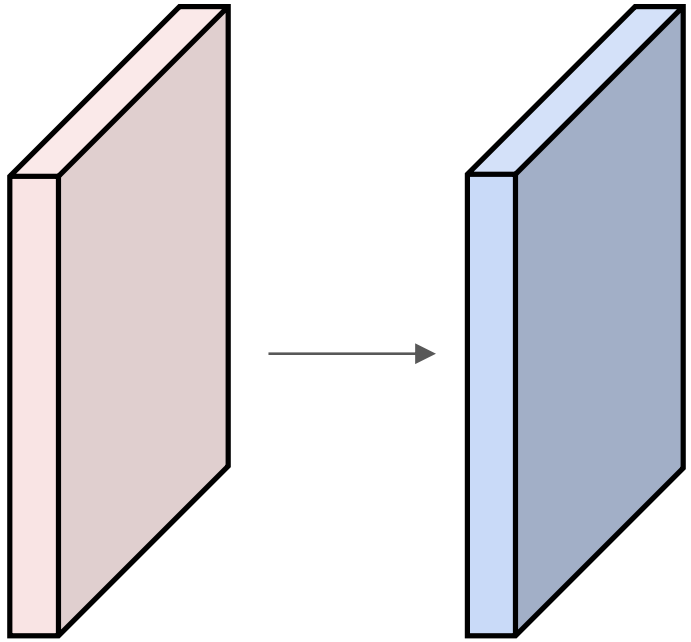
Convolution Example

Input volume: 3 x 32 x 32
10 5x5 filters with stride 1, pad 2

Output volume size: 10 x 32 x 32
Number of learnable parameters: 760
Number of multiply-add operations?



Convolution Example



Input volume: **3** x 32 x 32
10 **5x5** filters with stride 1, pad 2

Output volume size: **10 x 32 x 32**
Number of learnable parameters: 760
Number of multiply-add operations: **768,000**
10*32*32 = 10,240 outputs
Each output is the inner product of two **3**x**5x5** tensors (75 elems)
Total = 75*10240 = **768K**

Convolution Summary

Input: C_{in} x H x W
Hyperparameters:
- **Kernel size**: K_H x K_W
- **Number filters**: C_{out}
- **Padding**: P
- **Stride**: S

Weight matrix: C_{out} x C_{in} x K_H x K_W
giving C_{out} filters of size C_{in} x K_H x K_W
Bias vector: C_{out}
Output size: C_{out} x H' x W' where:
- H' = (H – K + 2P) / S + 1
- W' = (W – K + 2P) / S + 1

Common settings:
$K_H = K_W$ (Small square filters)
P = (K – 1) / 2 ("Same" padding)
C_{in}, C_{out} = 32, 64, 128, 256 (powers of 2)
K = 3, P = 1, S = 1 (3x3 conv)
K = 5, P = 2, S = 1 (5x5 conv)
K = 1, P = 0, S = 1 (1x1 conv)
K = 3, P = 1, S = 2 (Downsample by 2)

PyTorch Convolution Layer

CLASS torch.nn.Conv2d(*in_channels*, *out_channels*, *kernel_size*, *stride=1*, *padding=0*, *dilation=1*, *groups=1*, *bias=True*, *padding_mode='zeros'*, *device=None*, *dtype=None*) [SOURCE]

Applies a 2D convolution over an input signal composed of several input planes.

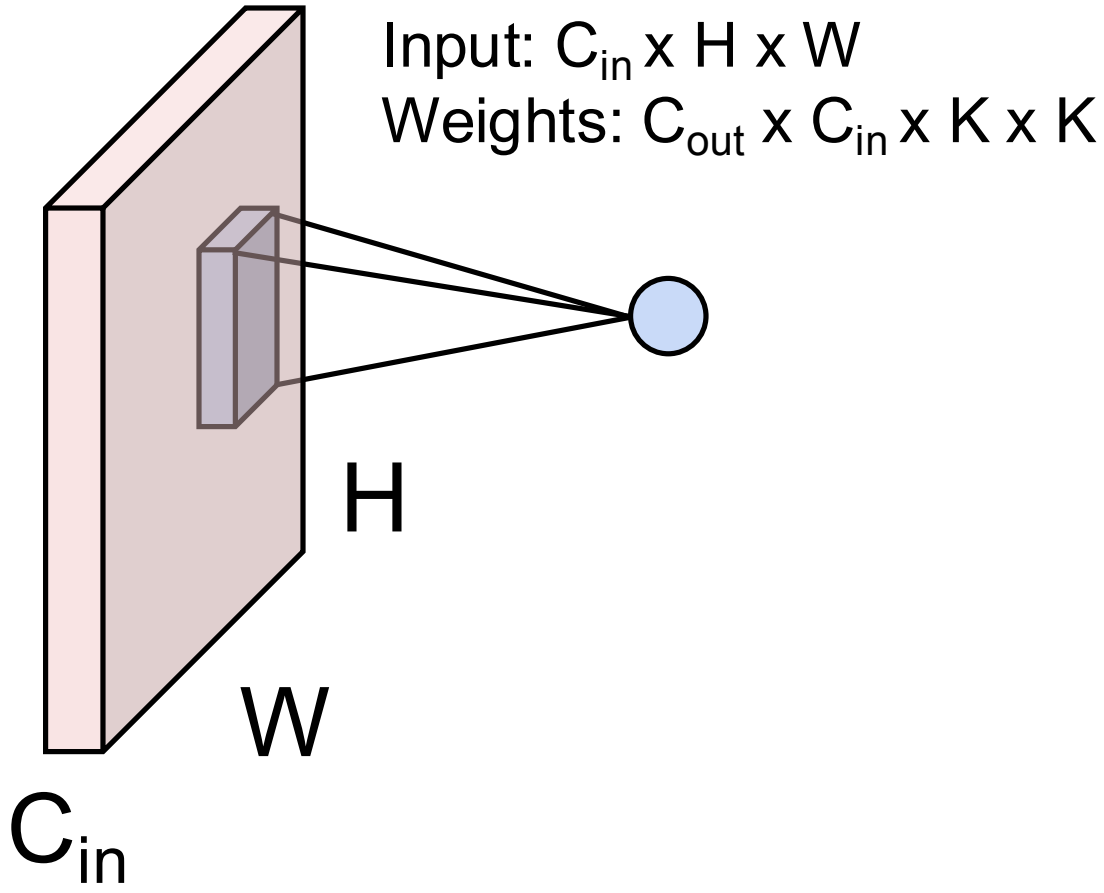
In the simplest case, the output value of the layer with input size (N, C_{in}, H, W) and output $(N, C_{\text{out}}, H_{\text{out}}, W_{\text{out}})$ can be precisely described as:

$$\text{out}(N_i, C_{\text{out}_j}) = \text{bias}(C_{\text{out}_j}) + \sum_{k=0}^{C_{\text{in}}-1} \text{weight}(C_{\text{out}_j}, k) \star \text{input}(N_i, k)$$

We didn't talk about groups or dilation…

Other Types of Convolution

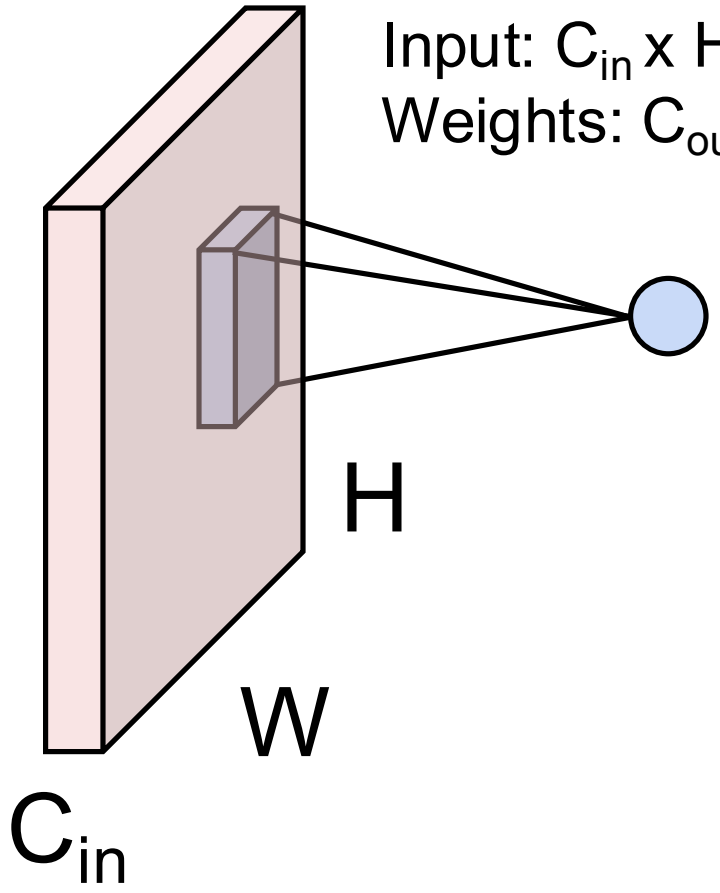
So far: 2D Convolution



Other Types of Convolution

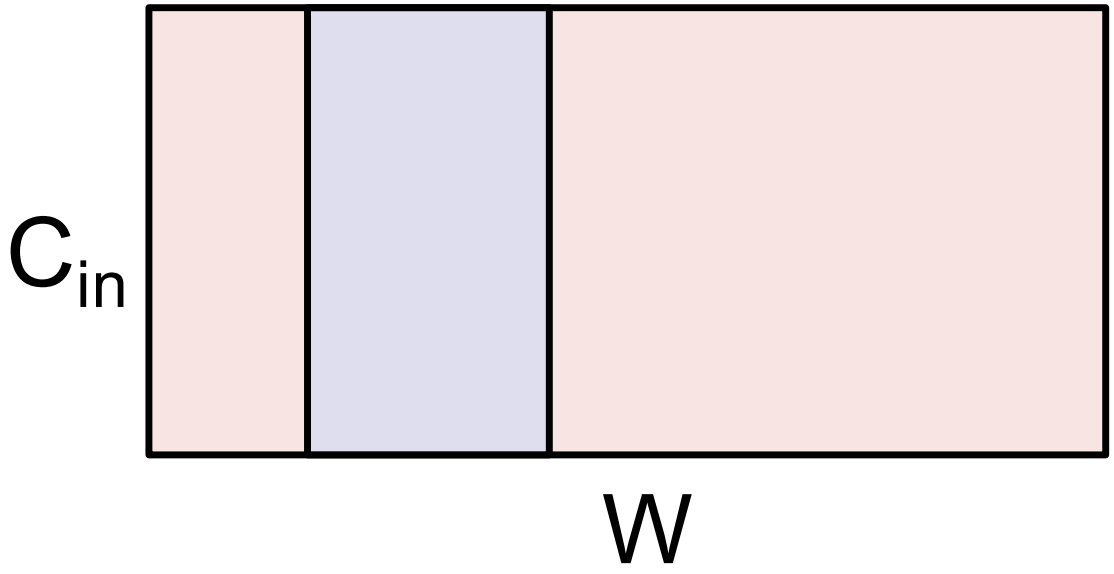
So far: 2D Convolution

Input: C_{in} x H x W
Weights: C_{out} x C_{in} x K x K



1D Convolution

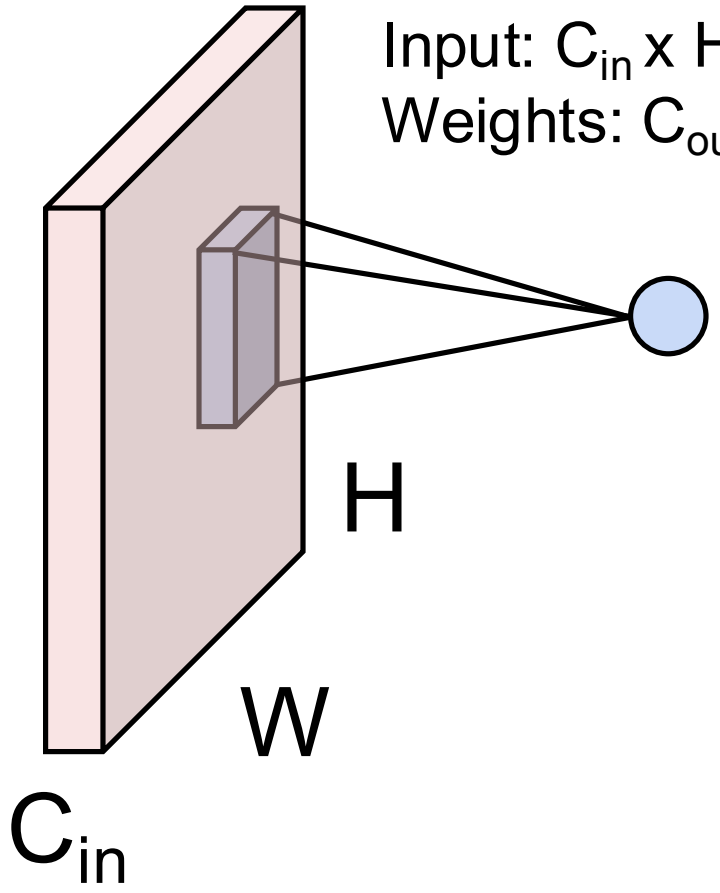
Input: C_{in} x W
Weights: C_{out} x C_{in} x K



Other Types of Convolution

So far: 2D Convolution

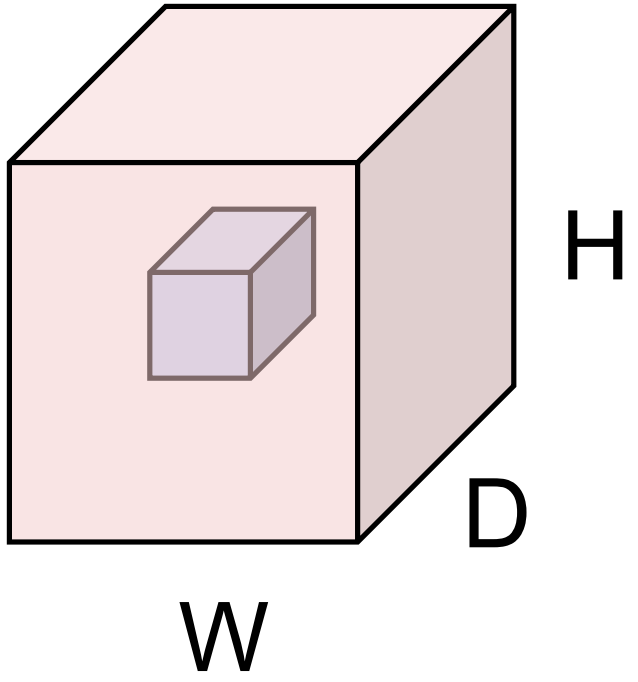
Input: C_{in} x H x W
Weights: C_{out} x C_{in} x K x K



3D Convolution

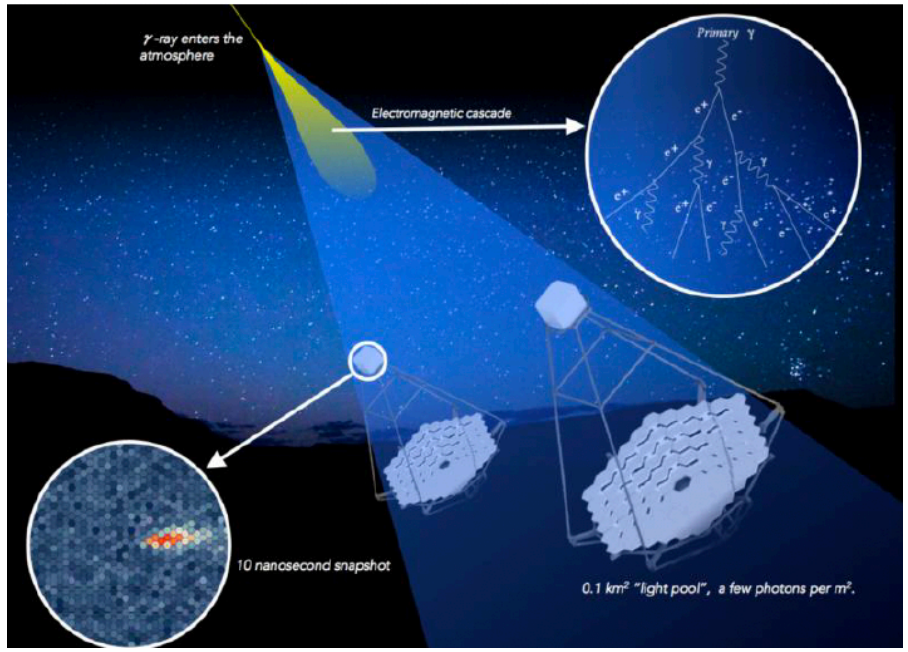
Input: C_{in} x H x W x D
Weights: C_{out} x C_{in} x K x K x K

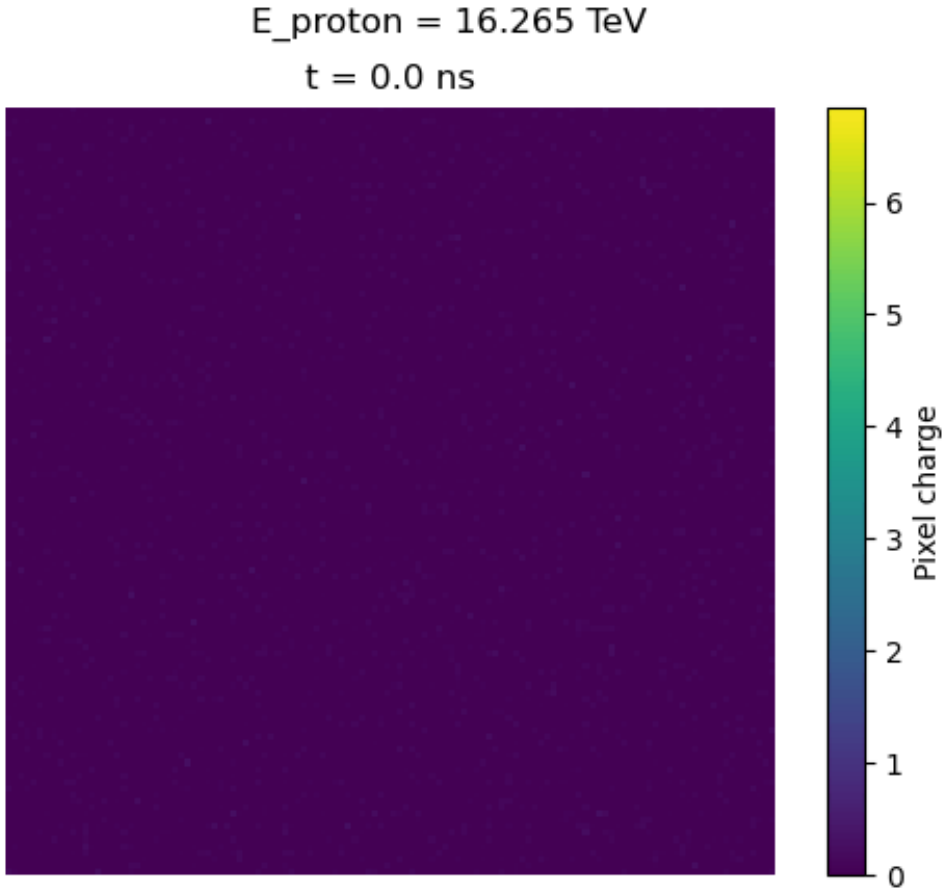
C_{in}-dim vector
at each point
in the volume



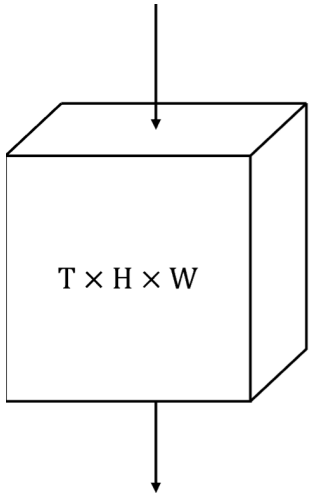
Elli Jobst (MPP)

3d cnn for gamma-ray atronomy





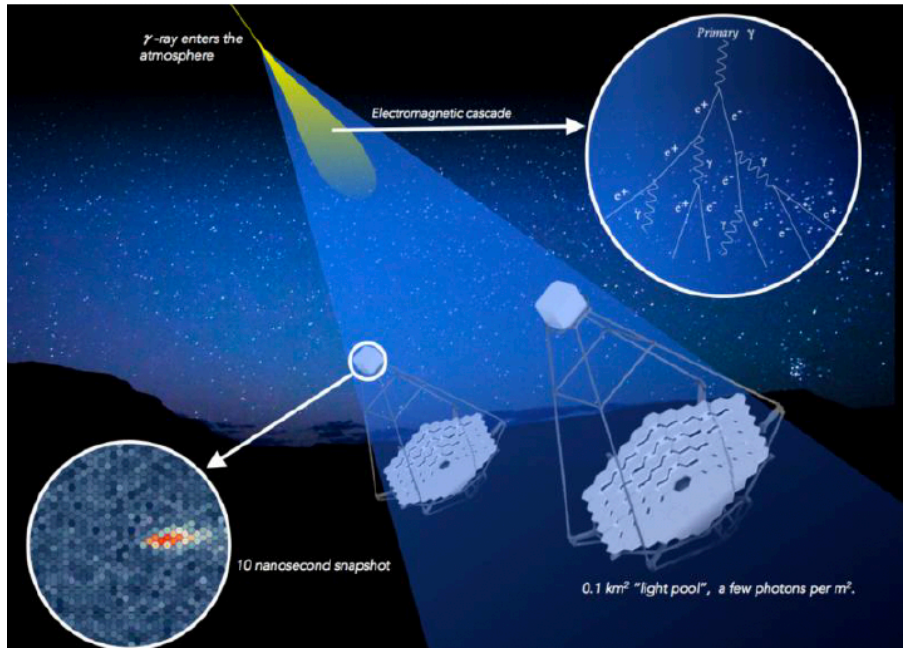
Analyze movies with a 3D CNN to
- Classify proton vs. photon
- Regress the energy and direction

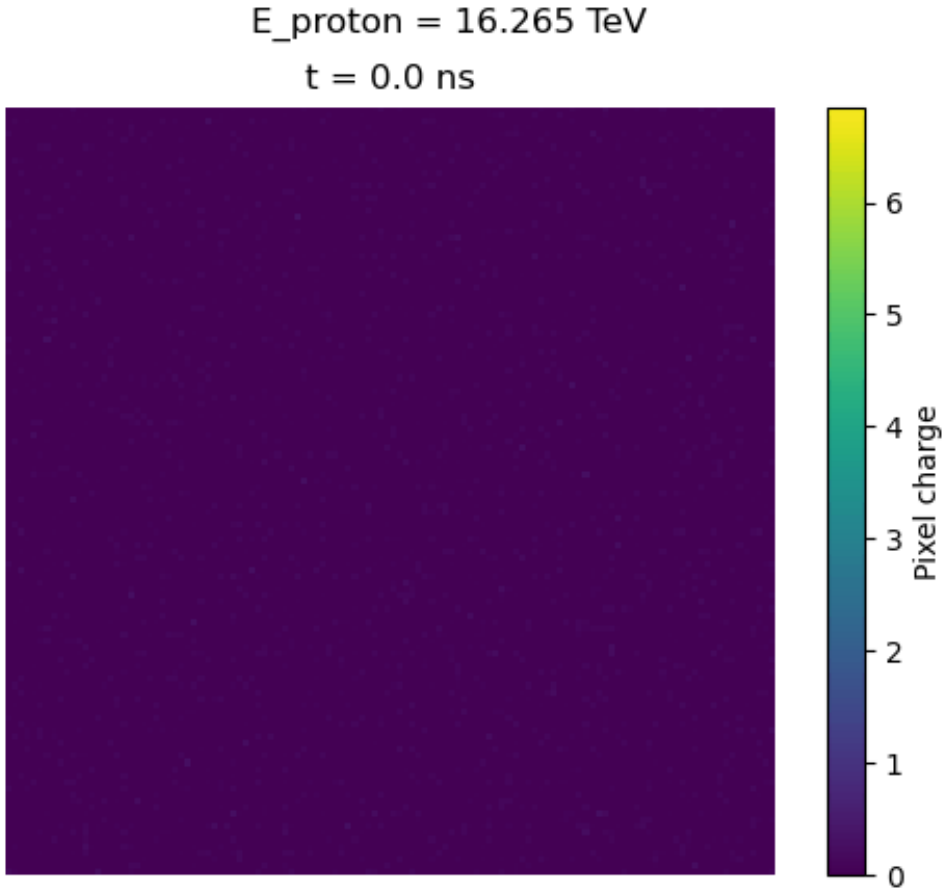


* Note, much better results with vision transformers!!
But this was a strong baseline

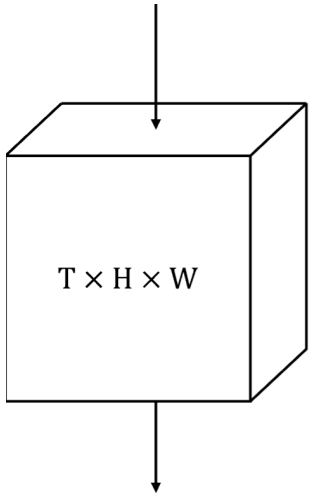
Elli Jobst (MPP)

3d cnn for gamma-ray atronomy



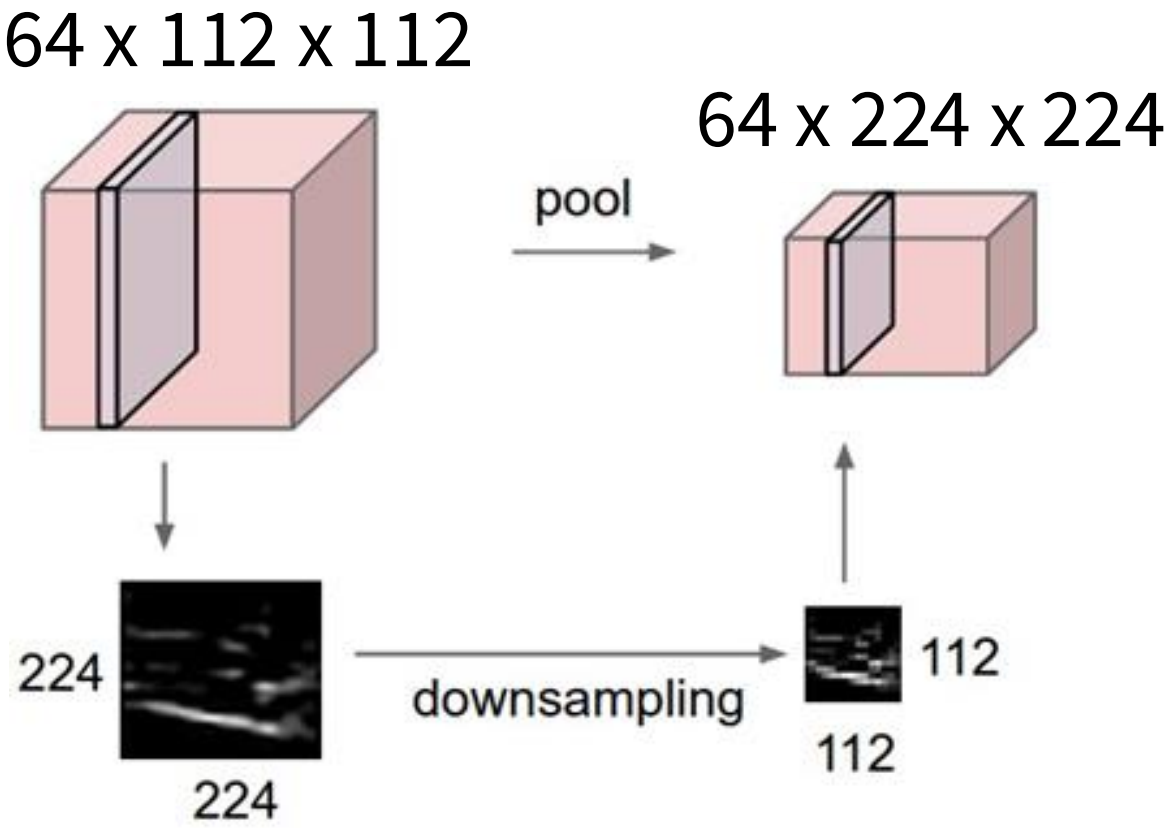


Analyze movies with a 3D CNN to
- Classify proton vs. photon
- Regress the energy and direction



* Note, much better results with vision transformers!!
But this was a strong baseline

Pooling Layers: Another way to downsample



Given an input C x H x W,
downsample each 1 x H x W plane

Hyperparameters:
Kernel Size
Stride
Pooling function

Pooling Layers: Another way to downsample

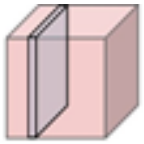
Single depth slice

x

1	1	2	4
5	**6**	7	**8**
3	2	1	0
1	2	3	**4**

y

64 x 224 x 224



Max pooling with 2x2 kernel size and stride 2

6	8
3	4

Gives **invariance** to small spatial shifts. No learnable parameters.

Pooling Summary

Input: C x H x W
Hyperparameters:
- **Kernel size**: K
- **Stride**: S
- **Pooling function**: max, avg

Output size: C x H' x W' where:
- H' = (H – K) / S + 1
- W' = (W – K) / S + 1

No learnable parameters

Common setting:
max, K=2, S=2 => Gives 2x downsampling

Pooling Summary

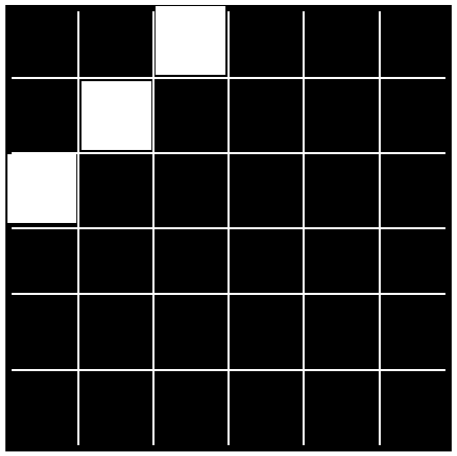
Input: C x H x W
Hyperparameters:
- **Kernel size**: K
- **Stride**: S
- **Pooling function**: max, avg

Output size: C x H' x W' where:
- H' = (H – K) / S + 1
- W' = (W – K) / S + 1

No learnable parameters

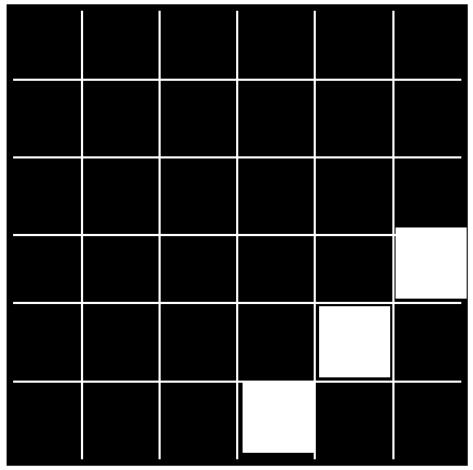
Common setting:
max, K=2, S=2 => Gives 2x downsampling

Remember how we opened the lecture?



$x \in \mathbb{R}^{5\times5}$

Warm-up: edge detector



$x' \in \mathbb{R}^{5\times5}$

Just tell me

Q: What weight vector, w', will optimally detect this pattern??

Soln: $w' = x$

If I want to have my perceptron able to classify *both* of these edges… need to apply both of the ***filters***: $w, w' \in \mathbb{R}^{5\times5}$

$$W = [w, w'] \in \mathbb{R}^{2\times5\times5}$$

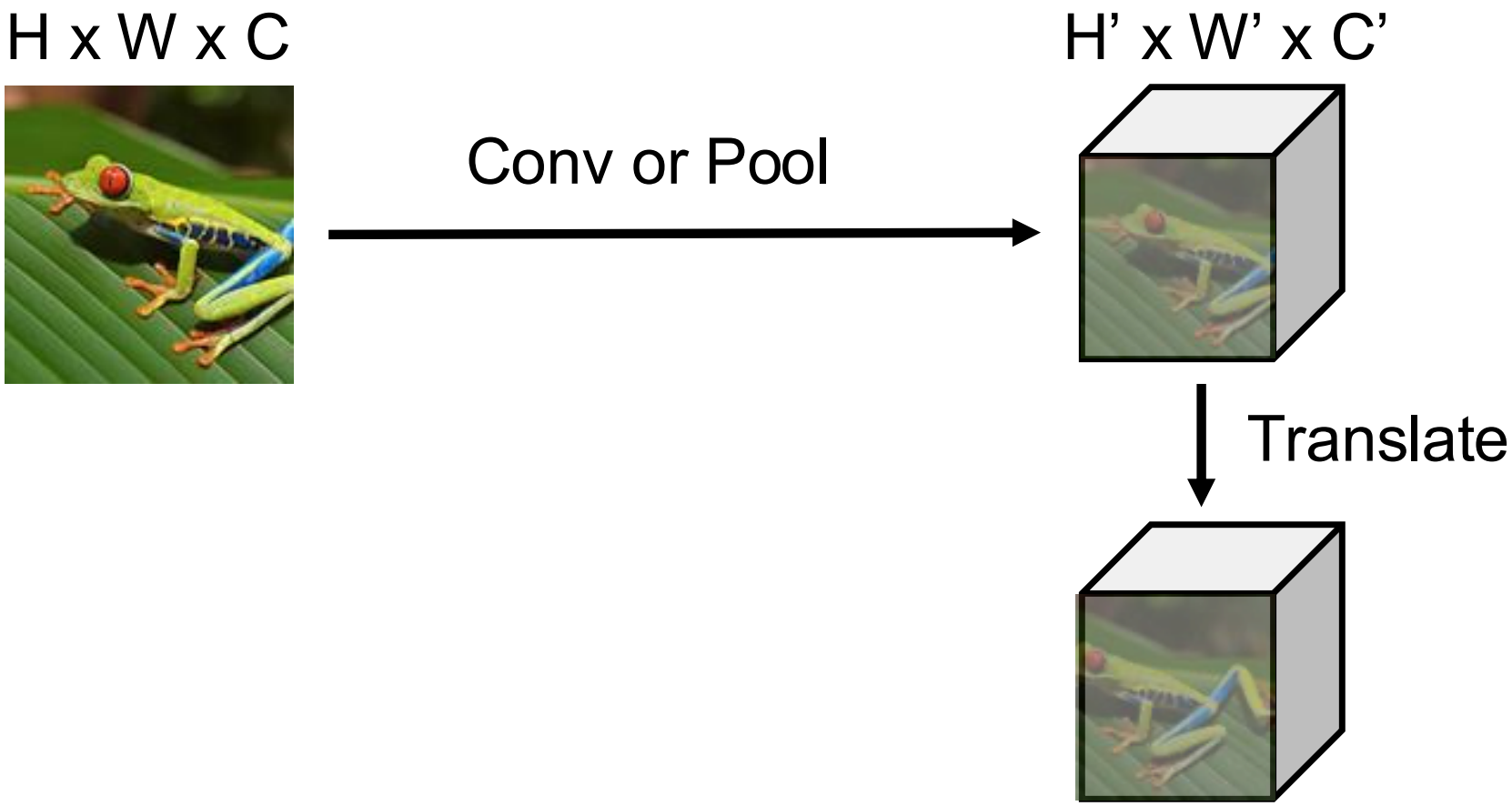
Then sum over the responses of all the filters:

$$z = \sum_{ijk} W_{ijk} x_{jk} \xrightarrow{\text{perceptron}} f_W(z) = \sigma(z)$$

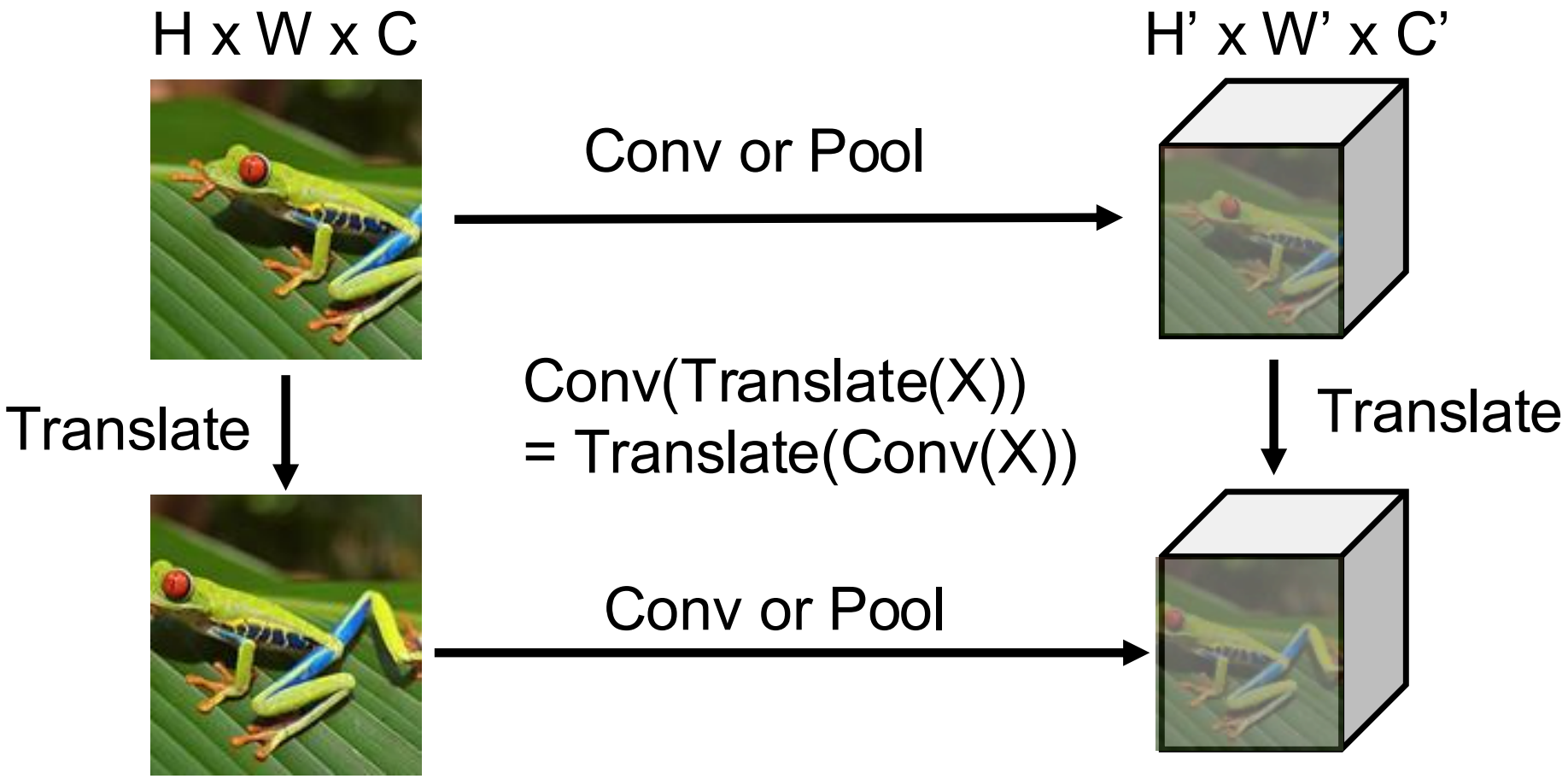
Very memory inefficient!! Can we do better?

How did we do?

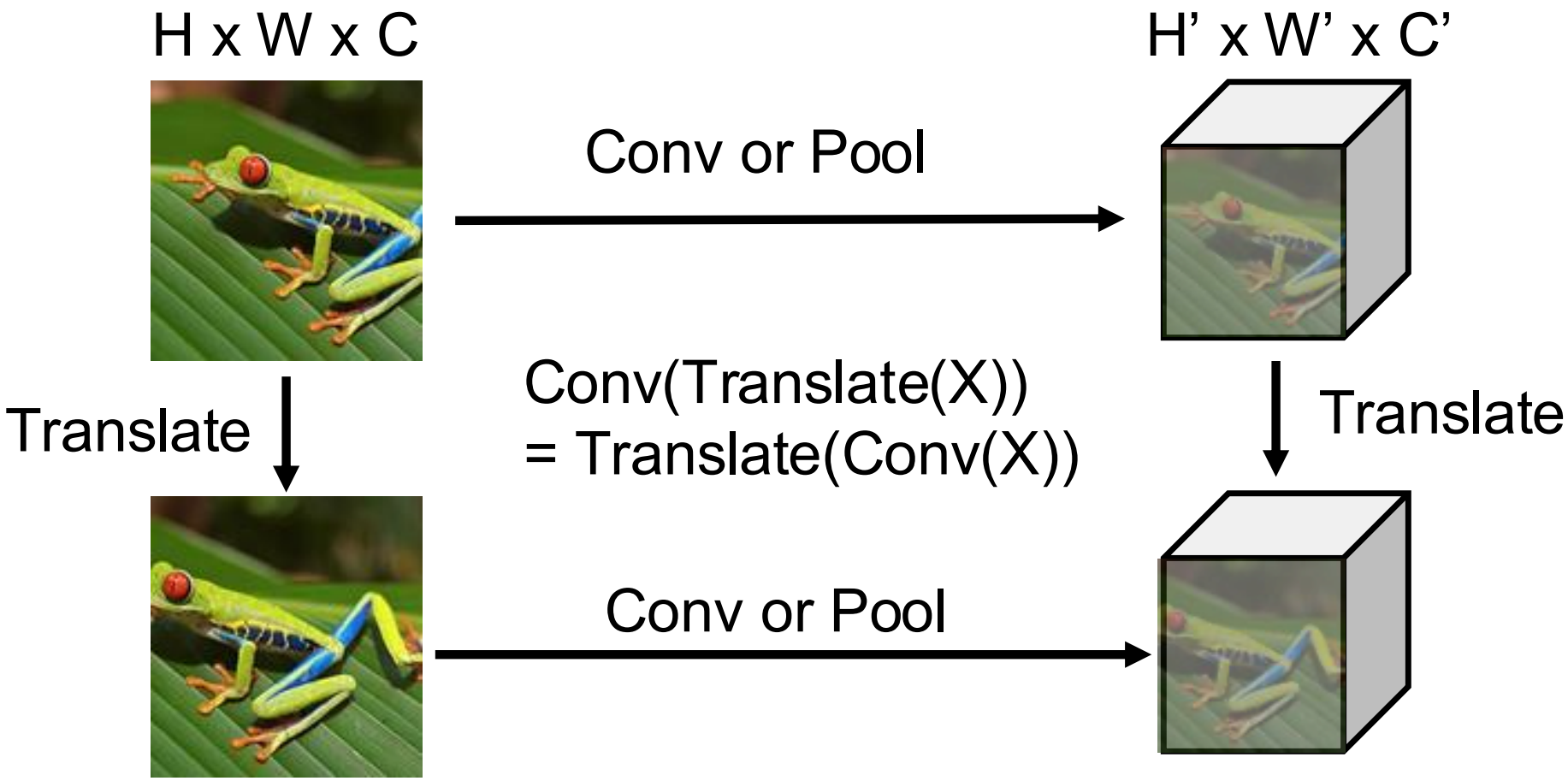
Convolution and Pooling: Translation Equivariance



Convolution and Pooling: Translation Equivariance



Convolution and Pooling: Translation Equivariance



Intuition: Features of images don't depend on their location in the image

Plan for today

1 Symmetries and inductive biases

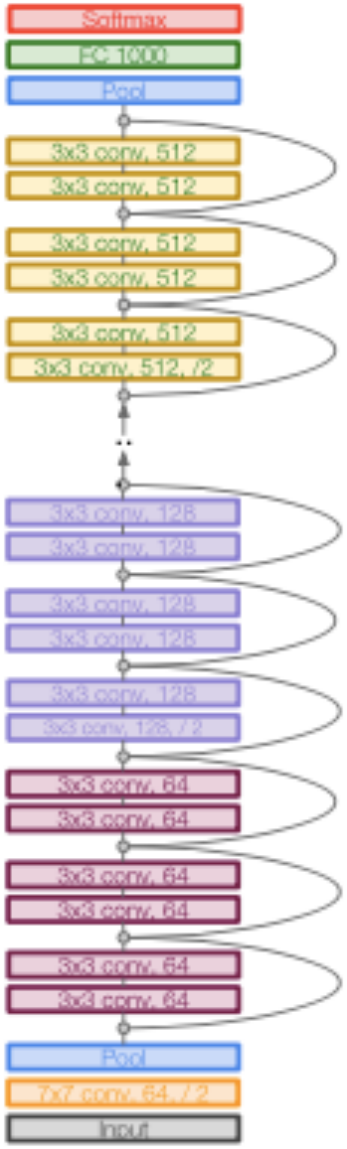
2 Convolutions

3 Convolutional Neural Networks

4 **The revolution of depth**

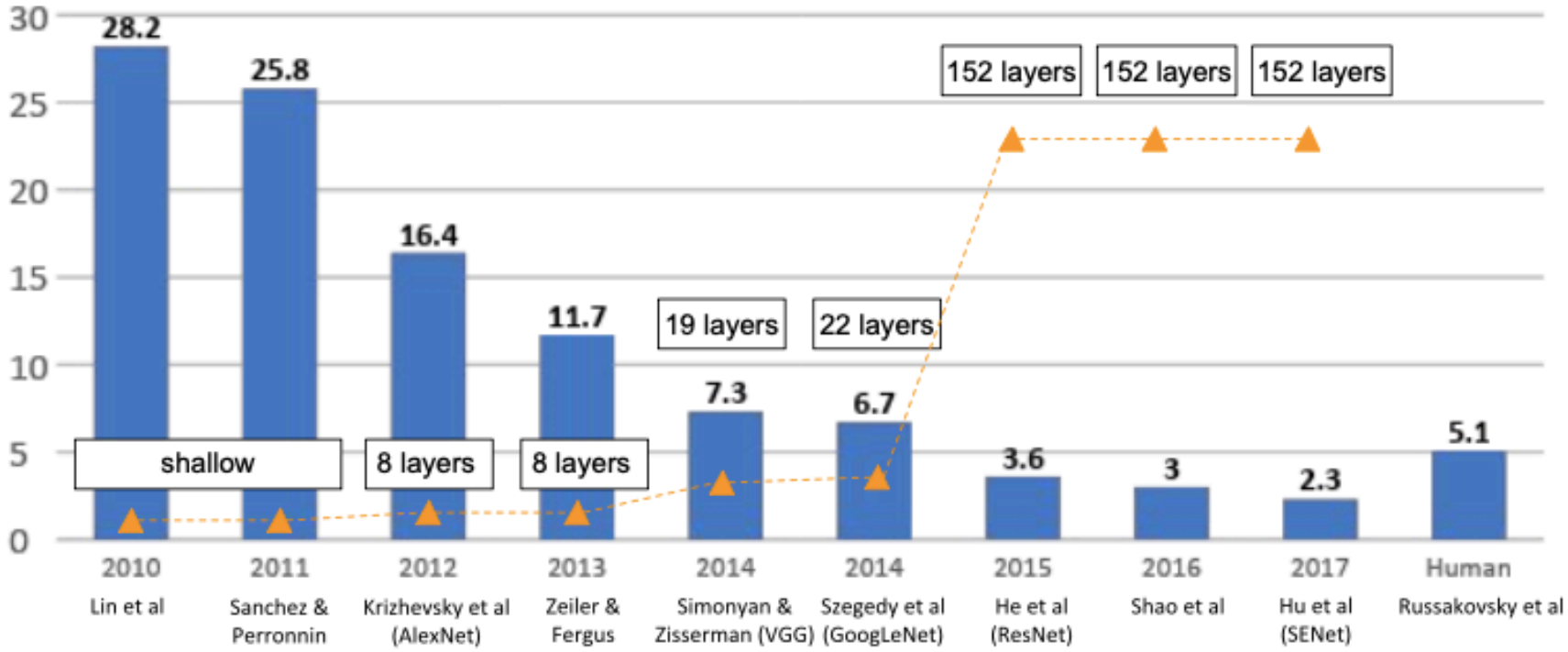
- Residual Neural Networks

ResNet: performance

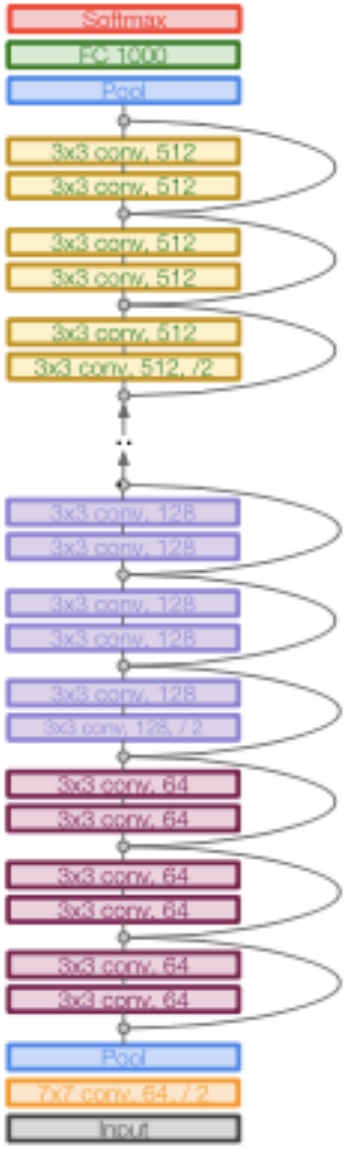


Learn the correction factor

What do we gain???

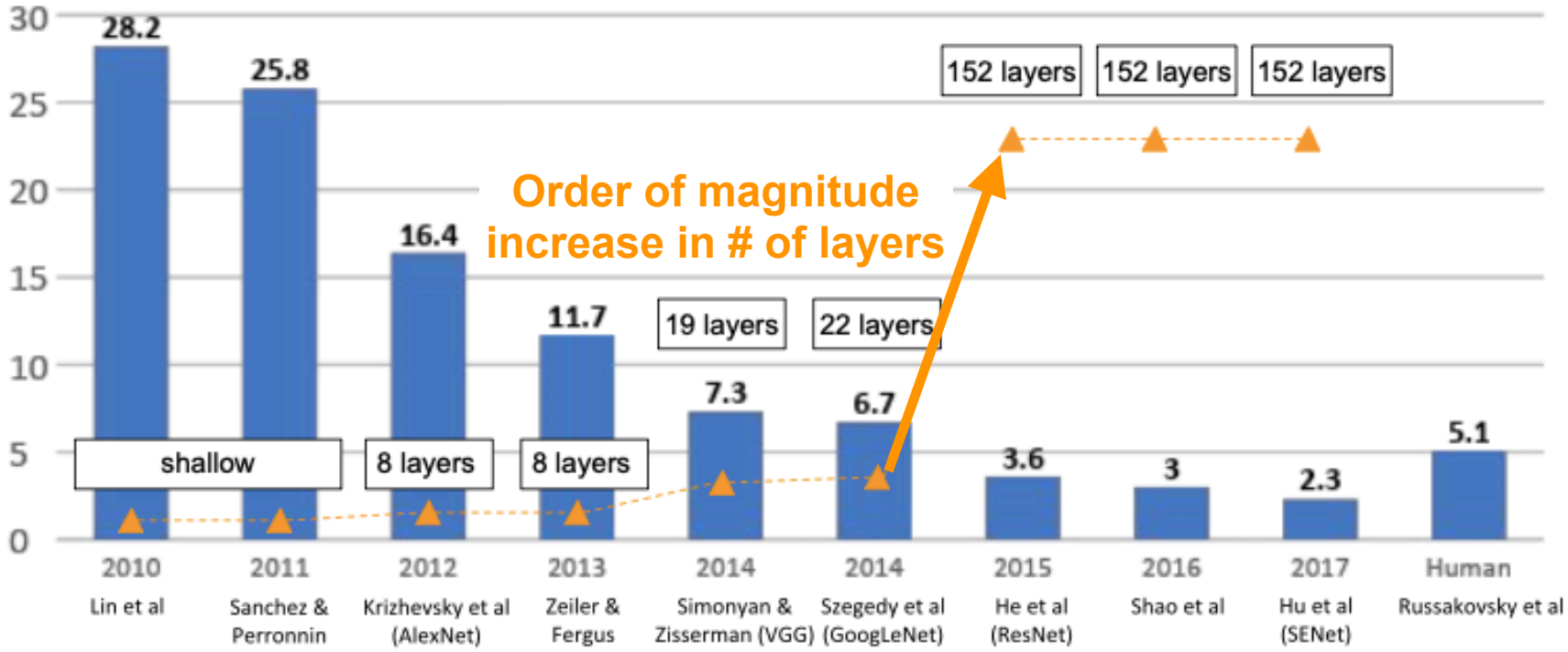


ResNet: performance

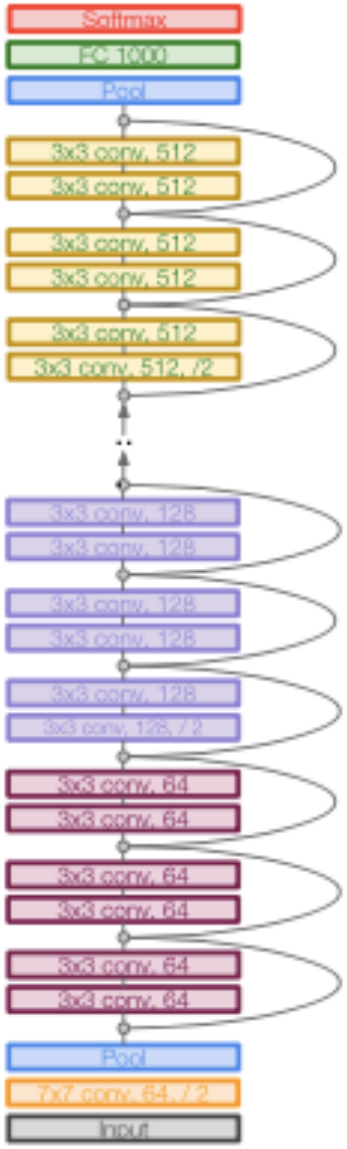


Learn the correction factor

What do we gain???

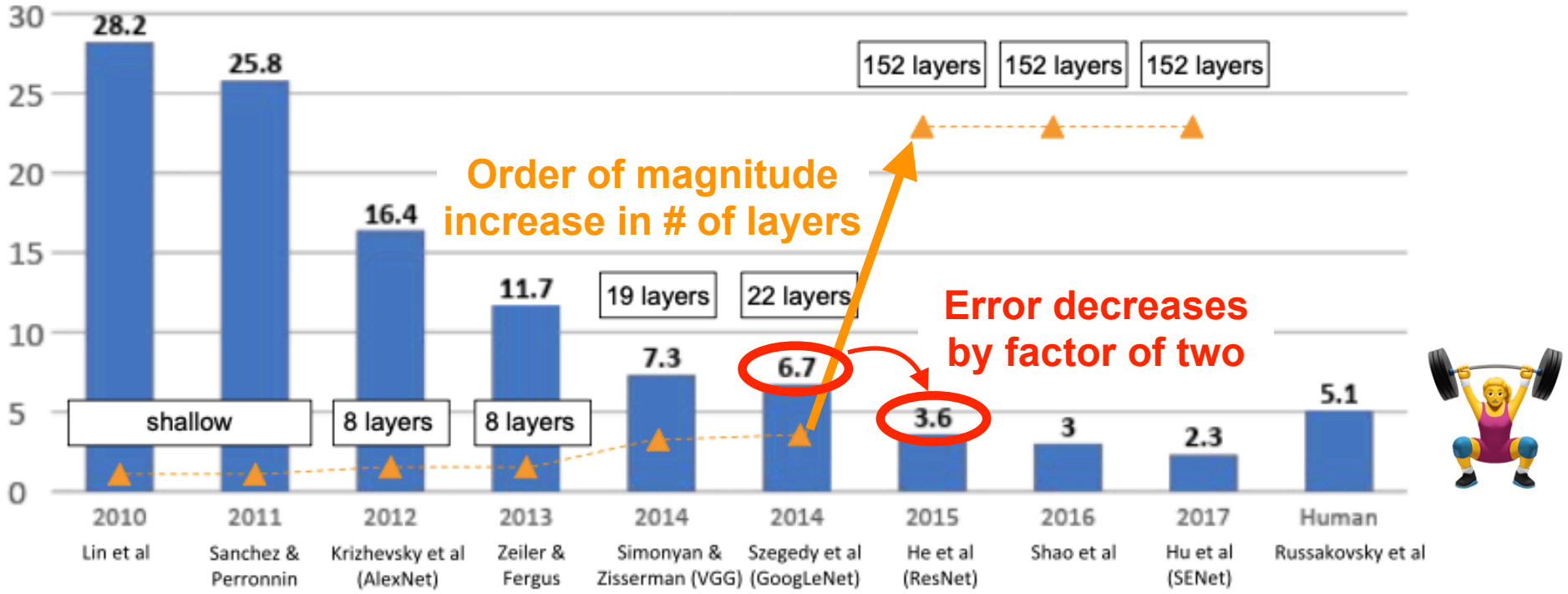


ResNet: performance

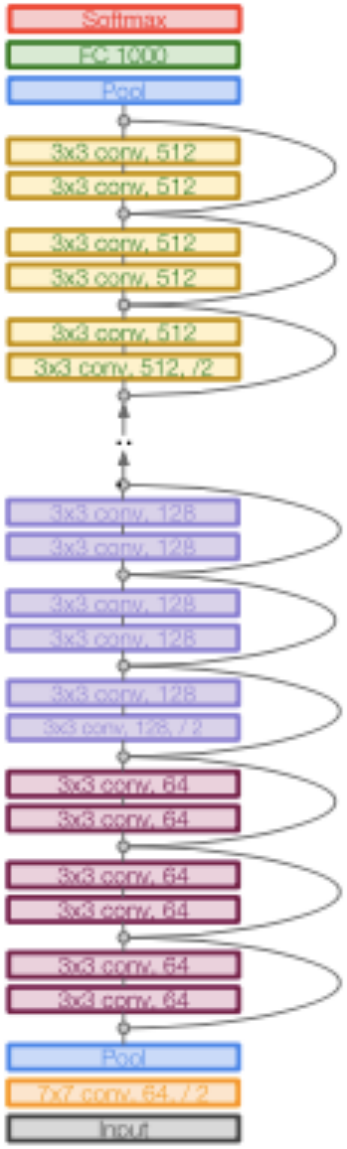


Learn the correction factor

What do we gain???

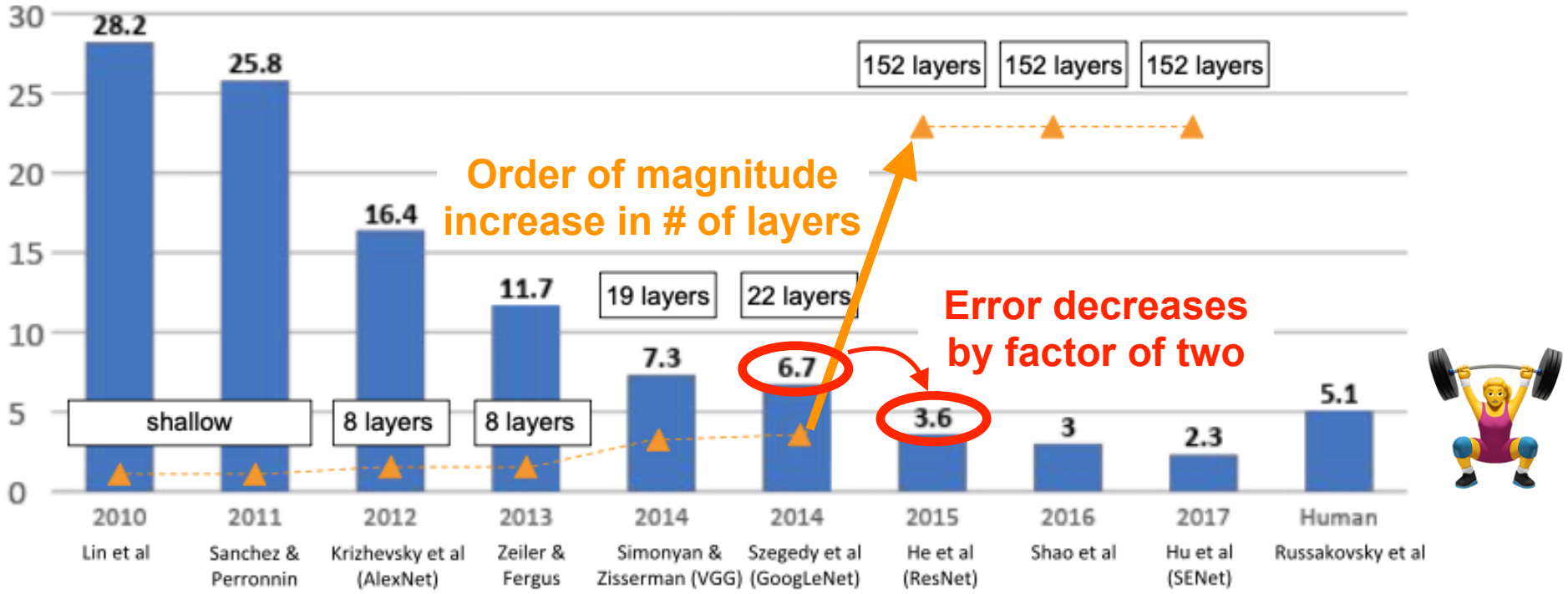


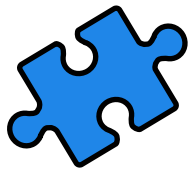
ResNet: performance



Learn the correction factor

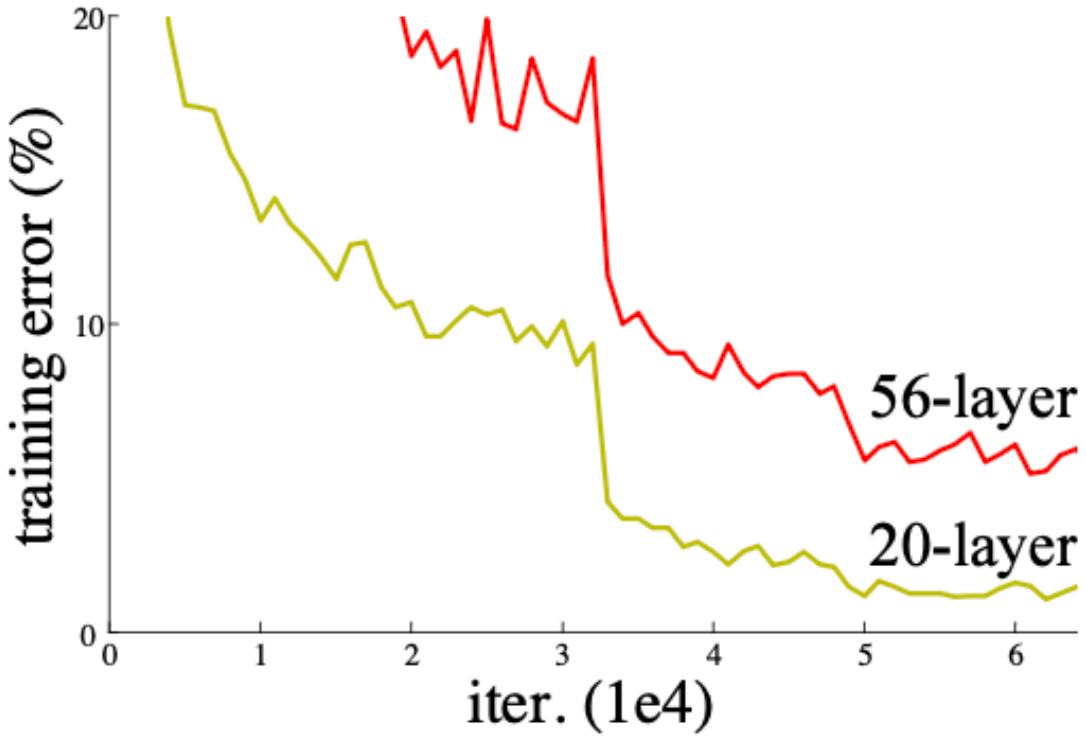
What do we gain???

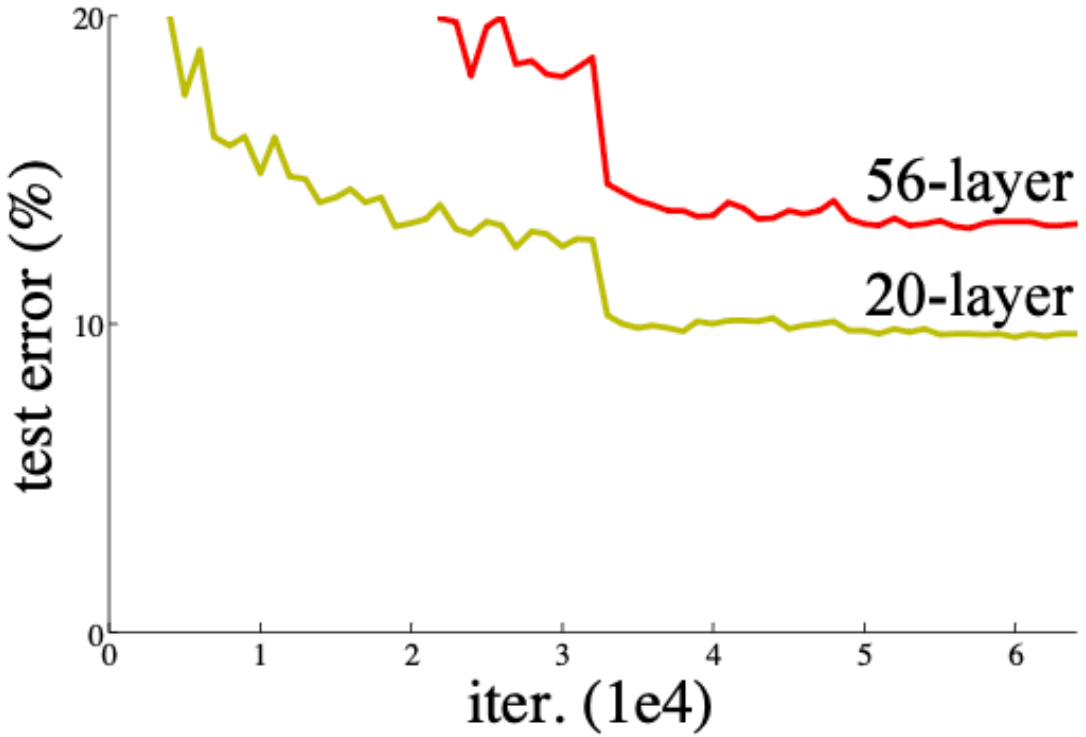




Also an element of transformers

The problem


training error (%)
iter. (1e4)
56-layer
20-layer


test error (%)
iter. (1e4)
56-layer
20-layer

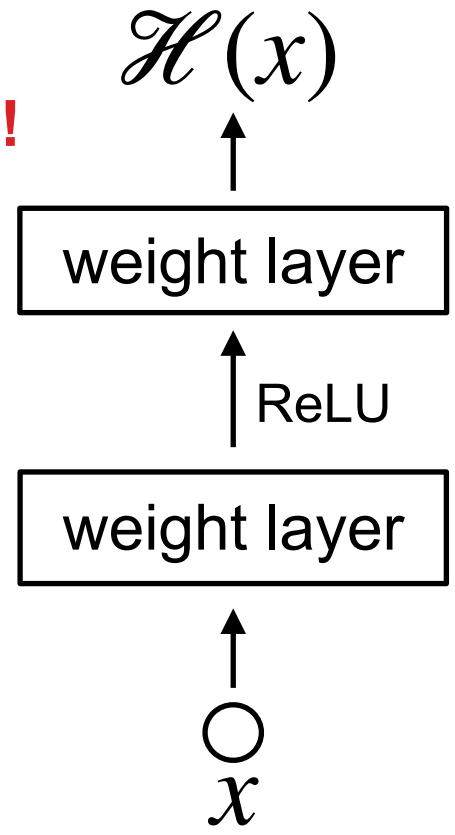
ResNet building block

288k citations

ResNets: 1512.03385
Slide CS231n lecture

⚠ If layer isn't needed,
need to learn the identity.

Degrades performance!



Normal layers

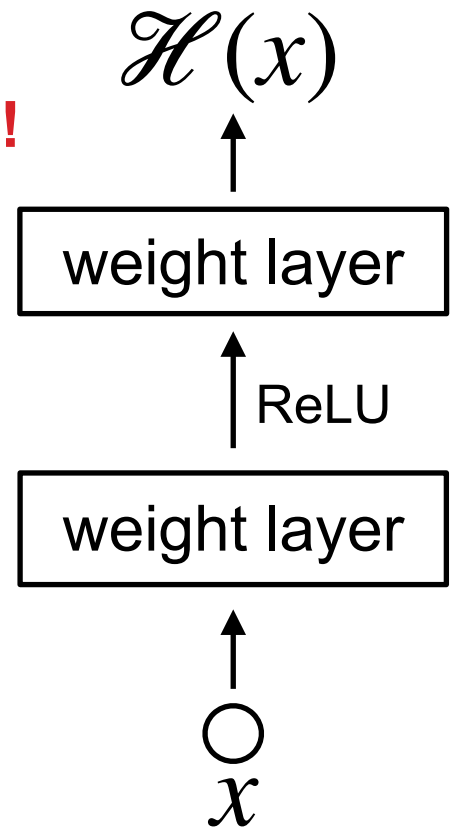
ResNet building block

288k citations

ResNets: 1512.03385
Slide CS231n lecture

If layer isn't needed,
need to learn the identity.

Degrades performance!



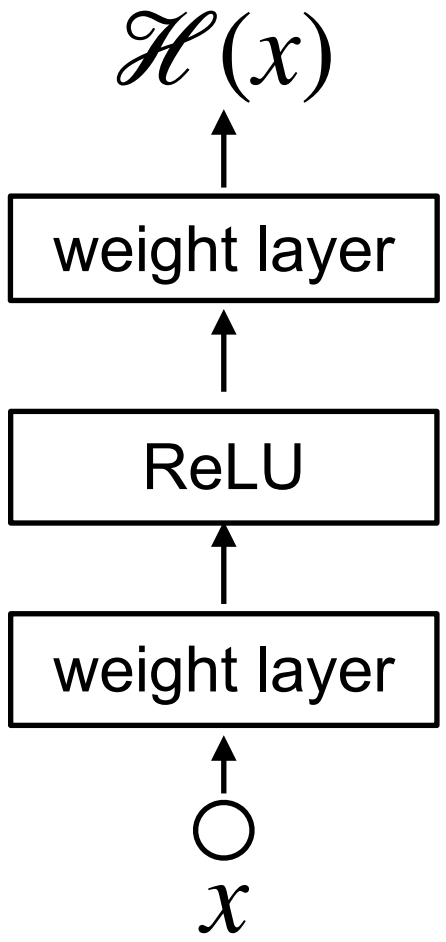
Normal layers

Learn the correction factor

Solves the vanishing gradients problem!

Residual block

ResNet building block



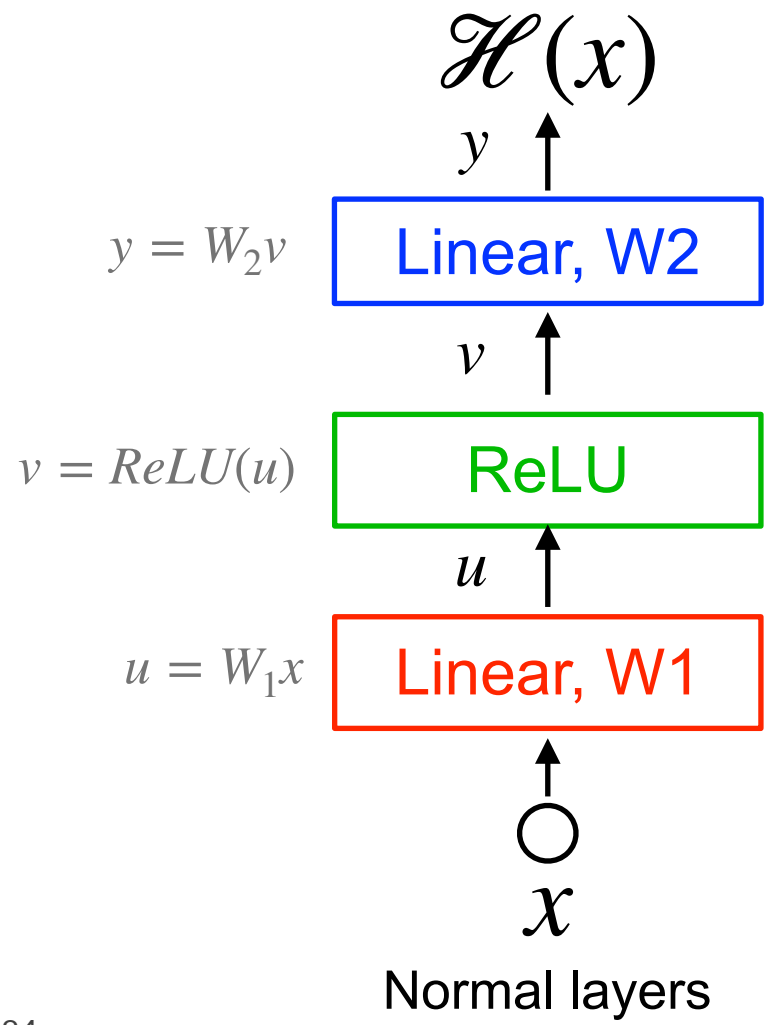
Normal layers

Learn the correction factor

Solves the vanishing gradients problem!

Residual block

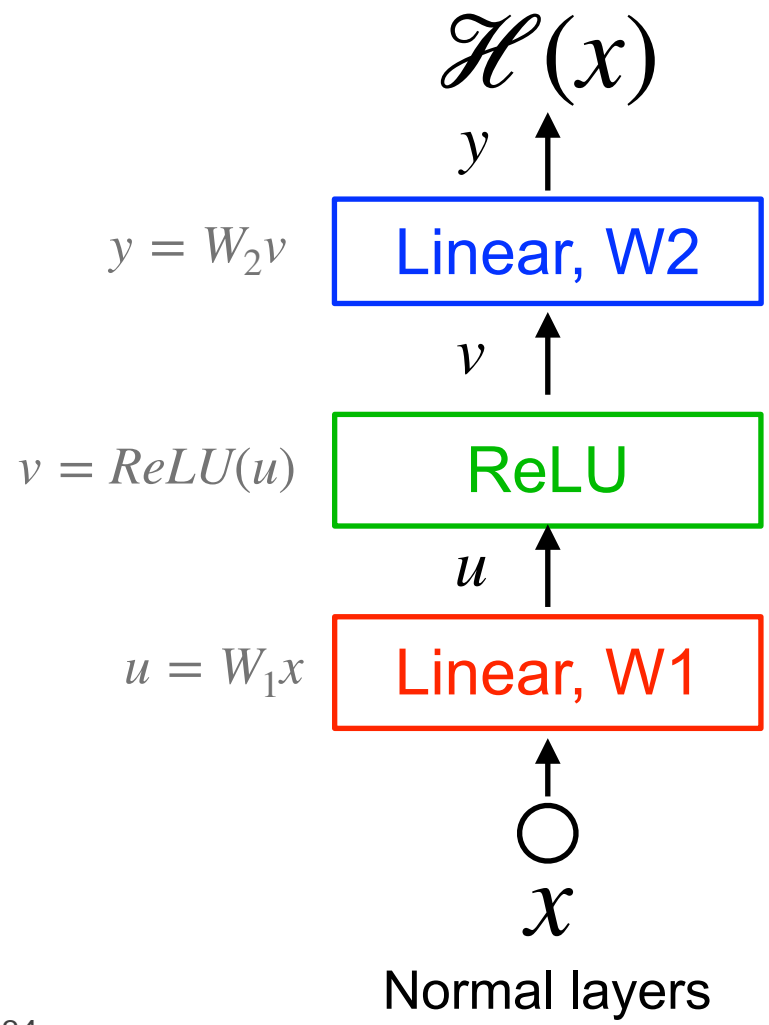
Plain network: computational graph



$$\frac{dy}{dv} = W_2$$

$$= \underset{n \times h}{W_2}$$

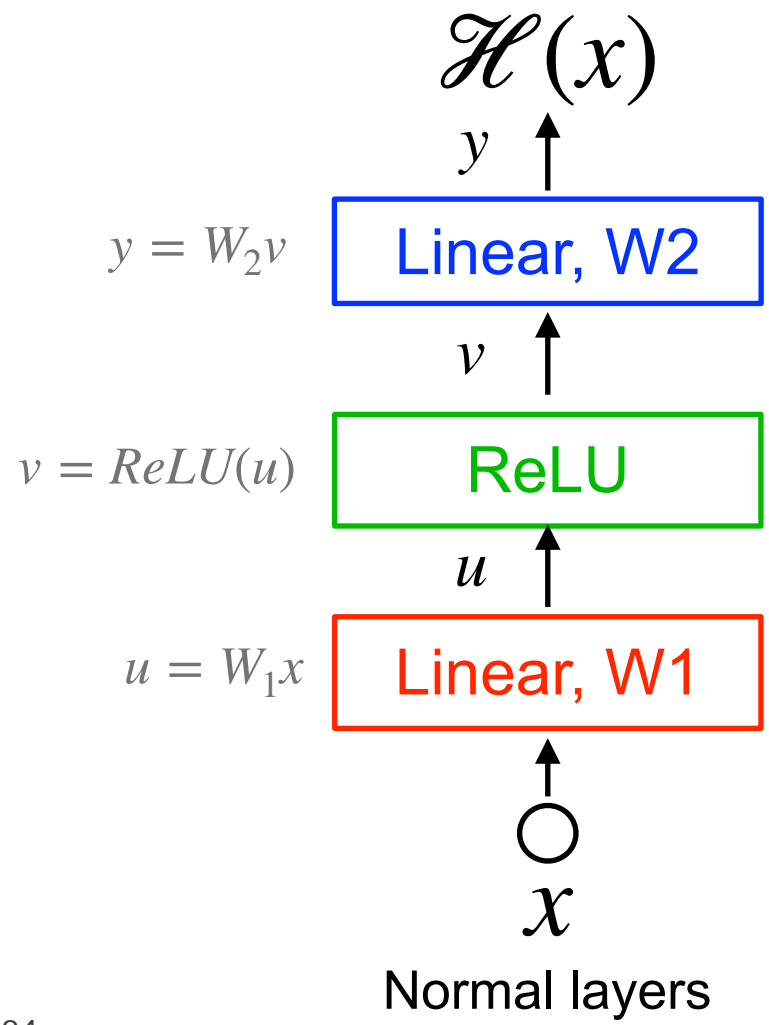
Plain network: computational graph



$$\frac{dy}{dv} = W_2$$

$$\frac{dH(x)}{dx} = = W_2 \quad (n \times h)$$

Plain network: computational graph

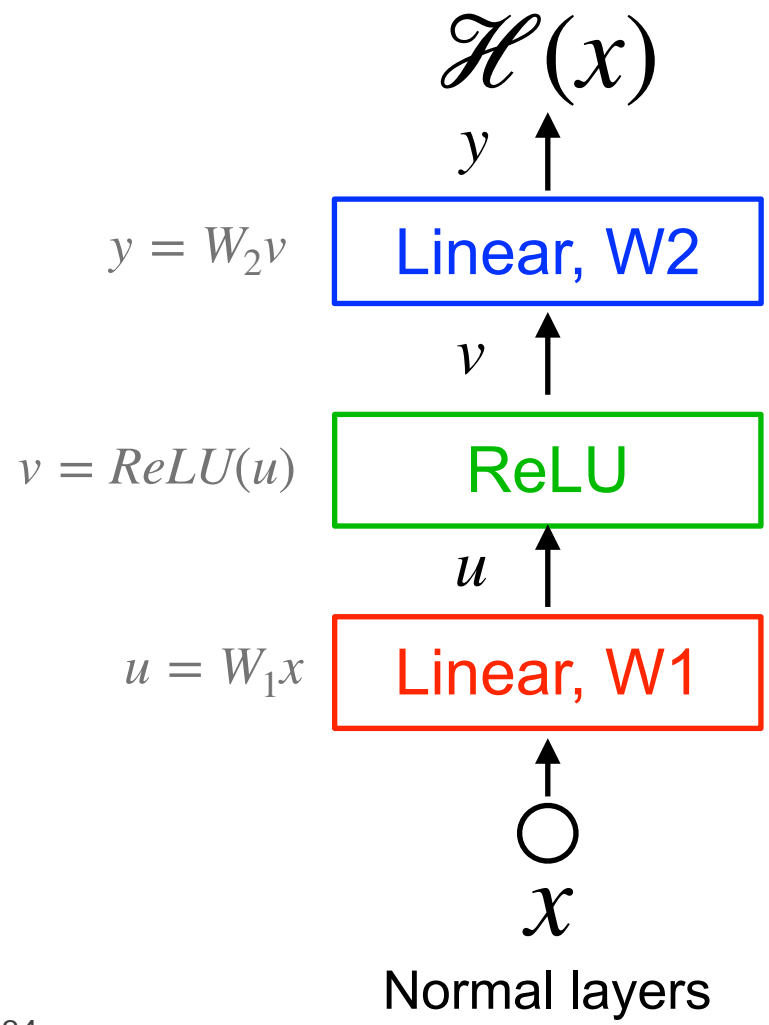


$$\frac{dy}{dv} = W_2$$

$$\frac{dH(x)}{dx} = \frac{dy}{dv}$$

$$= \underset{n \times h}{W_2}$$

Plain network: computational graph

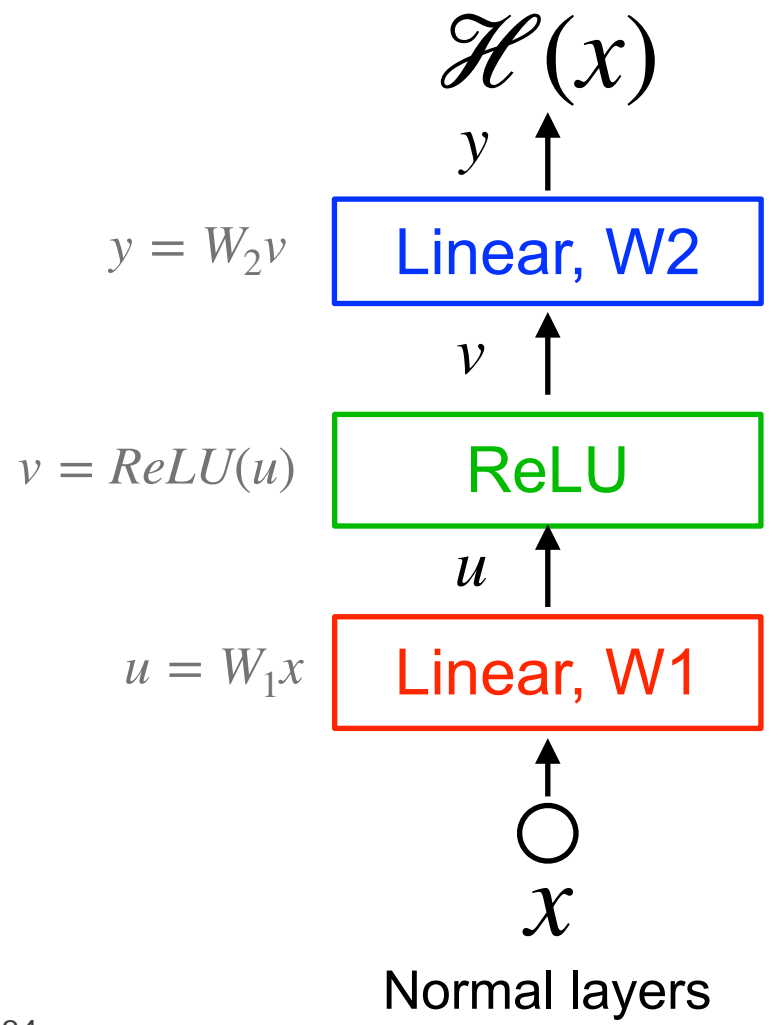


$$\frac{dy}{dv} = W_2$$

$$\frac{dH(x)}{dx} = \frac{dy}{dv}\frac{dv}{du}$$

$$= \underset{n \times h}{W_2}$$

Plain network: computational graph

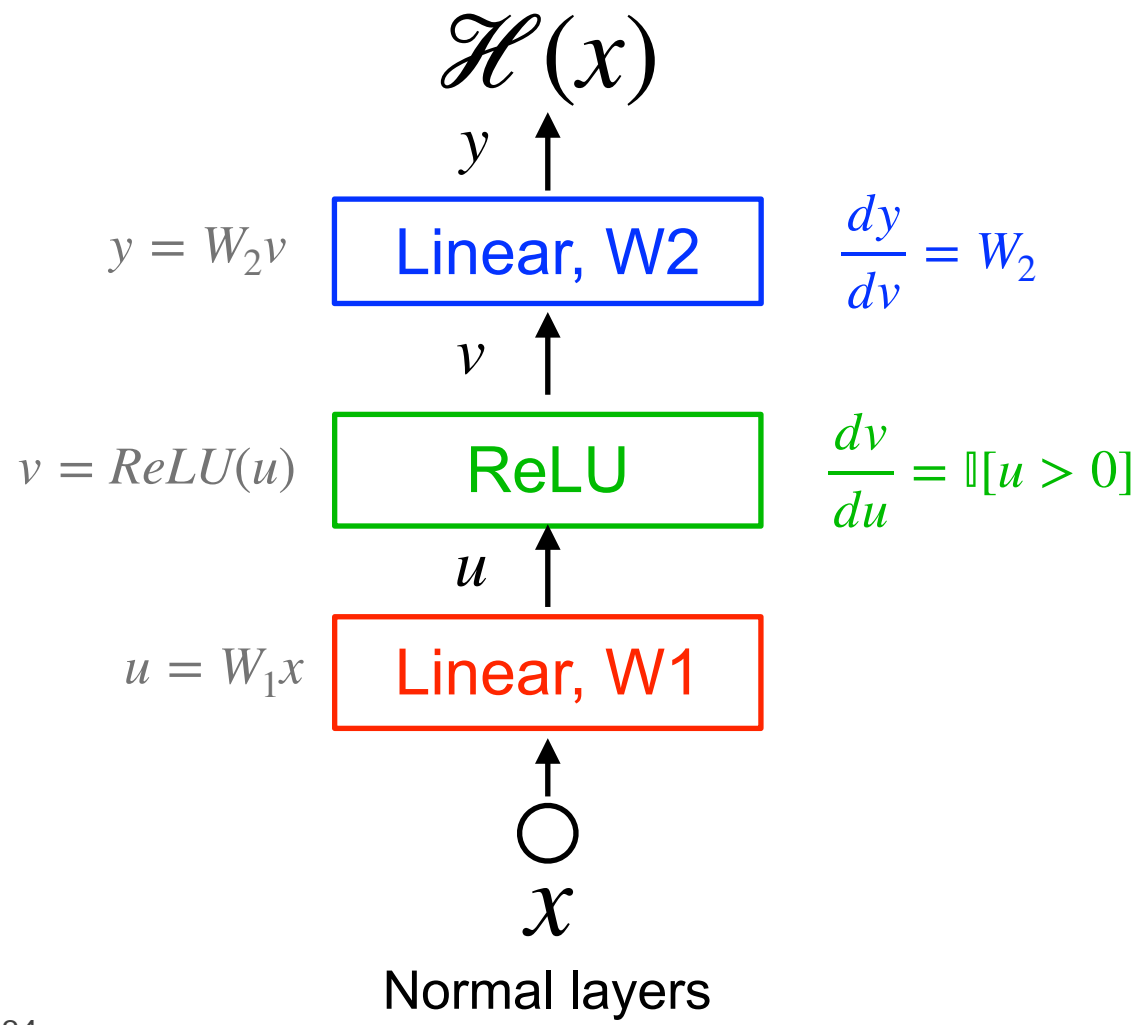


$$\frac{dy}{dv} = W_2$$

$$\frac{dH(x)}{dx} = \frac{dy}{dv}\ \frac{dv}{du}\ \frac{du}{dx}$$

$$= \underset{n \times h}{W_2}$$

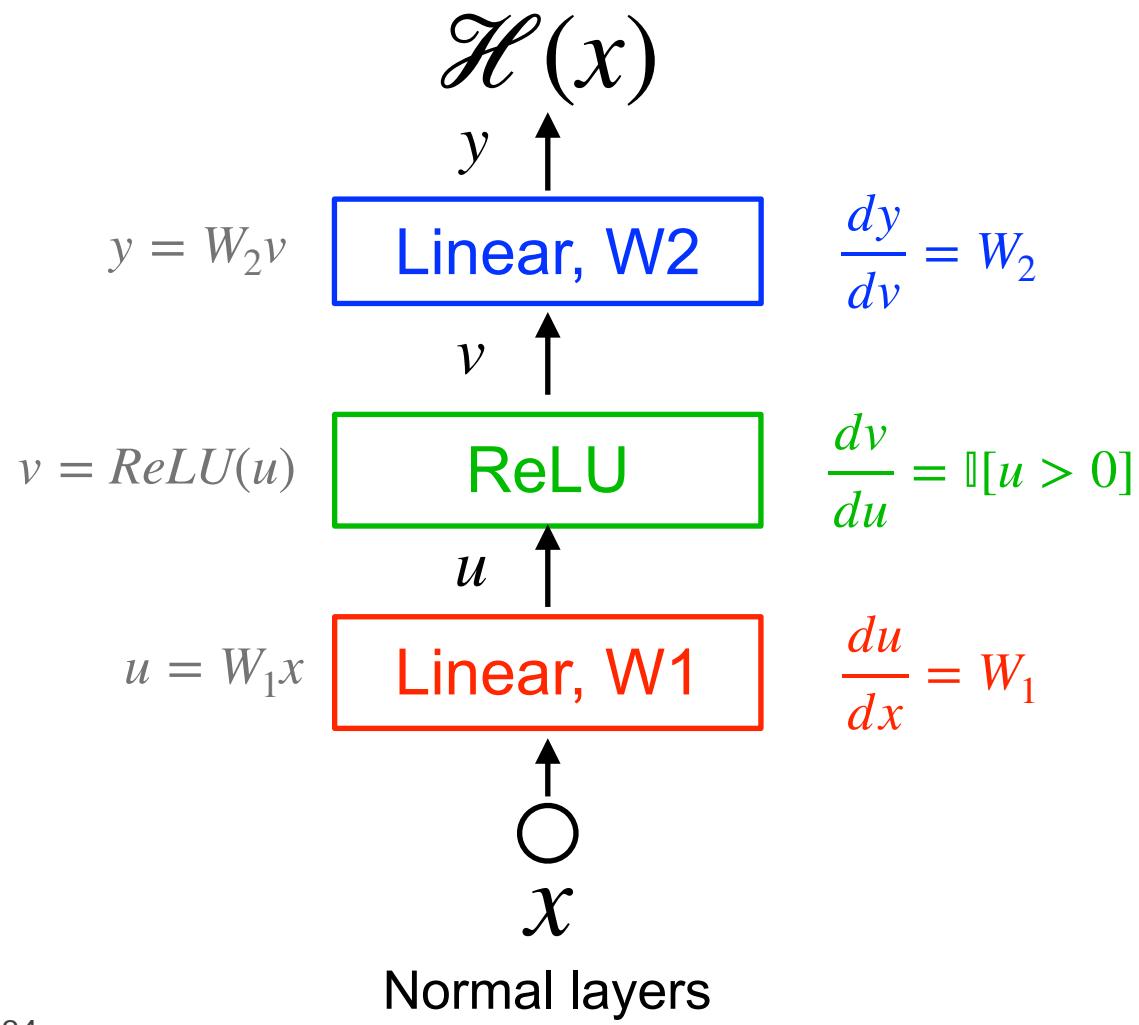
Plain network: computational graph



$$\frac{dH(x)}{dx} = \frac{dy}{dv}\,\frac{dv}{du}\,\frac{du}{dx}$$

$$= \underset{n \times h}{W_2}\ \underset{h \times h}{\text{diag}(u > 0)}$$

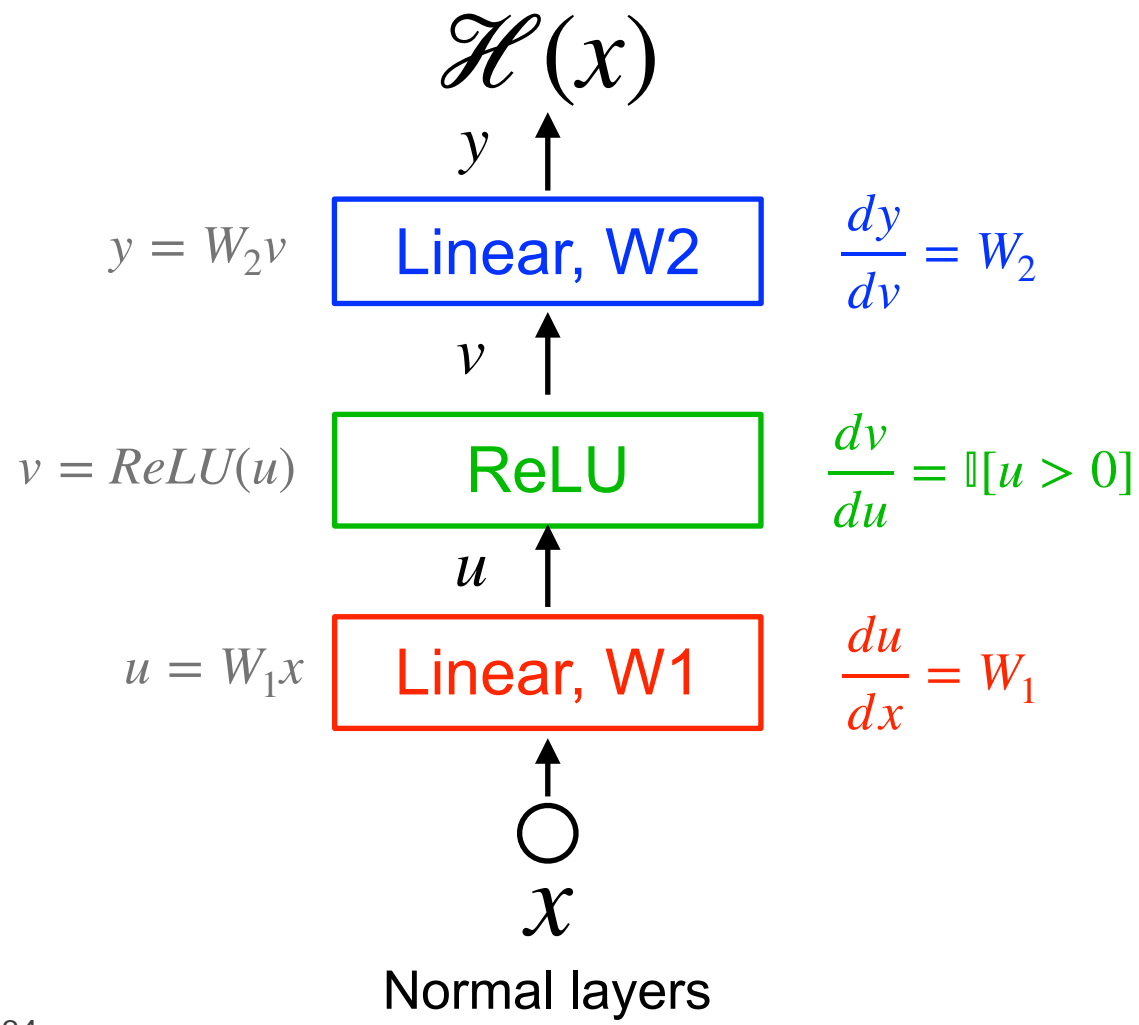
Plain network: computational graph



$$\frac{dH(x)}{dx} = \frac{dy}{dv}\,\frac{dv}{du}\,\frac{du}{dx}$$

$$= \underset{n \times h}{W_2}\ \underset{h \times h}{\text{diag}(u > 0)}\ \underset{h \times n}{W_1}$$

Plain network: computational graph

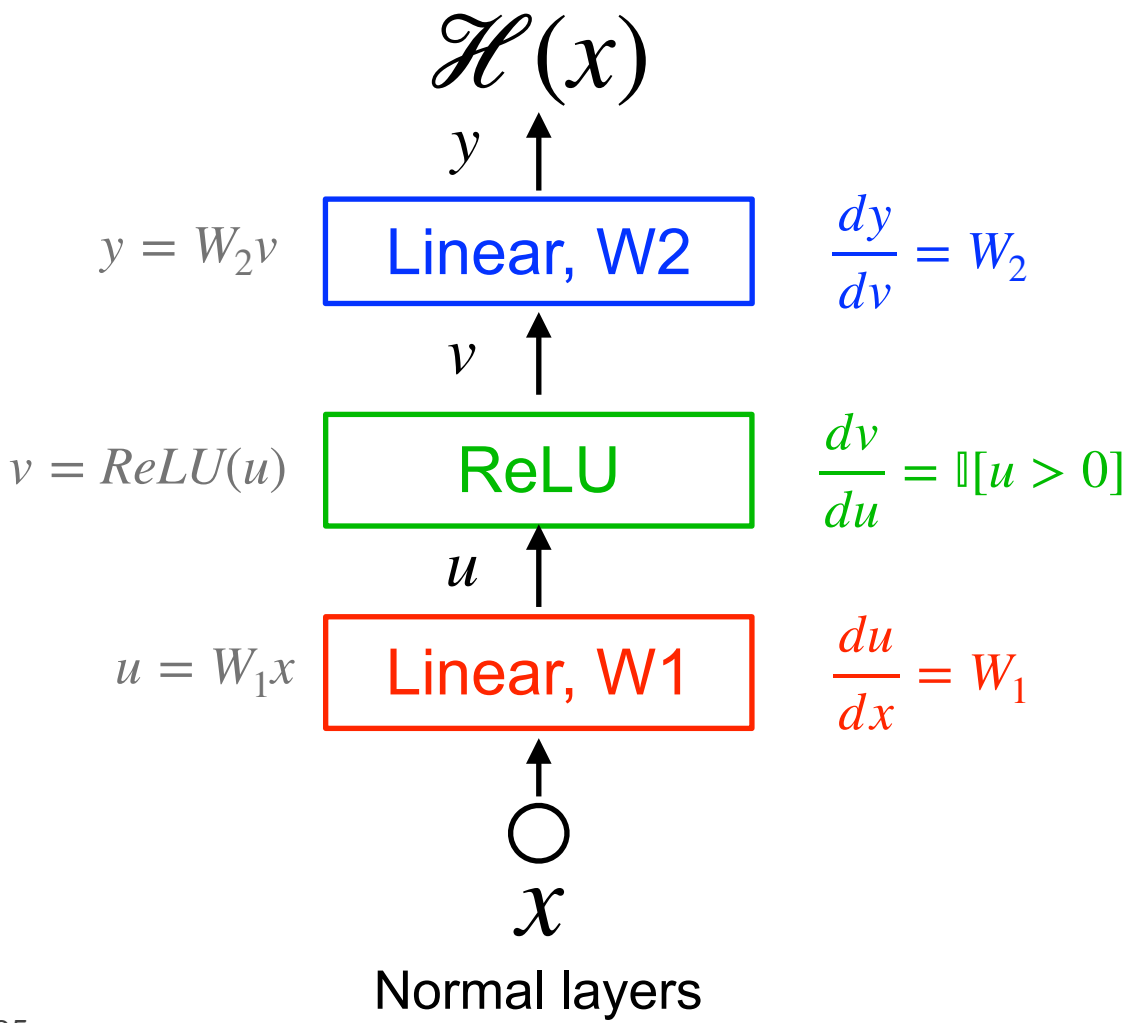


$$\frac{dH(x)}{dx} = \frac{dy}{dv}\,\frac{dv}{du}\,\frac{du}{dx}$$

$$= \underset{n \times h}{W_2}\ \underset{h \times h}{\text{diag}(u > 0)}\ \underset{h \times n}{W_1}$$

ReLU hasn't solved the problem entirely

Plain network: computational graph

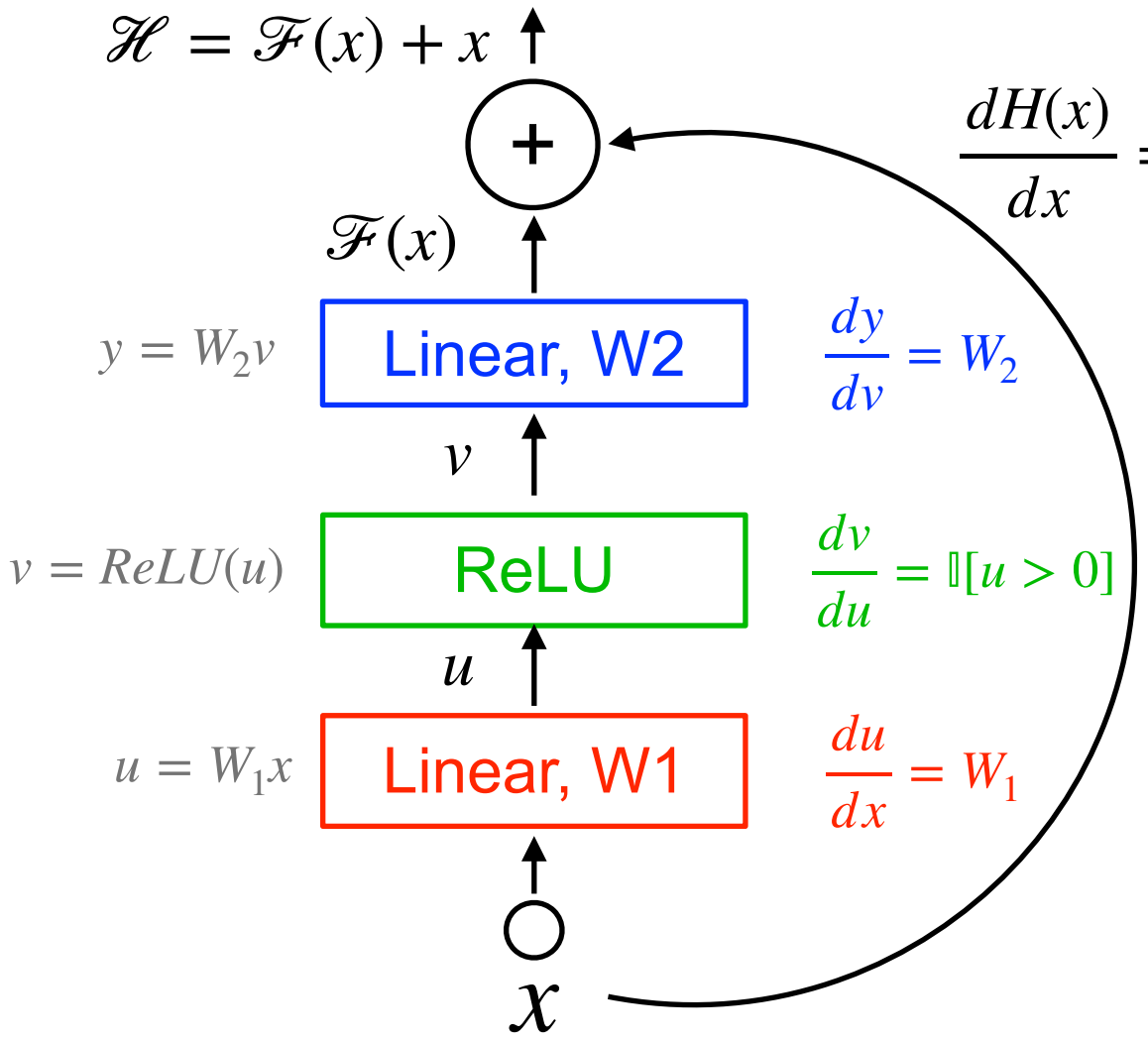


$$\frac{dH(x)}{dx} = \frac{dy}{dv}\ \frac{dv}{du}\ \frac{du}{dx}$$

$$= \underset{n \times h}{W_2}\ \underset{h \times h}{\text{diag}(u > 0)}\ \underset{h \times n}{W_1}$$

W2 and W1 are “gates” through which the gradient information flows

ResNet: computational graph

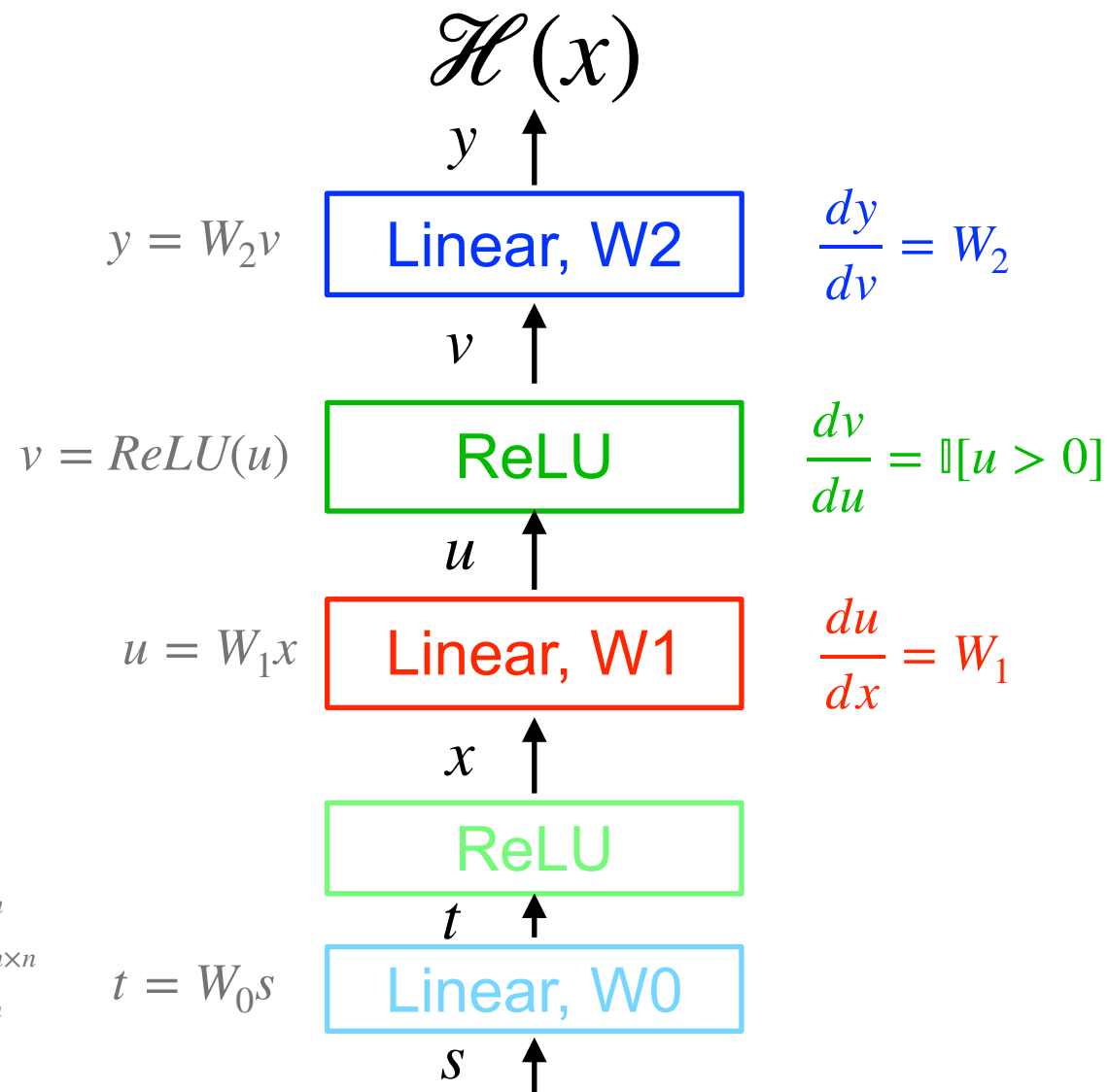


$$\frac{dH(x)}{dx} = \frac{dF(x)}{dx} + \frac{dx}{dx} = \frac{dy}{dv}\ \frac{dv}{du}\ \frac{du}{dx} + 1$$

$$= \underset{n \times h}{W_2}\ \underset{h \times h}{\text{diag}(u > 0)}\ \underset{h \times n}{W_1} + 1$$

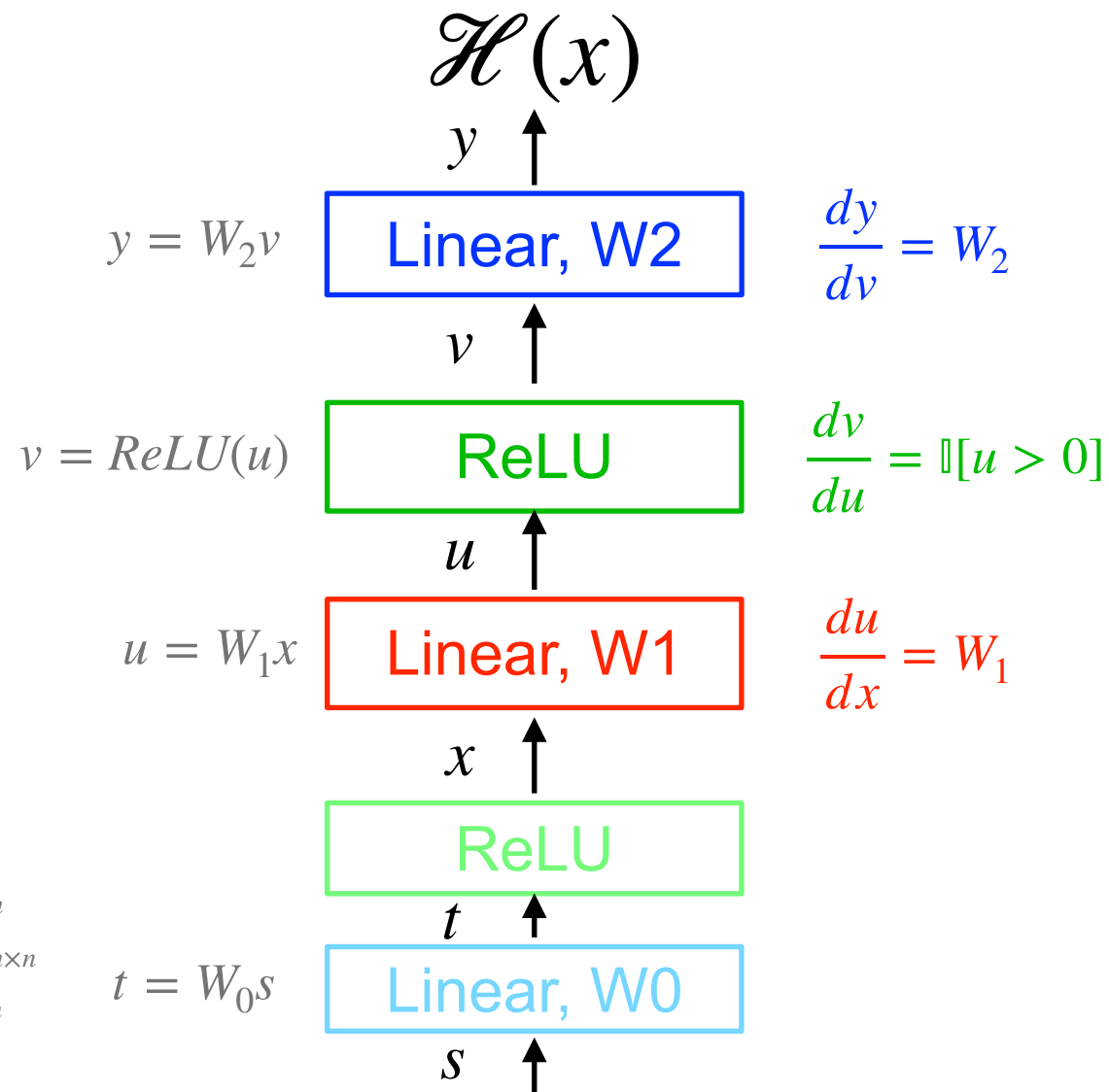
We now have a “gradient super highway”

Optimizing upstream layers



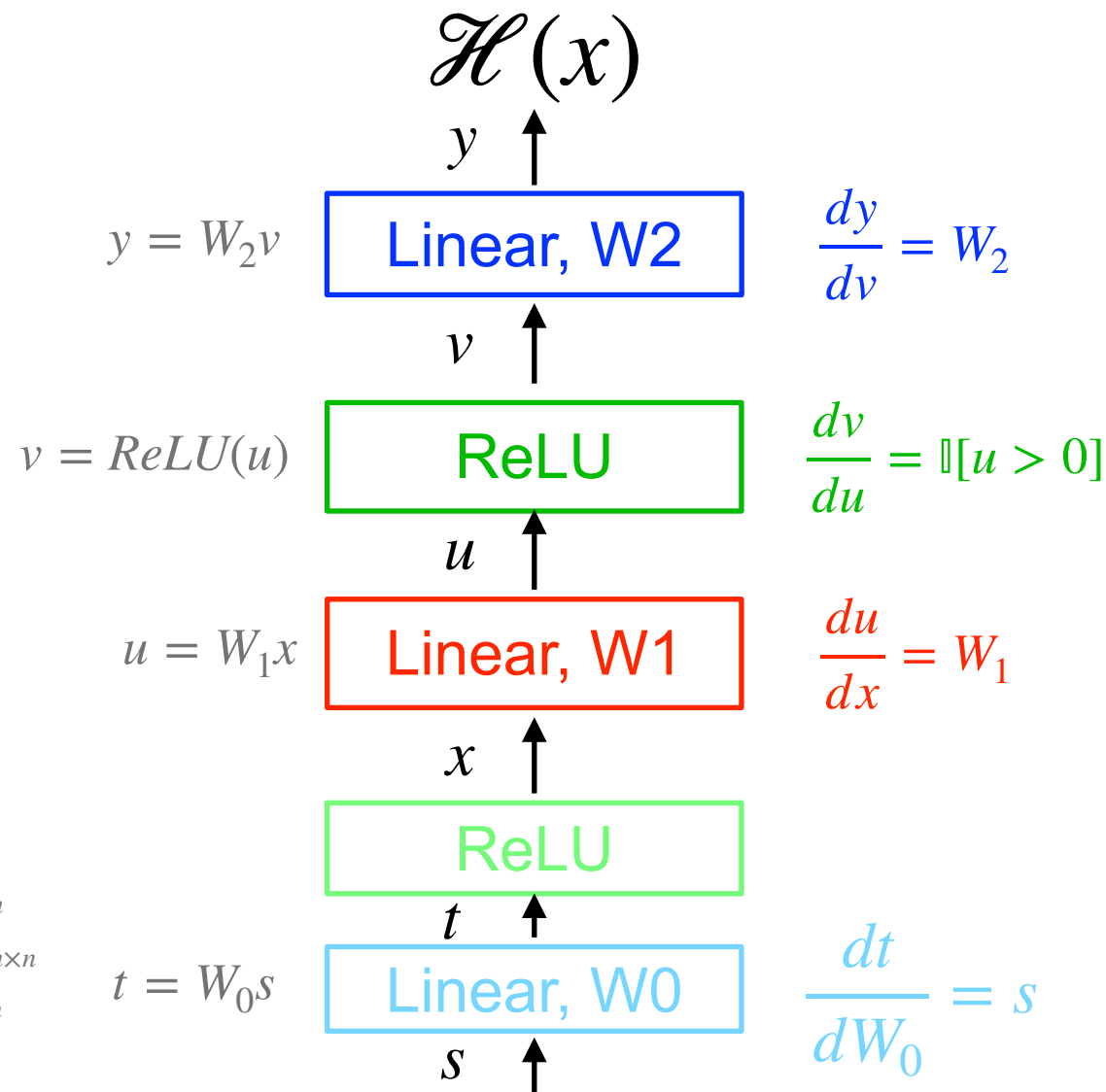
$$\frac{dH(x)}{dx} = \frac{dy}{dv}\,\frac{dv}{du}\,\frac{du}{dx}$$

Optimizing upstream layers



$$\frac{dH(x)}{dW_0} = \frac{dy}{dv}\,\frac{dv}{du}\,\frac{du}{dx}\,\frac{dx}{dt}\,\frac{dt}{dW_0}$$

Optimizing upstream layers

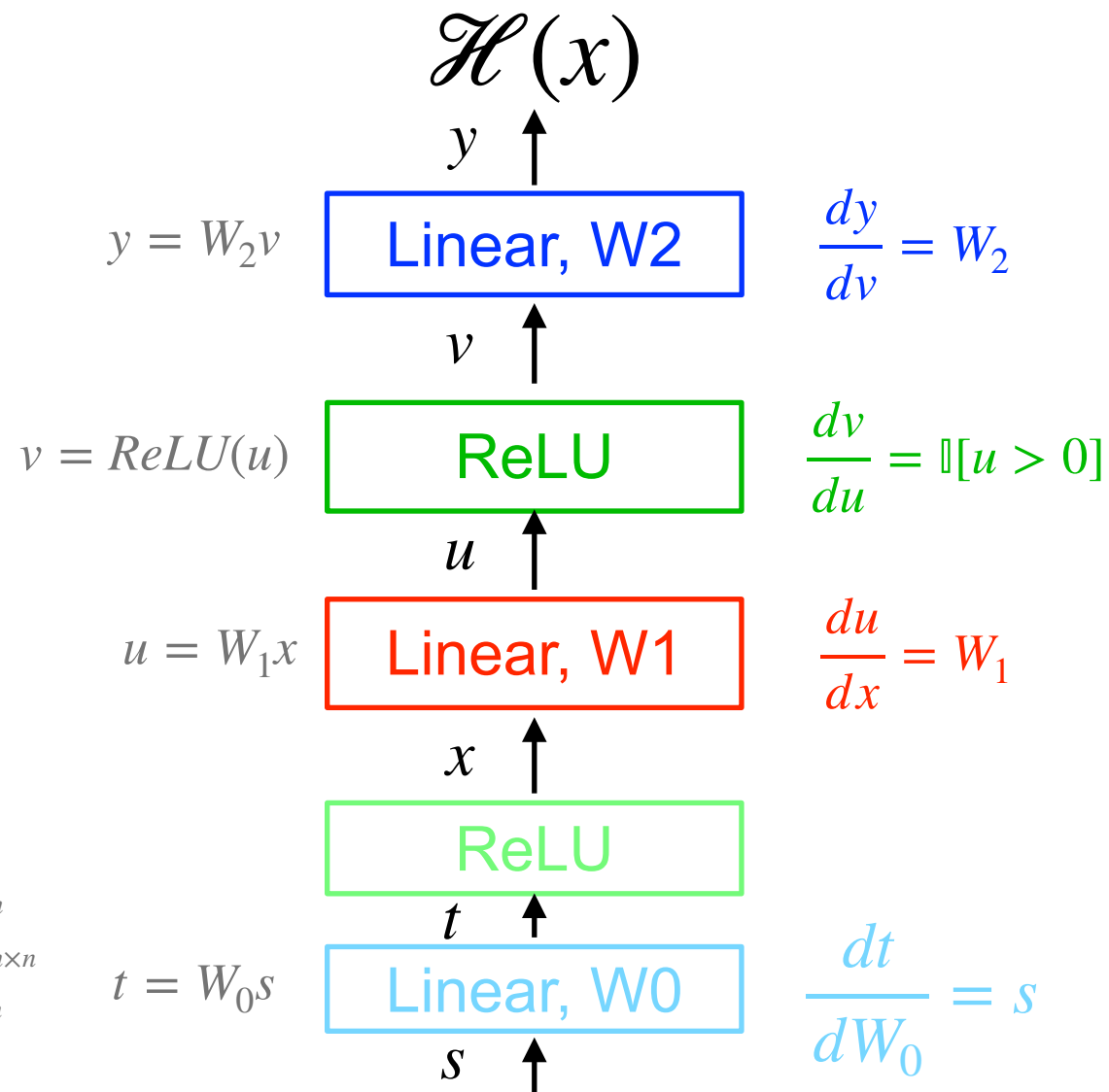


$t \in \mathbb{R}^n$
$W_0 \in \mathbb{R}^{h \times n}$
$s \in \mathbb{R}^h$

$$\frac{dH(x)}{dW_0} = \frac{dy}{dv}\frac{dv}{du}\frac{du}{dx}\frac{dx}{dt}\frac{dt}{dW_0}$$

$$= \underset{n \times h}{W_2}\ \underset{h \times h}{\mathbb{I}[u > 0]}\ \underset{h \times n}{W_1}\ \underset{n \times n}{\mathbb{I}[t > 0]}\ \underset{h}{s}$$

Optimizing upstream layers



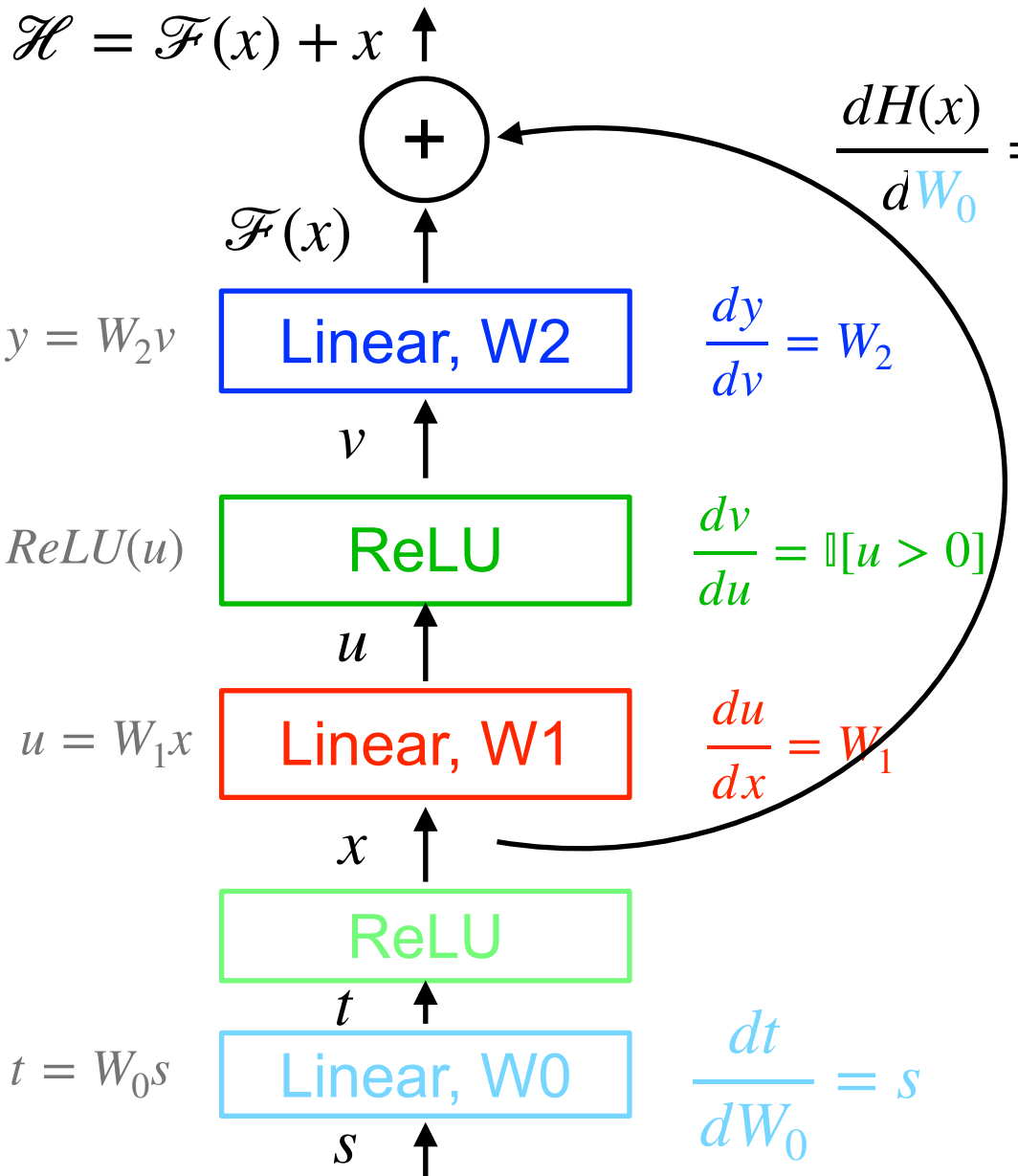
$$\frac{dH(x)}{dW_0} = \frac{dy}{dv}\frac{dv}{du}\frac{du}{dx}\frac{dx}{dt}\frac{dt}{dW_0}$$

$$= W_2 \mathbb{I}[u > 0] W_1 \mathbb{I}[t > 0] s$$

$n \times h$ $h \times h$ $h \times n$ $n \times n$ h

Whenever an upstream layer has weights going to 0, this kills the learning signal for the earlier layers

Optimizing upstream layers



$t \in \mathbb{R}^n$
$W_0 \in \mathbb{R}^{h \times n}$
$s \in \mathbb{R}^h$

$$\frac{dH(x)}{dW_0} = \frac{dF(x)}{dW_0} + \frac{dx}{dW_0} = \frac{dy}{dv}\frac{dv}{du}\frac{du}{dx}\frac{dx}{dt}\frac{dt}{dW_0} + \frac{dx}{dW_0}$$

$$= \underset{n \times h}{W_2}\ \underset{h \times h}{\mathbb{I}[u > 0]}\ \underset{h \times n}{W_1}\ \underset{n \times n}{\mathbb{I}[u > 0]}\ \underset{h}{s}$$

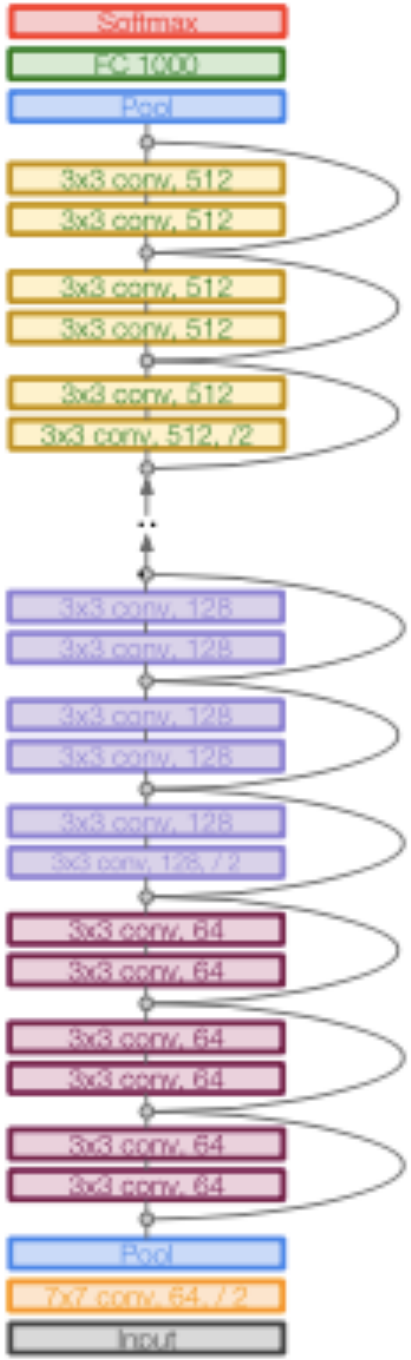
$$+ \underset{n \times n}{\mathbb{I}[u > 0]}\ \underset{h}{s}$$

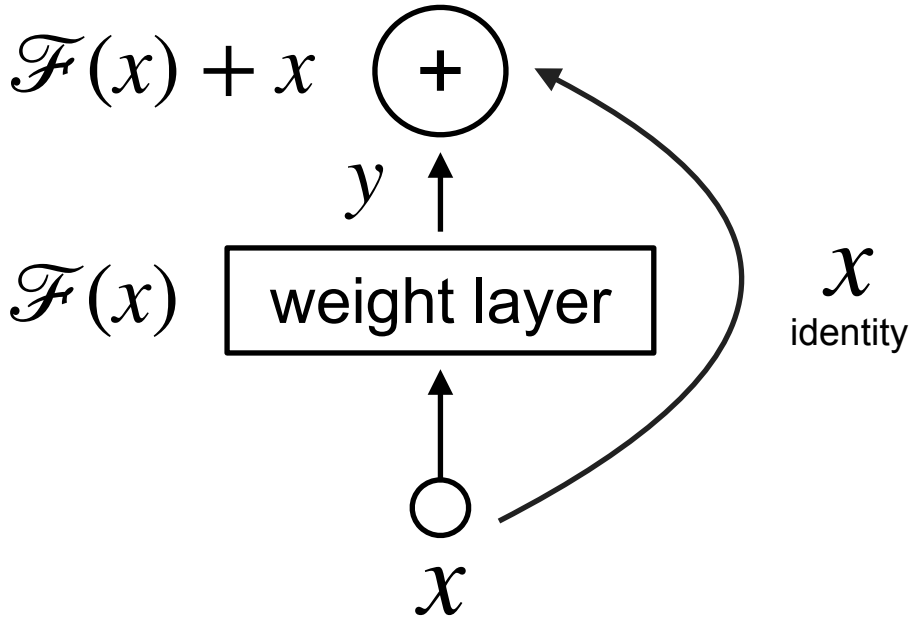
Even if my grad signal doesn't propagate through the first term… the last term acts like a "shallow NN" that got a gradient signal

directly from x

ResNets: 1512.03385
Slide CS231n lecture

Why always skipping two layers, not one?





$$y = Wx + x$$

- ✓ Same complexity as a linear layer
- ✓ Empirically, didn't see advantages when only skipping one layer

Backup

Outline

1. Multi-class classification: Softmax
2. Multi-layer perceptrons (intro to NNs)
3. Intro to pytorch

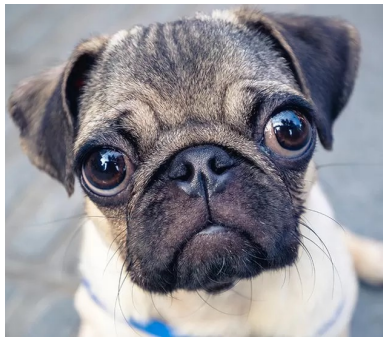
Types of data

Regression $p(y \mid X), y \in \mathbb{R}$ Ex: Housing prices

Classification $p(y \mid X), y \in [\text{class1}, \text{class2}, \ldots]$

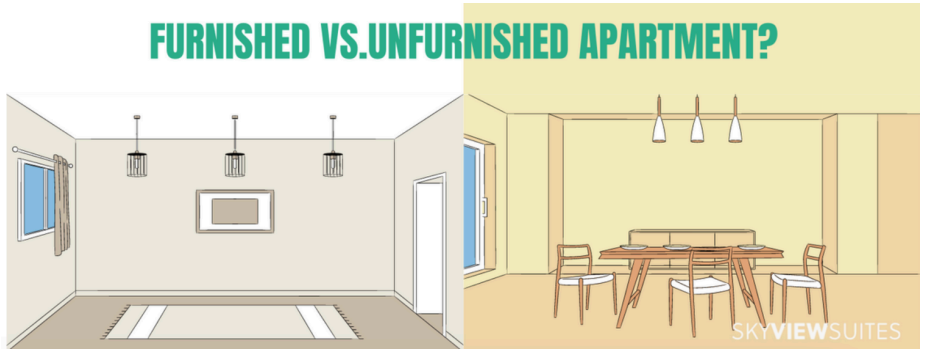
Yesterday: Binary classification

$y = [\text{cat}, \text{dog}]$



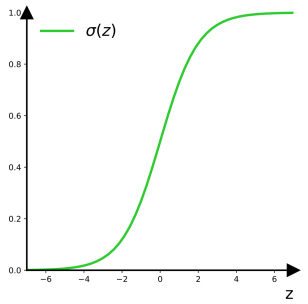
NEXT: Multi-class classification

$y = [\text{not furnished}, \text{furnished}, \text{semi-furnished}]$



Perceptron:

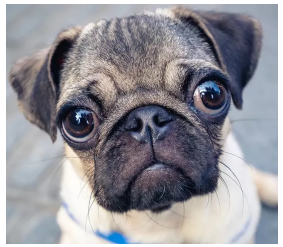
$f_\theta(x) = \sigma(\theta^T x)$

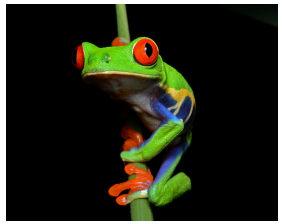


```
x_cat = np.zeros_like(df['furnishingstatus'])

for k, case in enumerate(df['furnishingstatus'].unique()):
    print(k,case)

    x_cat[df['furnishingstatus']==case] = k

print(x_cat)
```

```
0 furnished
1 semi-furnished
2 unfurnished
```

Beyond two classes…



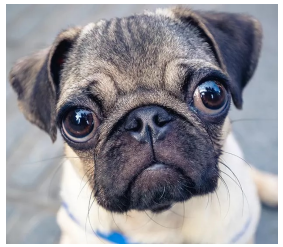


Extending the sigmoid…
what if we have more than
just cats and dogs?

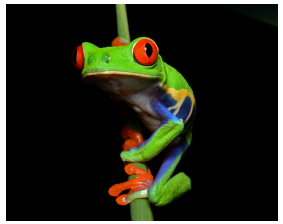
Targets: One hot vector!

Same idea… more cases!

$$y = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}$$



$$y = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}$$



$$y = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}$$

Specify the non-zero index in a set of classes

Fun fact!!

How you specify words in a dictionary for training language models

$$\text{a} = \begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \\ \vdots \\ 0 \end{pmatrix} \quad \text{aardvark} = \begin{pmatrix} 0 \\ 1 \\ 0 \\ 0 \\ \vdots \\ 0 \end{pmatrix} \quad \bullet \bullet \bullet \quad \text{exciting} = \begin{pmatrix} 0 \\ \vdots \\ 0 \\ 1 \\ 0 \\ \vdots \\ 0 \end{pmatrix}$$

Prediction: multidimensional outputs…

Linear models

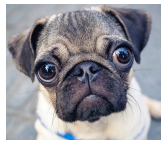
$$z = w^T x, \quad x, w \in \mathbb{R}^d, \quad z \in \mathbb{R}$$

Multi-dimensional linear models

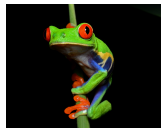
$$z = Wx, \quad x \in \mathbb{R}^d, \quad W \in \mathbb{R}^{K \times d}, \quad z \in \mathbb{R}^K$$

Example: K = 3

z_1: “cat-like”



z_2: “dog-like”



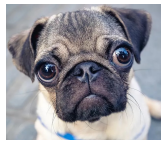
z_3: “frog-like”

Interpreting the output probabilistically

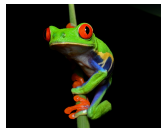
logits: $z = Wx, \quad z \in \mathbb{R}^K$

logits

z_1: “cat-like”



z_2: “dog-like”



z_3: “frog-like”

Interpreting the output probabilistically

logits: $z = Wx, \quad z \in \mathbb{R}^K$

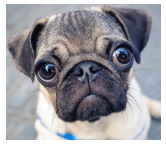
What conditions do I need for p_k to be a probability distribution?

✓

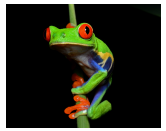
✓

logits

z_1: “cat-like”



z_2: “dog-like”



z_3: “frog-like”

Interpreting the output probabilistically

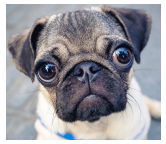
logits: $z = Wx, \quad z \in \mathbb{R}^K$

What conditions do I need for p_k to be a probability distribution?

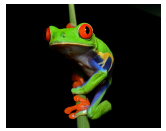
- ✓ Positivity $p_i > 0$
- ✓ Sums to unity: $\sum_{i=1}^{K} p_i = 1$

logits

z_1: “cat-like”



z_2: “dog-like”



z_3: “frog-like”

Interpreting the output probabilistically

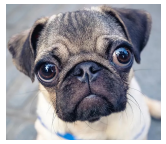
logits: $z = Wx, \quad z \in \mathbb{R}^K$

✓ Positivity $p_i > 0$

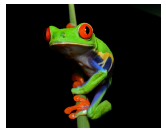
✓ Sums to unity: $\sum_{i=1}^{K} p_i = 1$

logits

3.2



5.1



-1.7

Interpreting the output probabilistically

logits: $z = Wx, \quad z \in \mathbb{R}^K$

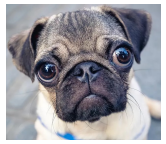
✓ Positivity $p_i > 0$

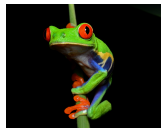
✓ Sums to unity: $\sum_{i=1}^{K} p_i = 1$

Q: What are some functions that are always positive?

logits


3.2


5.1 ⟶


-1.7

Interpreting the output probabilistically

logits: $z = Wx, \quad z \in \mathbb{R}^K$

✓ Positivity $p_i > 0$

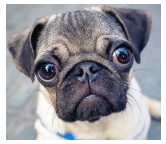
✓ Sums to unity: $\sum_{i=1}^{K} p_i = 1$

Q: What are some functions that are always positive?

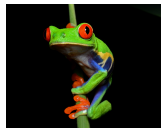
Type your A in chat!

logits

3.2



5.1 ⟶



-1.7

x^2 $\exp$ σ ReLU $|x|$

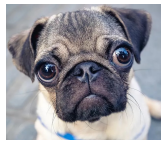
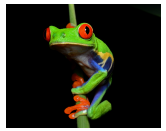
Interpreting the output probabilistically

logits: $z = Wx, \quad z \in \mathbb{R}^K$

✓ Positivity $p_i > 0$

✓ Sums to unity: $\sum_{i=1}^{K} p_i = 1$

Q: What are some functions that are always positive?

logits		Unnormalized probabilities
3.2		24.5
5.1	exp →	164.0
-1.7		0.18

x^2

σ

ReLU

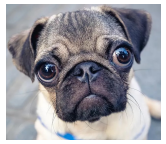
$|x|$

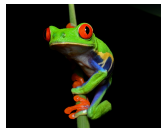
Interpreting the output probabilistically

logits: $z = Wx, \quad z \in \mathbb{R}^K$

✓ Positivity $p_i > 0$

✓ Sums to unity: $\sum_{i=1}^{K} p_i = 1$





	logits		Unnormalized probabilities
	3.2		24.5
	5.1	exp →	164.0
	-1.7		0.18

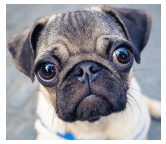
Interpreting the output probabilistically

logits: $z = Wx, \quad z \in \mathbb{R}^K$

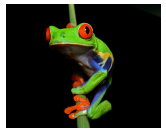
✓ Positivity $p_i > 0$

✓ Sums to unity: $\sum_{i=1}^{K} p_i = 1$

3.2



5.1



-1.7

logits → (exp) → Unnormalized probabilities

24.5

164.0

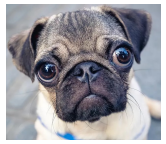
0.18

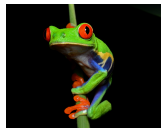
Interpreting the output probabilistically

logits: $z = Wx, \quad z \in \mathbb{R}^K$

✓ Positivity $p_i > 0$

✓ Sums to unity: $\sum_{i=1}^{K} p_i = 1$





	logits		Unnormalized probabilities		Class probabilities
	3.2		24.5		0.13
	5.1	exp →	164.0	normalize →	0.87
	-1.7		0.18		0.00

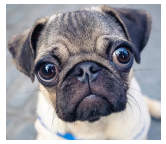
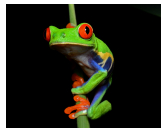
Interpreting the output probabilistically

logits: $z = Wx, \quad z \in \mathbb{R}^K$

Softmax function:

$$p_i = \frac{\exp(z_i)}{\sum_{i=1}^{K} \exp(z_i)}$$

- ✓ Positivity $p_i > 0$
- ✓ Sums to unity: $\sum_{i=1}^{K} p_i = 1$

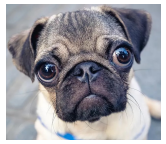
	logits		Unnormalized probabilities		Class probabilities
	3.2		24.5		0.13
	5.1	exp →	164.0	normalize →	0.87
	-1.7		0.18		0.00

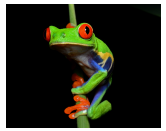
Loss function: Cross entropy

logits: $z = Wx, \quad z \in \mathbb{R}^K$

Softmax function:

$$p_i = \frac{\exp(z_i)}{\sum_{i=1}^{K} \exp(z_i)}$$





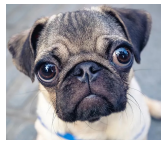
	logits		Class probabilities
	3.2		0.13
	5.1	Softmax →	0.87
	-1.7		0.00

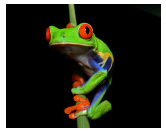
Loss function: Cross entropy

logits: $z = Wx, \quad z \in \mathbb{R}^K$

Softmax function:

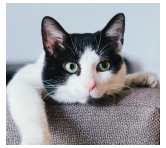
$$p_i = \frac{\exp(z_i)}{\sum_{i=1}^{K} \exp(z_i)}$$





	logits		Class probabilities	Target
	3.2		0.13	1
	5.1	Softmax →	0.87	0 = y
	-1.7		0.00	0

Correct probabilities

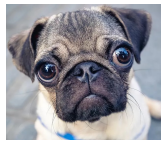
 = x

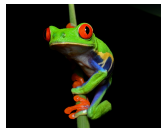
Loss function: Cross entropy

logits: $z = Wx, \quad z \in \mathbb{R}^K$

Softmax function:

$$p_i = \frac{\exp(z_i)}{\sum_{i=1}^{K} \exp(z_i)}$$





logits

3.2

5.1

-1.7

Softmax

Class probabilities

0.13

0.87

0.00

Compare probability distributions

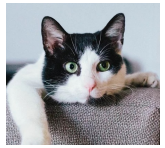
Target

1

0 = y

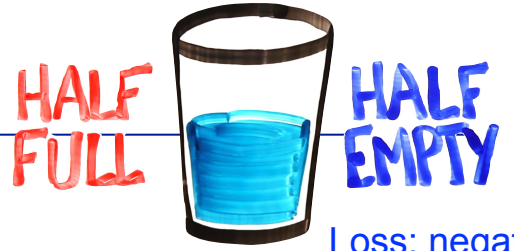
0

Correct probabilities



= x

Loss function: Cross entropy

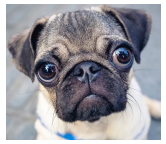


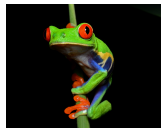
Loss: negative log likelihood

logits: $z = Wx, \quad z \in \mathbb{R}^K$

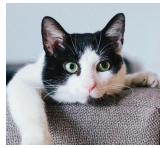
Softmax function:

$$p_i = \frac{\exp(z_i)}{\sum_{i=1}^{K} \exp(z_i)}$$





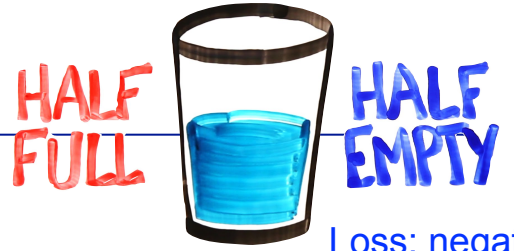
	logits		Class probabilities		Target
	3.2		0.13		1
Softmax →	5.1	Compare probability distributions	0.87		0 = y
	-1.7		0.00		0



= x

Correct probabilities

Loss function: Cross entropy



Loss: negative log likelihood

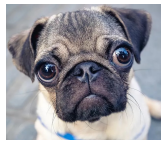
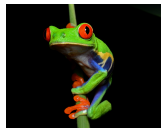
logits: $z = Wx, \quad z \in \mathbb{R}^K$

Softmax function:

$$p_i = \frac{\exp(z_i)}{\sum_{i=1}^{K} \exp(z_i)}$$

Cross entropy loss

$$\mathscr{L} = -\log P(Y = y_i \,|\, X = x)$$
$$= -\log \frac{\exp(z_{y_i})}{\sum_j \exp(z_j)}$$

logits

3.2

5.1

-1.7

Softmax →

Class probabilities

0.13

0.87

0.00

Compare probability distributions ↔

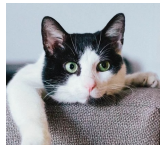
Target

1

0 = y

0

Correct probabilities

 = x

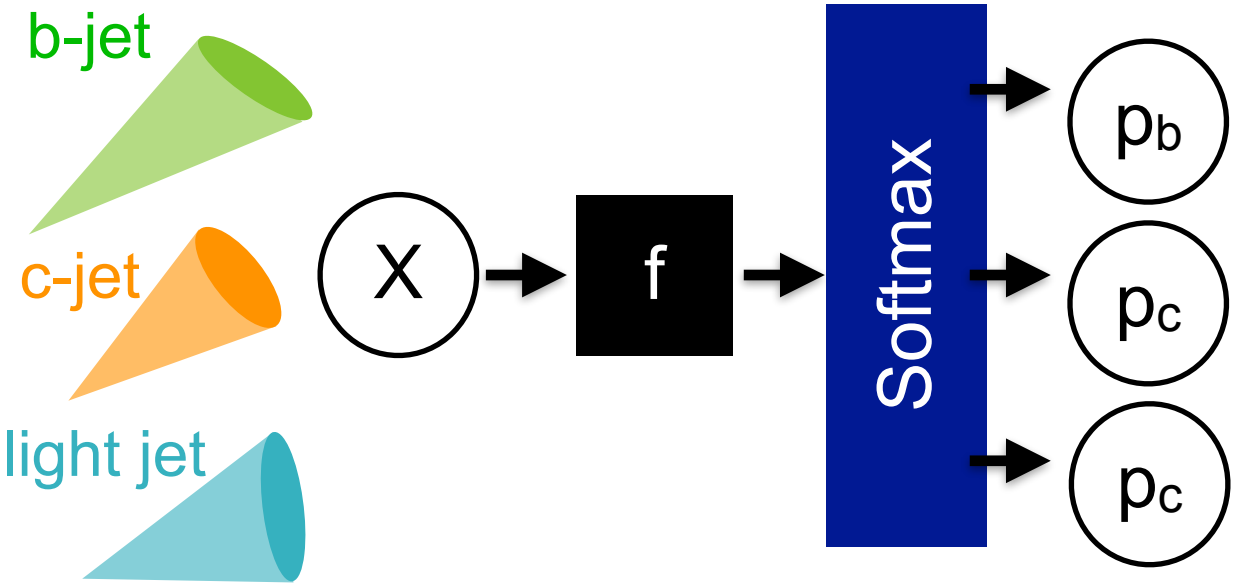
Example: Cross-entropy for particle ID

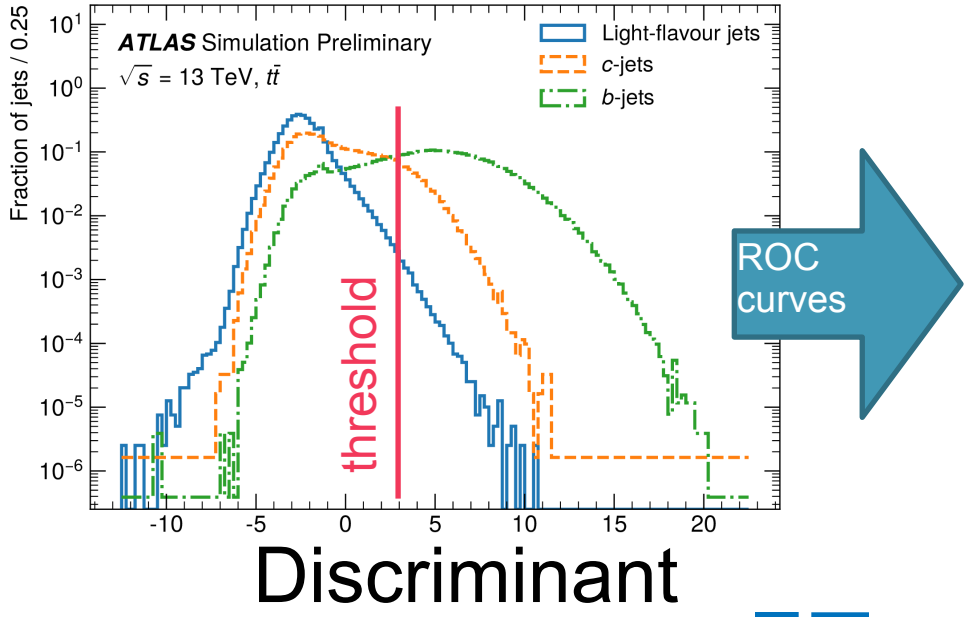
$$y = [\text{b-jet}, \text{c-jet}, \text{light jet}]$$

Softmax function:

$$p_i = \frac{\exp(z_i)}{\sum_{i=1}^{K} \exp(z_i)}$$

$$\text{Discriminant} = \log \frac{p_b}{f_c p_c + (1 - f_c) p_l}$$



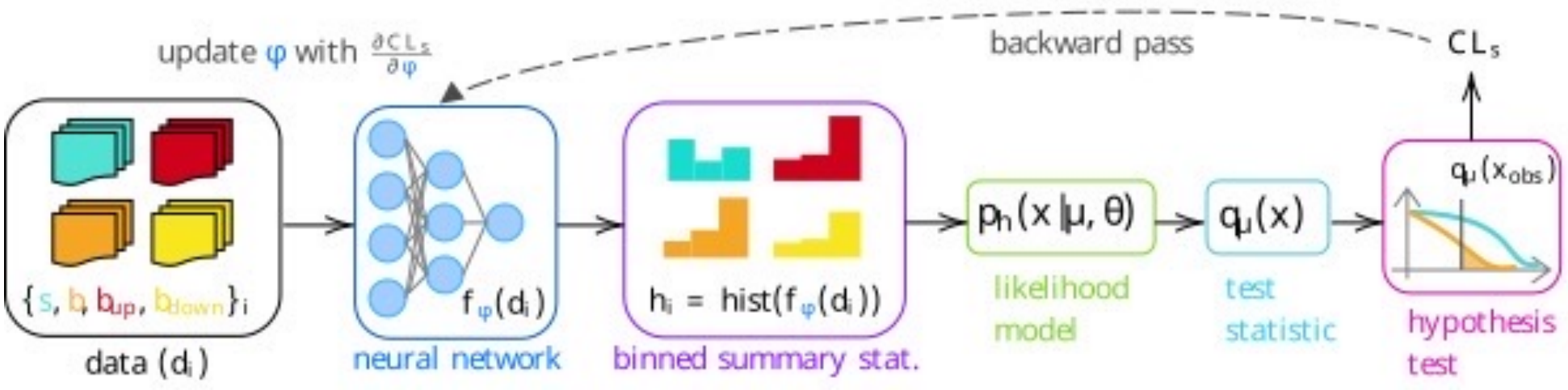


Why is Softmax awesome?

Softmax function:

$$p_i = \frac{\exp(z_i)}{\sum_{i=1}^{K} \exp(z_i)}$$

Use Softmax as a **differentiable histogram** to optimize analysis with systematic uncertainties!!



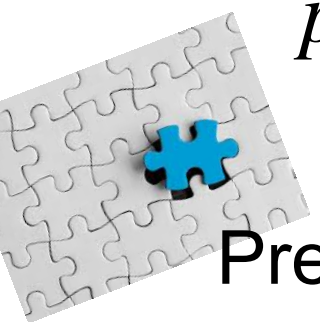
Why is Softmax awesome?

Predict knots for a spline!

Generative model Baran will explain next Thursday!

Softmax function:

$$p_i = \frac{\exp(z_i)}{\sum_{i=1}^{K} \exp(z_i)}$$



Predict parameters of a spline $f_i(z_j)$

Model returns:

Use SOFTMAX to enforce the range of validity!

- widths Δx_k and heights Δy_k
- Derivatives of internal knots